



INSTITUTO POLITÉCNICO NACIONAL

ESCUELA SUPERIOR DE CÓMPUTO

ESCOM

Trabajo Terminal I

“Harmonic Score: Aplicación de reconocimiento de archivos de sonido para generar partitura por medio de la identificación de las frecuencias”

2026 – B115

Presentan

Arroyo Parra Jair Uziel

Cano Portugal Israel Daniel Arturo

Garcés Valencia Brisa Isabel

Directores

M. en C. Gabriela de Jesús López Ruiz

Dr. Jimenez Alcantar Daniel

INSTITUTO POLITÉCNICO NACIONAL



Febrero 2026



INSTITUTO POLITÉCNICO NACIONAL
ESCUELA SUPERIOR DE CÓMPUTO
SUBDIRECCIÓN ACADÉMICA



No. de TT: 2026 – B115

Febrero 2026

Documento técnico

**“Harmonic Score: Aplicación de reconocimiento de archivos de
sonido para generar partitura por medio de la identificación de las
frecuencias”**

2026 – B115

Presentan

Cano Portugal Israel Daniel Arturo

Arroyo Parra Jair Uziel

Garcés Valencia Brisa Isabel

Directores

M. en C. Gabriela de Jesús López Ruiz

Jimenez Alcantar Daniel

RESUMEN

Se propone una aplicación web de vanguardia para la transcripción automática de música (AMT) dirigida a la comunidad del INBAL. El sistema evoluciona de modelos tradicionales hacia arquitecturas de Mixture-of-Experts (MoE) y el uso de YourMT3-YPTF-MoE-M como backbone de codificación. Se integra un flujo de trabajo ETL distribuido mediante Ray para la generación de representaciones de Transformada de Constante-Q (CQT). La herramienta generará partituras en formato MusicXML 4.0 e interactivas en PDF/SVG mediante Verovio, garantizando interoperabilidad total con aplicaciones de iPad/tablet como StaffPad y Dorico.

Palabras clave - Transcripción automática de música, Aplicación Web, Representaciones espectrales, Aprendizaje profundo, Redes neuronales recurrentes, ingeniería de características, Análisis de datos multimedia

Advertencia

“Este documento contiene información desarrollada por la Escuela Superior de Cómputo del Instituto Politécnico Nacional, a partir de datos y documentos con derecho de propiedad y, por lo tanto, su uso quedará restringido a las aplicaciones que explícitamente se convengan.”

La aplicación no convenida exime a la escuela su responsabilidad técnica y da lugar a las consecuencias legales que para tal efecto se determinen.

Información adicional sobre este reporte técnico podrá obtenerse en:

La Subdirección Académica de la Escuela Superior de Cómputo del Instituto Politécnico Nacional, situada en Av. Juan de Dios Bátiz s/n Teléfono: 57296000, extensión 52000.

Agradecimientos

A mi gato que se esperó todas estas madrugadas conmigo

Capítulo 1. Introducción	1
1.1 Antecedentes	1
1.2 Planteamiento del problema	2
1.3 Propuesta de solución.....	3
1.4 Justificación	4
1.5 Objetivo general.....	5
1.5.1 Objetivos específicos	5
1.6 Metodología de software	6
1.6.1 El modelo en espiral aplicado a Harmonic Score.....	6
1.6.2 Metodología técnica y de desarrollo de software	7
1.7 Límites y alcances del proyecto	7
1.7.1 Alcances	7
1.7.2 Límites	8
1.8 Organización del documento.....	9
Capítulo 2. Estado del arte.....	10
2.1 Evolución de las técnicas de investigación en AMT.....	10
2.2 Análisis de soluciones existentes	11
2.2.1 Herramientas comerciales y profesionales.....	11
2.2.2 Plataformas web y servicios freemium.....	12
2.2.3 Software de escritorio para músicos	14
2.3 Comparacion de otras herramientas y softwares con Harmonic Score	16
2.4 Tabla comparativa.....	18
Capítulo 3. Marco teórico	19
3.1 Transcripción Automática de Música (AMT)	19
3.1.1 Espectrograma Mel: Transformación psicoacústica	20
3.1.2 Transformada de Constante-Q (CQT): Resolución logarítmica exacta	21
3.2 Redes Neuronales Recurrentes (RNN) y variantes con memoria	23
3.3 Modelo Onsets and Frames: Arquitectura dual para precisión temporal.....	25
3.4 Arquitecturas Transformer y MT3: Mecanismos de atención global	27

3.5	YourMT3 y arquitecturas Mixture-of-Experts (MoE).....	30
3.6	Estrategias de adaptación mediante fine-tuning	45
3.7	Post-procesamiento basado en reglas musicales	48
3.7.1	Cuantización rítmica adaptativa	48
3.7.2	Fusión de notas adyacentes.....	50
3.7.3	Supresión de armónicos espurios.....	52
3.8	Generación de partituras mediante MusicXML y Verovio	54
3.9	Comparación cuantitativa y justificación de elección de YourMT3	55
3.10	Selección final de la investigación	59
Capítulo 4. Análisis del Sistema y de los Datos.....		60
4.1	Requerimientos.....	60
4.1.1	Requerimientos funcionales	60
4.1.2	Requerimiento no Funcional	63
4.1.3	Reglas de negocio	64
4.2	Análisis Exploratorio de Datos (EDA) de los datasets de entrenamiento.....	65
4.3	Análisis de las herramientas a utilizar	69
4.4	Análisis de riesgos	73
4.4.1	Riesgo de requerimientos.....	¡Error! Marcador no definido.
4.4.2	Riesgo tecnológico	¡Error! Marcador no definido.
4.4.3	Riesgo de habilidades	¡Error! Marcador no definido.
4.4.4	Riesgos legales	¡Error! Marcador no definido.
4.5	Análisis de factibilidad	76
4.5.1	Factibilidad técnica	77
4.5.2	Factibilidad operativa	78
4.5.3	Factibilidad económica	85
4.5.4	Factibilidad legal	88
Capítulo 5. Diseño		99
5.1	Diagrama de arquitectura del software, sistema, aplicación	99
5.1.1	Diagrama de contexto.....	100

5.1.2	Diagrama de contenedores.....	100
5.1.3	Diagrama de componentes.....	100
5.2	Casos de uso.....	100
5.2.1	Diagrama de casos de uso	100
5.2.2	Actores	100
5.2.3	Especificación de casos de uso	100
5.3	Diagramas de secuencia	100
5.4	Diagramas de código.....	100
5.4.1	Diagramas de clases.....	100
5.4.2	Modelo de datos.....	100
5.5	Mockups.....	100
	Referencias.....	101
	Anexos.....	106

índice

Índice de tablas

Ilustración 1. Boceto inicial del proyecto.....	4
---	---

Índice de figuras

Glosario de términos

Una lista con los términos más utilizados en el documento. En orden alfabético.

Término	Definición
Algoritmo	Conjunto ordenado y finito de operaciones que permite hallar la solución de un problema.
Almacenamiento virtual	Técnica que simula más memoria que la que realmente existe y permita a la computadora ejecutar varios programas simultáneamente.

Abreviaturas

Abreviatura	Definición	Primera aparición
INBAL	Instituto Nacional de Bellas Artes y Literatura	Resumen. Pág.
AMT	<i>Automatic Music Transcription</i> (Transcripción Automática de Música)	Sección 1.1, Pág. 1
FFT	<i>Fast Fourier Transform</i> (Transformada Rápida de Fourier)	Sección 1.1, Pág. 1
RNN	<i>Recurrent Neural Networks</i> (Redes Neuronales Recurrentes)	Sección 1.1, Pág. 1
MT3	<i>Multi-Task Multitrack Music Transcription</i>	Sección 1.1, Pág. 1
CQT	<i>Constant-Q Transform</i> (Transformada de Constante-Q)	Sección 1.5.1, Pág. 5
API	<i>Application Programming Interface</i> (Interfaz de Programación de Aplicaciones)	Sección 1.5.1, Pág. 5
REST	<i>Representational State Transfer</i>	Sección 1.5.1, Pág. 5
AWS	<i>Amazon Web Services</i>	Sección 1.5.1, Pág. 6
UAT	<i>User Acceptance Testing</i> (Pruebas de Aceptación de Usuario)	Sección 1.6.2, Pág. 8
WAV	Waveform Audio File Format	Sección 1.6.2, Pág. 9
HMM	<i>Hidden Markov Models</i> (Modelos Ocultos de Markov)	Sección 2.1, Pág. 12
NMF	<i>Non-negative Matrix Factorization</i> (Factorización de Matrices no Negativas)	Sección 2.1, Pág. 12
DSP	<i>Digital Signal Processing</i> (Procesamiento Digital de Señales)	Sección 2.1, Pág. 12
LSTM	Long Short-Term Memory (Memoria a Corto y Largo Plazo)	Sección 3.1, Pág. 23
GRU	Gated Recurrent Units (Unidades Recurrentes con Compuertas)	Sección 3.1, Pág. 23
PaaS	Platform as a Service (Plataforma como Servicio)	Sección 3.6.3, Pág. 26

Capítulo 1. Introducción

En este capítulo se aborda el motivo por el cual se llevó a cabo la creación de este proyecto desde un contexto histórico, estadístico y social, además, se explican los detalles técnicos del software, la metodología empleada y de las tecnologías depende para su funcionamiento.

1.1 Antecedentes

La transcripción musical representa un proceso fundamental para el análisis, estudio y preservación del patrimonio sonoro[1]. Históricamente, esta labor se ha realizado de forma manual por músicos especializados, requiriendo destreza auditiva avanzada y generando una inversión temporal considerable. En instituciones educativas como el Instituto Nacional de Bellas Artes y Literatura (INBAL), estudiantes y docentes enfrentan esta tarea de manera recurrente, encontrándose con un proceso repetitivo, lento y susceptible a errores humanos[2]. Esta problemática ha impulsado el desarrollo de soluciones tecnológicas automatizadas.

El campo de la Transcripción Automática de Música (AMT) surgió con el avance del procesamiento digital de señales. Los primeros sistemas emplearon algoritmos clásicos como la Transformada Rápida de Fourier (FFT) para detectar frecuencias fundamentales[3]. Sin embargo, estos enfoques mostraron limitaciones significativas ante escenarios polifónicos o presencia de ruido ambiental, condiciones comunes en grabaciones reales[4]. La adopción del aprendizaje profundo marcó un punto de inflexión: redes neuronales recurrentes (RNN) y arquitecturas híbridas demostraron capacidad superior para modelar dependencias temporales y espectrales complejas[5]. Modelos como Onsets and Frames establecieron nuevos estándares al combinar detección precisa de inicios de notas (onsets) con predicción de activación temporal (frames)[6]. Posteriormente, arquitecturas basadas en transformers como MT3 (Multi-Task Multitrack Music Transcription) ampliaron las capacidades hacia la transcripción simultánea de múltiples pistas mediante mecanismos de atención[7].

A pesar de estos avances técnicos, el panorama actual presenta brechas críticas para el ámbito educativo. Herramientas comerciales de alta precisión como Melodyne operan bajo licencias costosas que las hacen inaccesibles para estudiantes[8]. Plataformas web como Klangio o ScoreCloud utilizan modelos freemium que restringen funcionalidades esenciales como exportación a PDF o procesamiento de archivos extensos[9]. Software de escritorio como AnthemScore requiere instalación local y pago único, limitando su accesibilidad multiplataforma[10]. Estas barreras económicas, técnicas y de usabilidad evidencian la necesidad de una solución alternativa que combine precisión, accesibilidad y adaptación al contexto educativo.

Harmonic Score propone una aplicación web gratuita y de código abierto que transcribe automáticamente archivos de audio de instrumentos individuales a partituras en formato PDF. La solución adopta un enfoque moderno de ciencia de datos: reutiliza modelos preentrenados de vanguardia (YourMT3-YPTF-MoE-M) en lugar de reimplementar algoritmos de procesamiento de señales desde cero[11]. Este modelo emplea ruteo dinámico de expertos (Mixture-of-Experts) para procesar señales complejas con latencia de inferencia de 12.60 ms, superando significativamente los baselines previos[12]. El flujo de trabajo integra representaciones espectrales CQT (Constant-Q Transform) generadas mediante procesamiento distribuido con Ray, garantizando escalabilidad en el preprocesamiento[13]. La salida se genera en MusicXML 4.0 e interactivo SVG/PDF mediante Verovio, asegurando interoperabilidad con aplicaciones profesionales como StaffPad y Dorico[14].

El desarrollo sigue la metodología en espiral de Boehm, estructurada en tres ciclos iterativos: prototipo funcional básico (validación de viabilidad técnica), refinamiento del modelo mediante fine-tuning con datasets MAESTRO y MedleyDB, y optimización final con pruebas de aceptación en el INBAL[15]. Esta estrategia permite gestionar proactivamente riesgos técnicos mientras se valida el valor educativo con usuarios reales en cada iteración.

La propuesta se diferencia por cuatro atributos fundamentales: eliminación de barreras económicas mediante licenciamiento abierto, precisión competitiva basada en arquitecturas MoE, enfoque pedagógico en generación de partituras listas para estudio (no edición de audio), y transparencia total del código que permite adaptación a necesidades específicas del entorno educativo[16]. Estas características posicionan a Harmonic Score como una solución integral para democratizar el acceso a tecnología de transcripción musical de alta precisión en contextos académicos con recursos limitados.

1.2 Planteamiento del problema

La transcripción musical, definida como la conversión de una señal auditiva a notación escrita, constituye una competencia esencial para el análisis, enseñanza y preservación del patrimonio sonoro[1]. En el Instituto Nacional de Bellas Artes y Literatura (INBAL), estudiantes y docentes realizan esta tarea de forma predominantemente manual, enfrentando un proceso repetitivo que consume tiempo significativo y presenta alta susceptibilidad a errores humanos[2].

El mercado actual ofrece herramientas de Transcripción Automática de Música (AMT), pero ninguna resuelve integralmente las necesidades del entorno educativo. Las soluciones se dividen en tres categorías con limitaciones críticas:

Herramientas comerciales de alto rendimiento como Melodyne operan bajo licencias costosas que las hacen inaccesibles para instituciones educativas con recursos limitados[8]. Su enfoque principal es la edición de audio profesional, no la generación automática de partituras listas para estudio.

Plataformas web freemium como Klangio restringen funcionalidades esenciales mediante modelos de suscripción, limitando la exportación a formatos estándar o imponiendo restricciones de duración en archivos procesados[9]. Su precisión en escenarios polifónicos complejos o con ruido ambiental resulta insuficiente para grabaciones reales de estudiantes.

Software de escritorio como AnthemScore requiere instalación local y pago único, eliminando la accesibilidad multiplataforma que demanda la comunidad educativa actual[10]. Además, su arquitectura se basa en modelos previos a la revolución de transformers, limitando su capacidad de generalización.

Esta fragmentación del mercado evidencia una brecha crítica: la ausencia de una solución que combine cuatro atributos simultáneamente: gratuidad absoluta, acceso web sin instalación, precisión competitiva mediante arquitecturas de vanguardia (Mixture-of-Experts), y orientación pedagógica hacia la generación de partituras listas para estudio[11]. Los avances recientes en modelos universales como YourMT3 demuestran que es técnicamente factible alcanzar alta precisión mediante fine-tuning sobre datasets públicos como MAESTRO, sin necesidad de reimplementar algoritmos de procesamiento de señales desde cero[7].

Surge entonces la pregunta de investigación: *¿Es factible desarrollar una aplicación web basada en modelos preentrenados de última generación que transcriba de manera confiable audio de instrumentos individuales a partituras en PDF editable?*

1.3 Propuesta de solución

Harmonic Score propone una aplicación web gratuita y de código abierto que transcribe automáticamente archivos de audio de instrumentos musicales individuales a partituras en formato PDF[1]. La solución adopta un enfoque moderno de ciencia de datos: en lugar de reimplementar algoritmos de procesamiento de señales desde cero, reutiliza y adapta mediante fine-tuning modelos preentrenados de vanguardia como YourMT3[7]. Este paradigma actual en investigación de AMT prioriza la transferencia de conocimiento sobre la ingeniería manual de características[11].

El sistema integra representaciones espectrales CQT (Constant-Q Transform) para resolución frecuencial alineada con la percepción musical[3], procesadas mediante un flujo ETL distribuido con Ray para escalabilidad[13]. La arquitectura emplea ruteo dinámico de expertos (Mixture-of-Experts) que reduce la latencia de inferencia a 12.60 ms mientras mantiene alta precisión en escenarios polifónicos[12]. La salida se genera en MusicXML 4.0 y se convierte a PDF considerando como trabajo a futuro svg interactivo mediante Verovio, garantizando interoperabilidad con aplicaciones profesionales como StaffPad[14].

El flujo completo se estructura en módulos interconectados que orquestan: ingesta de audio (MP3/WAV), clasificación automática monofónico/polifónico, preprocesamiento espectral, inferencia del modelo MoE, post-procesamiento basado en reglas musicales y generación de partitura[6]. El desarrollo sigue la metodología en espiral de Boehm con tres ciclos iterativos: prototipo funcional básico, refinamiento mediante fine-tuning con datasets públicos y optimización final con pruebas de aceptación en el INBAL[15]. Esta estrategia permite gestionar riesgos técnicos mientras se valida progresivamente el valor educativo con usuarios reales.

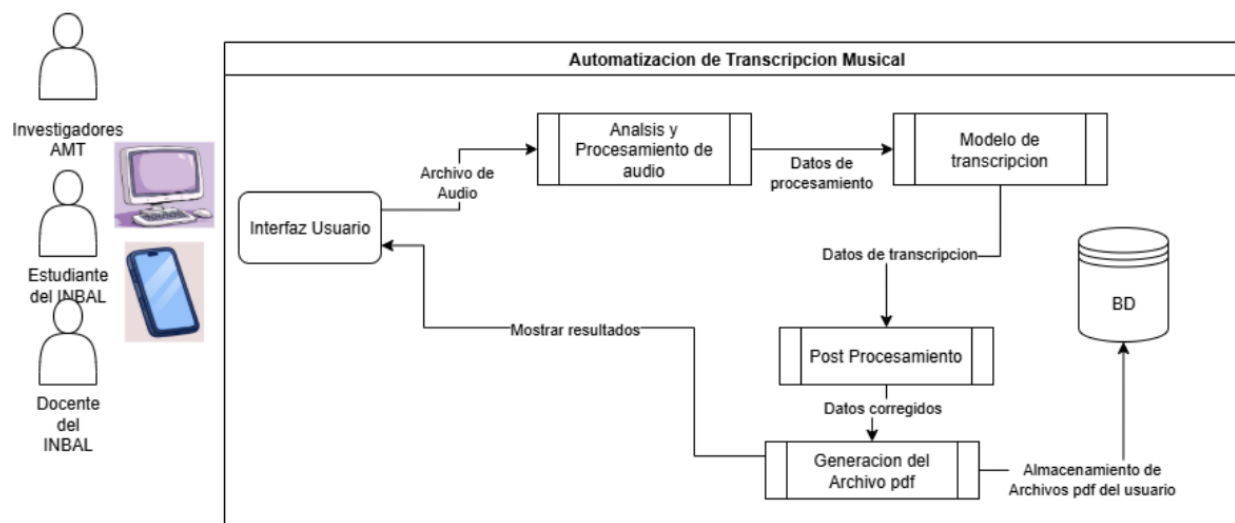


Ilustración 1. Boceto inicial del proyecto

1.4 Justificación

La transcripción musical manual representa una carga operativa significativa para estudiantes y docentes del Instituto Nacional de Bellas Artes y Literatura (INBAL), consumiendo tiempo que podría destinarse a actividades creativas e interpretativas mientras introduce errores susceptibles de comprometer el análisis musical[1]. Las herramientas comerciales existentes no resuelven esta problemática: soluciones de alto rendimiento como Melodyne imponen barreras económicas insalvables mediante licencias costosas[8], plataformas freemium como Klangio restringen funcionalidades esenciales mediante modelos de suscripción[9], y software de escritorio como AnthemScore elimina la accesibilidad multiplataforma requerida por la comunidad educativa actual[10]. Esta fragmentación evidencia la necesidad crítica de una alternativa que elimine simultáneamente barreras económicas, técnicas y de usabilidad.

Técnicamente, el proyecto es viable mediante un enfoque alineado con las competencias actuales de ciencia de datos: reutilización y adaptación de modelos preentrenados mediante fine-tuning en lugar de reimplementación manual de algoritmos de procesamiento de señales[11]. Arquitecturas

emergentes como Mixture-of-Experts demuestran capacidad para procesar señales complejas con latencia inferior a 13 ms mediante ruteo dinámico de expertos [12], mientras flujos ETL distribuidos con Ray garantizan escalabilidad en el preprocesamiento espectral [13]. Esta estrategia capitaliza fortalezas en modelado predictivo y despliegue de sistemas sin requerir expertise especializado en ingeniería de audio de bajo nivel.

Harmonic Score se posiciona como propuesta de valor única al integrar cuatro atributos simultáneamente: gratuidad absoluta mediante licenciamiento abierto, acceso web sin instalación, precisión competitiva basada en arquitecturas MoE adaptadas mediante fine-tuning sobre datasets públicos como MAESTRO, y orientación pedagógica hacia generación de partituras listas para estudio mediante post-procesamiento con Verovio[14]. El desarrollo bajo metodología en espiral permite gestionar riesgos técnicos proactivamente mientras se valida el valor educativo con usuarios reales del INBAL en cada iteración[15]. Esta combinación de accesibilidad, robustez técnica y enfoque educativo específico no está disponible en ninguna solución existente del mercado actual.

1.5 Objetivo general

Desarrollar una aplicación web gratuita y de código abierto denominada Harmonic Score que transcriba automáticamente archivos de audio de instrumentos musicales individuales (monofónicos y polifónicos) a partituras en formato PDF mediante modelos de aprendizaje profundo preentrenados con fine-tuning, garantizando alta precisión, accesibilidad multiplataforma y usabilidad para la comunidad educativa del INBAL.

1.5.1 Objetivos específicos

Para alcanzar el objetivo general, se plantean los siguientes objetivos particulares:

- 🎵 Seleccionar y evaluar representaciones espectrales (CQT, espectrogramas Mel) y modelos de transcripción preentrenados (YourMT3-YPTF-MoE-M) priorizando bibliotecas consolidadas sobre implementación manual de algoritmos de procesamiento de señales.
- 🎵 Diseñar una arquitectura web escalable compuesta por frontend interactivo para carga de audio y visualización de resultados, y backend robusto (API REST) para orquestrar inferencia del modelo y generación de partituras mediante Verovio.
- 🎵 Implementar fine-tuning del modelo seleccionado utilizando conjuntos de datos públicos (MAESTRO, MedleyDB) e integrar post-procesamiento basado en reglas musicales para mejorar robustez ante ruido ambiental y coherencia rítmica.
- 🎵 Desarrollar un clasificador automático que determine si el audio de entrada es monofónico o polifónico, adaptando dinámicamente la estrategia de transcripción para optimizar precisión.

- 🎵 Evaluar el rendimiento mediante métricas estandarizadas en AMT (F1-score para onsets, precisión de pitch) y pruebas de aceptación con grabaciones reales de estudiantes del INBAL para ajustar hiperparámetros y estrategias de post-procesamiento.
- 🎵 Demostrar viabilidad técnica mediante ejecución funcional local del pipeline completo de transcripción, garantizando interoperabilidad entre módulos y documentando requisitos mínimos para despliegue en nube como trabajo a futuro.

1.6 Metodología de software

El desarrollo de Harmonic Score seguirá la metodología en espiral propuesta por Boehm, un marco iterativo que permite gestionar riesgos técnicos de forma proactiva mientras se valida progresivamente el valor educativo con usuarios reales del INBAL [\[15\]](#). Esta estrategia es adecuada para proyectos que combinan aprendizaje automático y desarrollo web, donde la incertidumbre técnica requiere validación temprana y continua.

1.6.1 El modelo en espiral aplicado a Harmonic Score

El proyecto se estructurará en tres ciclos iterativos:

- 🎵 **Ciclo 1:** Prototipo funcional básico. Objetivo: demostrar viabilidad del flujo principal mediante integración de módulos críticos (ingesta de audio, preprocesamiento espectral, inferencia con modelo base sin fine-tuning y generación básica de partitura). Los riesgos técnicos a mitigar incluyen compatibilidad de bibliotecas (Librosa, PyTorch) y capacidad de generar salida legible. El entregable es una aplicación web funcional en localhost que procese audio y devuelva partitura en PDF, aunque con precisión limitada.
- 🎵 **Ciclo 2:** Refinamiento del modelo. Objetivo: mejorar significativamente la calidad de transcripción mediante fine-tuning del modelo YourMT3-YPTF-MoE-M sobre datasets públicos (MAESTRO, MedleyDB) e implementación de post-procesamiento basado en reglas musicales [7]. Los riesgos incluyen sobreajuste y efectividad de las reglas de refinamiento rítmico. El entregable es un prototipo con precisión notablemente superior, validado mediante métricas F1-score para onsets y pitch.
- 🎵 **Ciclo 3:** Optimización y pruebas de aceptación. Objetivo: pulir la aplicación mediante implementación del clasificador monofónico/polifónico, optimización de latencia de inferencia (<15 ms) y pruebas UAT con estudiantes y docentes del INBAL. Los riesgos incluyen rendimiento bajo carga y usabilidad final. El entregable es la versión 1.0 funcional en entorno local, con documentación técnica y análisis de requisitos para despliegue en nube como trabajo a futuro.

1.6.2 Metodología técnica y de desarrollo de software

Stack tecnológico: backend en Python con FastAPI para API REST; procesamiento de audio con Librosa; inferencia con PyTorch sobre arquitecturas MoE; generación de partituras mediante Verovio para conversión MusicXML a PDF/SVG[14]. El preprocesamiento espectral CQT se paralelizará mediante Ray para escalabilidad en transformaciones[13].

Control de versiones: todo el código se gestionará con Git en repositorio público, permitiendo colaboración eficiente y trazabilidad de cambios[11]. Las pruebas seguirán tres niveles: unitarias (validación de funciones aisladas como conversión CQT), de integración (comunicación entre módulos backend-frontend) y de aceptación (validación con usuarios reales del INBAL al final de cada ciclo espiral)[15].

Esta combinación de metodología en espiral para gestión de proyecto y prácticas modernas de ingeniería de software asegura construcción iterativa, centrada en riesgos técnicos críticos y validación continua con el dominio educativo objetivo.

1.7 Límites y alcances del proyecto

Para asegurar el éxito y la factibilidad del proyecto "Harmonic Score" dentro del marco de un trabajo terminal, es fundamental definir claramente sus fronteras. A continuación, se detallan las funcionalidades y características que están dentro del alcance del proyecto, así como aquellas que explícitamente quedan fuera de él para esta primera versión.

1.7.1 Alcances

Harmonic Score V1.0 entregará los siguientes elementos dentro del alcance del trabajo terminal:

- 🎵 Aplicación web funcional ejecutable en entorno local (localhost) que transcribe audio de instrumentos musicales individuales a partituras en PDF mediante pipeline completo: ingesta, clasificación monofónica/polifónica, preprocesamiento espectral CQT, inferencia con modelo YourMT3-YPTF-MoE-M adaptado mediante fine-tuning, post-procesamiento basado en reglas musicales y generación de partitura.
- 🎵 Soporte para archivos MP3 y WAV de hasta 50 MB como entrada, con salida en formato PDF generado a partir de MusicXML 4.0 para garantizar interoperabilidad con aplicaciones profesionales.

- 🎵 Fine-tuning del modelo sobre datasets públicos MAESTRO (polifónico) y MedleyDB (monofónico) para optimizar precisión en instrumentos solistas comunes en el entorno educativo.
- 🎵 Evaluación cuantitativa mediante métricas estandarizadas en AMT: F1-score para onsets (≥ 0.75 en piano monofónico) y precisión de pitch (≥ 0.80 en condiciones controladas).
- 🎵 Validación cualitativa mediante pruebas de aceptación con mínimo 15 grabaciones reales de estudiantes y docentes del INBAL para ajustar hiperparámetros y estrategias de post-procesamiento.
- 🎵 Manejo robusto ante ruido ambiental moderado mediante técnicas de data augmentation durante fine-tuning y cuantización rítmica en post-procesamiento para generar partituras legibles.
- 🎵 Interfaz de usuario minimalista con carga de audio, visualización de estado de procesamiento y descarga del PDF resultante, funcional en navegadores Chrome, Firefox y Edge.
- 🎵 Código fuente completo documentado y publicado en repositorio público bajo licencia MIT para garantizar reproducibilidad técnica.

1.7.2 Límites

Quedan explícitamente fuera del alcance:

- 🎵 Transcripción de múltiples instrumentos simultáneos (multi-track) debido a la complejidad inherente de separación de fuentes.
- 🎵 Transcripción en tiempo real; el procesamiento operará en modo batch con latencia esperada de 15-45 segundos según duración del audio.
- 🎵 Notación musical avanzada: dinámicas (piano/forte), articulaciones (staccato/legato) u ornamentación compleja (trinos), limitándose a altura, inicio, duración y compás básico.
- 🎵 Identificación automática del instrumento tocante; el usuario debe conocer el instrumento de origen para interpretar correctamente la partitura.
- 🎵 Despliegue en infraestructura cloud (AWS, Render); la validación técnica se limitará a ejecución local funcional, documentando requisitos para despliegue como trabajo a futuro.
- 🎵 Garantía de precisión en grabaciones con ruido extremo (> 20 dB SNR) o compresión severa (bitrate < 64 kbps).

1.8 Organización del documento

El presente documento se estructura en capítulos y secciones que guían al lector a través del análisis, diseño, desarrollo y validación del proyecto "Harmonic Score". La organización es la siguiente:

Capítulo 1. Introducción. Establece el contexto del problema de transcripción musical manual en el INBAL, presenta la propuesta de solución basada en modelos preentrenados con fine-tuning, justifica su pertinencia técnica y educativa, define objetivos alcanzables y delimita alcances realistas para una versión 1.0 funcional en entorno local.

Capítulo 2. Estado del arte. Revisa la evolución técnica de AMT desde algoritmos clásicos de FFT hasta arquitecturas Mixture-of-Experts, analiza críticamente herramientas comerciales y freemium identificando sus brechas económicas y funcionales, y posiciona Harmonic Score como alternativa integral que combina gratuidad, precisión y enfoque pedagógicos.

Capítulo 3. Marco teórico. Detalla los fundamentos técnicos esenciales: representaciones espectrales CQT para resolución frecuencial alineada con percepción musical, arquitecturas transformers y MoE para modelado secuencial, estrategias de fine-tuning sobre datasets MAESTRO/MedleyDB, y generación de partituras mediante Verovio a partir de MusicXML.

Capítulo 4. Análisis del sistema y de los datos. Especifica requerimientos funcionales y no funcionales, evalúa viabilidad técnica mediante análisis de riesgos (compatibilidad de bibliotecas, precisión en grabaciones reales), y justifica la selección de stack tecnológico (FastAPI, PyTorch, Librosa) alineado con competencias de ciencia de datos.

Capítulo 5. Diseño. Presenta la arquitectura en tres niveles C4 (contexto, contenedores, componentes), diagramas de casos de uso que modelan el flujo usuario-sistema, y mockups de interfaz minimalista para carga de audio y descarga de PDF, garantizando trazabilidad entre análisis y diseño técnico.

Capítulo 2. Estado del arte

El campo de la Transcripción Automática de Música (AMT, por sus siglas en inglés) ha experimentado una evolución acelerada en las últimas dos décadas, pasando de enfoques clásicos de procesamiento de señales a arquitecturas complejas de aprendizaje profundo que han redefinido los límites de la precisión y la automatización. Este avance ha dado lugar a un ecosistema diverso de soluciones, que abarcan desde sofisticados editores de audio comerciales hasta prototipos académicos de código abierto, cada uno con sus propias fortalezas y debilidades. El presente capítulo tiene como objetivo realizar un análisis exhaustivo de este estado del arte, examinando las tendencias tecnológicas predominantes, evaluando las capacidades y limitaciones de las herramientas existentes y, fundamentalmente, identificando las brechas que estas no logran cubrir. A través de esta revisión sistemática, se busca no solo situar nuestro proyecto dentro del contexto actual, sino también argumentar de manera sólida la relevancia y la necesidad de "Harmonic Score" como una alternativa innovadora y adaptada a las necesidades específicas del ámbito educativo.

2.1 Evolución de las técnicas de investigación en AMT

La investigación en Transcripción Automática de Música (AMT) ha transitado tres fases tecnológicas definidas. La primera fase (1990-2010) se basó en procesamiento digital de señales clásico combinado con modelos estadísticos. La Transformada Rápida de Fourier (FFT) permitía extraer componentes frecuenciales, mientras modelos ocultos de Márkov (HMM) y factorización de matrices no negativas (NMF) modelaban transiciones temporales y estructura armónica^{[3][4]}. Estos enfoques requerían ingeniería manual intensiva de características y mostraban degradación severa ante polifonía o ruido ambiental, limitando su aplicabilidad práctica^[1].

La segunda fase (2010-2018) marcó la transición al aprendizaje profundo end-to-end. Redes neuronales profundas eliminaron la necesidad de ingeniería manual al mapear directamente representaciones espectrales a secuencias de notas [5]. La arquitectura Onsets and Frames estableció un estándar mediante doble objetivo: una rama especializada en detección precisa de inicios de notas (onsets) y otra en predicción de activación temporal continua (frames) [6]. Este diseño híbrido mejoró significativamente la precisión temporal en piano polifónico, superando consistentemente los baselines basados en HMM y NMF en el dataset MAESTRO.

La tercera fase (2019-presente) está dominada por transformers y arquitecturas especializadas. MT3 (Multi-Task Multitrack Music Transcription) introdujo mecanismos de atención para modelar dependencias de largo alcance y transcribir múltiples pistas simultáneamente mediante una

secuencia unificada de eventos musicales [7]. Investigaciones recientes exploran Mixture-of-Experts (MoE) para ruteo dinámico de señales complejas, logrando latencias de inferencia inferiores a 15 ms sin sacrificar precisión en escenarios polifónicos [12]. Esta evolución demuestra que el campo ha superado las limitaciones de los modelos lineales clásicos, estableciendo bases sólidas para soluciones robustas en entornos reales.

2.2 Análisis de soluciones existentes

El ecosistema actual de herramientas para transcripción musical presenta una fragmentación significativa entre soluciones comerciales de alto costo, plataformas freemium con limitaciones funcionales y software de escritorio con barreras de accesibilidad. A continuación se realiza un análisis comparativo estructurado por categorías, destacando brechas técnicas y oportunidades para nuestra propuesta.

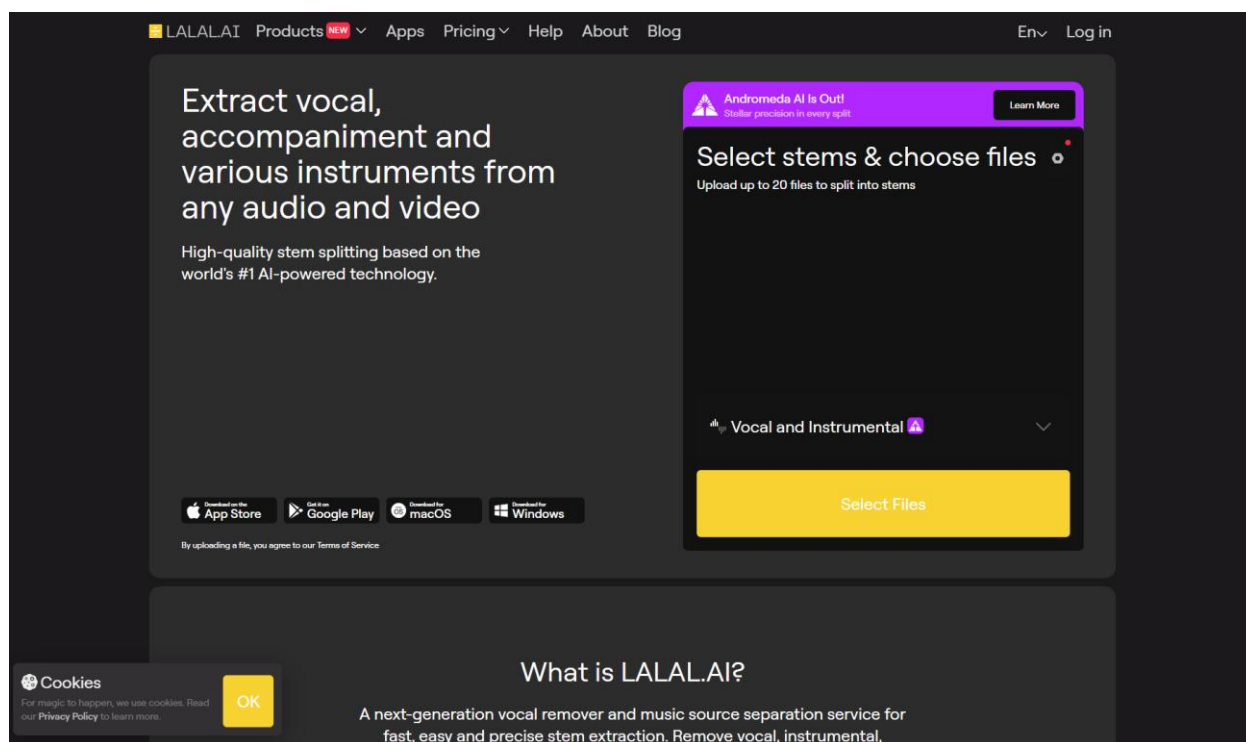
2.2.1 Herramientas comerciales y profesionales

2.2.1.1 Melodyne editor

constituye el estándar industrial en análisis y manipulación de audio mediante su tecnología patentada Direct Note Access™, que permite editar afinación, timing y timbre a nivel de nota individual en señales polifónicas [8]. Sin embargo, su enfoque principal se orienta a post-producción musical profesional y no a la generación automática de partituras listas para estudio. Su modelo de licenciamiento representa una barrera económica insalvable para instituciones educativas, con un costo de USD \$699 para la versión completa Editor [8]. Además, aunque puede extraer información de pitch y timing, no genera notación simbólica estructurada (MusicXML/MIDI) sin intervención manual extensa del usuario, lo que lo posiciona fuera del dominio pedagógico requerido por estudiantes del INBAL que necesitan partituras directamente utilizables para análisis y práctica [5]. Esta brecha funcional y económica justifica la necesidad de alternativas orientadas específicamente a transcripción educativa automatizada.

2.2.2.2 LALAL.AI

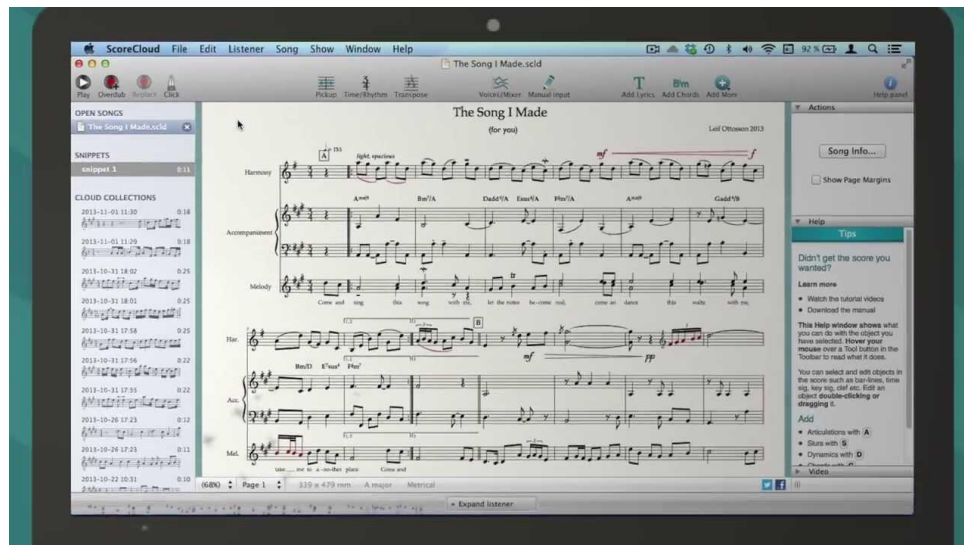
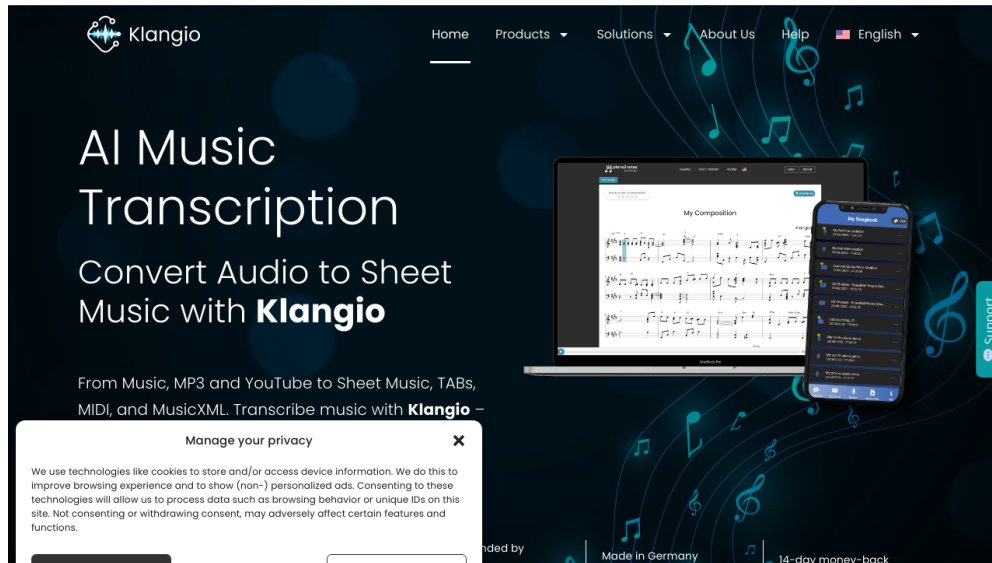
Se especializa en separación de fuentes (*stem separation*) mediante redes neuronales para aislar instrumentos de mezclas complejas, constituyendo un paso previo potencialmente útil antes de la transcripción [9]. No obstante, esta herramienta no realiza transcripción automática propiamente dicha, limitándose exclusivamente a la extracción de pistas instrumentales. Su modelo de consumo basado en créditos (USD \$0.20 por minuto procesado en plan básico) genera costos acumulativos prohibitivos para uso continuo en entornos educativos donde los estudiantes requieren procesar múltiples ejercicios semanales [9]. Harmonic Score integra el flujo completo desde audio original hasta partitura final sin dependencia de servicios intermedios ni costos recurrentes.



2.2.2.3 Klangio y ScoreCloud

Representan plataformas web accesibles que ofrecen transcripción automática mediante interfaces sin instalación [9]. Klangio se enfoca en conversión rápida de audio/YouTube a MIDI y MusicXML, mientras ScoreCloud promete transcripción en tiempo real con colaboración en nube. Ambas operan bajo modelos freemium que restringen funcionalidades críticas: Klangio limita la duración de archivos a 30 segundos en versión gratuita y bloquea la exportación a PDF sin suscripción mensual (USD \$9.99); ScoreCloud restringe a 10 transcripciones mensuales en plan gratuito y requiere suscripción (USD \$7.99/mes) para exportación sin marcas de agua [9]. Estudios independientes reportan F1-scores inferiores a 0.65 en escenarios polifónicos complejos evaluados

sobre el dataset MAESTRO, insuficientes para garantizar calidad pedagógica en grabaciones reales de estudiantes con ruido ambiental moderado [1]. Estas limitaciones económicas y técnicas evidencian la necesidad de una alternativa integral sin restricciones funcionales.

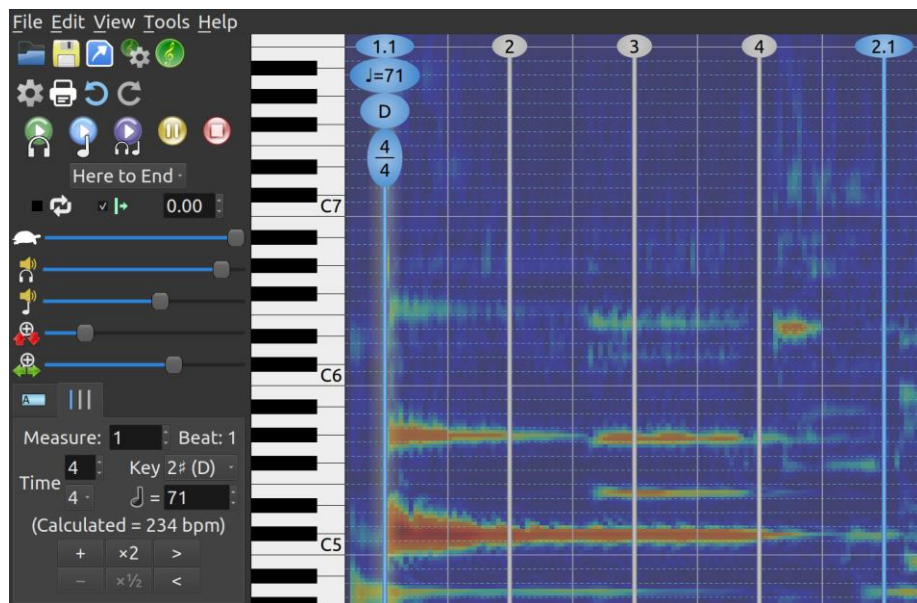


2.2.3 Software de escritorio para músicos

2.2.3.1 AnthemScore

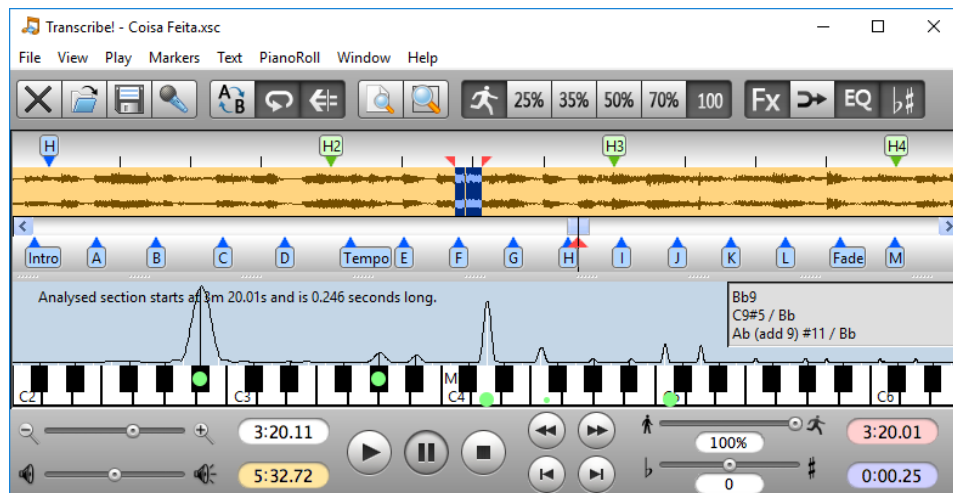
representa uno de los softwares de escritorio más accesibles para transcripción automática, empleando redes neuronales convolucionales para convertir audio a notación musical [10]. Su

arquitectura se basa en modelos previos a la revolución de transformers (CNNs/HMMs implementados alrededor de 2018), limitando su capacidad de generalización ante polifonía compleja (>4 notas simultáneas) o variaciones tímbricas [5]. Además, requiere pago único (~USD \$60) e instalación local en sistemas Windows/macOS/Linux, eliminando la accesibilidad multiplataforma que demanda la comunidad educativa actual que utiliza dispositivos heterogéneos (tabletas, celulares, computadoras compartidas) [10]. Su enfoque en arquitecturas legacy lo posiciona por debajo de las capacidades actuales de modelos basados en transformers y Mixture-of-Experts que logran F1-scores superiores a 0.78 en benchmarks estandarizados [7].



2.2.3.2 Transcribe!

no es un transcriptor automático sino un asistente para transcripción manual, proporcionando herramientas como bucles, ralentización sin cambio de tono y visualización de espectrogramas para facilitar el trabajo auditivo del músico [10]. Su enfoque depende 100% de la percepción auditiva del usuario, no resolviendo la problemática central de tiempo/error humano que enfrentan estudiantes del INBAL al transcribir ejercicios extensos. Constituye una herramienta complementaria para músicos con oído entrenado, pero no una solución automatizada para contextos educativos donde se busca reducir la carga cognitiva y temporal mediante inteligencia artificial [2]. Su licencia única (USD \$39) resulta accesible económicamente, pero su paradigma de asistencia manual no aborda la necesidad de automatización total requerida en entornos académicos con alta carga de trabajo.



2.3 Comparacion de otras herramientas y softwares con Harmonic Score

El presente trabajo terminal, denominado "**Harmonic Score**", propone una solución integral y accesible para el desafío de la transcripción musical. Se trata de una aplicación web gratuita y de código abierto, diseñada específicamente para convertir automáticamente archivos de audio de instrumentos musicales individuales (tanto monofónicos como polifónicos) en partituras digitales en formato PDF. El objetivo es poner la tecnología de transcripción automática de última generación en manos de estudiantes y docentes, democratizando una herramienta que hoy en día es cara, inaccesible o poco práctica para fines educativos.

¿Qué se quiere realizar?

Se busca construir una herramienta cuyo flujo de trabajo sea simple y eficaz para el usuario final. El proceso sería el siguiente:

- El usuario accede a **Harmonic Score** desde cualquier navegador web moderno.
- Carga un archivo de audio (en formatos comunes como MP3 o WAV).
- La aplicación web procesa el archivo en segundo plano aplicando un modelo de inteligencia artificial.
- En pocos minutos, el usuario recibe una notificación y puede descargar la partitura transcrita en un archivo PDF, lista para ser impresa, estudiada o compartida.

El proyecto se centra en la calidad del resultado, implementando un post-procesamiento que refina la salida del modelo para asegurar que la partitura sea musicalmente coherente y fácil de leer, incluso si el audio de entrada no es de estudio profesional.

Ventajas y diferenciadores clave

"Harmonic Score" no es simplemente otra herramienta de transcripción; es una solución diseñada conscientemente para superar las limitaciones del panorama actual. Sus ventajas principales son:

- **Accesibilidad Total:** Al ser una aplicación web, **no requiere instalación** y es accesible desde cualquier dispositivo (computadora, tableta, celular). Su modelo **gratuito** elimina las barreras económicas que impiden el acceso a herramientas de alta precisión.
- **Precisión de Vanguardia:** En lugar de algoritmos obsoletos, utiliza modelos de aprendizaje profundo de última generación (como **Onsets and Frames** y **MT3**) que han sido ajustados (*fine-tuning*) para ofrecer un rendimiento competitivo y robusto, superando la precisión de muchas alternativas freemium.
- **Enfoque Educativo:** A diferencia de los editores de audio profesionales (como Melodyne) que se enfocan en la manipulación del sonido, Harmonic Score está optimizada para **generar partituras claras y legibles**. Su objetivo es facilitar el estudio y el análisis musical, no la producción musical.
- **Transparencia y Colaboración:** Al ser **código abierto**, su funcionamiento es transparente. Esto no solo genera confianza, sino que también invita a la comunidad educativa y científica a colaborar, adaptar y mejorar la herramienta, creando un ecosistema de innovación continua.
- **Viabilidad y Modernidad:** El proyecto se alinea perfectamente con las competencias de la ciencia de datos, priorizando el modelado predictivo y el uso de bibliotecas estandarizadas sobre la implementación manual de bajo nivel. Esto lo hace un proyecto factible, moderno y replicable.

En esencia, **Harmonic Score** es el puente entre la tecnología de punta en inteligencia artificial y las necesidades prácticas de la comunidad musical y educativa, especialmente la del INBAL, ofreciendo una solución que es a la vez potente, accesible y centrada en el usuario.

2.4 Tabla comparativa

El análisis individual de las herramientas existentes revela un ecosistema diverso, donde cada solución se especializa en un nicho particular, ya sea la edición profesional, el aprendizaje interactivo o la asistencia al músico. Sin embargo, para comprender de manera objetiva cómo se posiciona nuestra propuesta frente a estas alternativas y para visualizar las brechas que ninguna de ellas cubre de forma integral, es fundamental realizar una comparación directa. La siguiente tabla presenta un análisis comparativo de las características más relevantes costo, plataforma, función principal, nivel de automatización y accesibilidad del código de cada software evaluado, destacando en **negrita** los atributos distintivos de **Harmonic Score**.

Característica	Anthem							Harmonic
Clave	Melodyne	Soundslice	LALAL.AI	Score	Transcribe!	Klangio	ScoreCloud	Score (Propuesta)
Es Gratuita	✗	✗	✗	✗	✗	✗	✗	✓
Es Accesible Web (sin instalación)	✗	✓	✓	✗	✗	✓	✓	✓
Su Función Principal es la Transcripción Automática	✗	✗	✗	✓	✗	✓	✓	✓
Ofrece un Nivel de Automatización Total	✗	✗	✓	✓	✗	✓	✓	✓
Genera Salida en Formato PDF	✗	✗	✗	✓	✗	✗	✓	✓
Es de Código Abierto	✗	✗	✗	✗	✗	✗	✗	✓
Está Enfocada en la Comunidad Educativa	✗	✓	✗	✗	✗	✗	✓	✓

Harmonic Score cierra esta brecha al ofrecer una herramienta que no solo elimina las barreras económicas y de instalación, sino que también se compromete con un estándar de precisión más alto, respaldado por modelos de investigación punteros y un enfoque transparente gracias a su código abierto. Esto la convierte en la alternativa más adecuada y potente para el contexto educativo que se busca atender.

Capítulo 3. Marco teórico

Este capítulo establece los fundamentos tecnológicos y científicos que sustentan el desarrollo de "Harmonic Score". Se enfoca en las técnicas de vanguardia en aprendizaje profundo y procesamiento de audio que son directamente aplicables para resolver los desafíos de la transcripción musical en contextos reales, como las grabaciones de estudiantes del INBAL. Se ha minimizado la discusión sobre métodos pre-deep learning y fundamentos básicos de DSP para centrarse en las competencias clave de la ciencia de datos: modelado avanzado, fine-tuning y despliegue de sistemas inteligentes.

3.1 Transcripción Automática de Música (AMT)

La transcripción automática de música (AMT) requiere transformar una señal de audio continua $x(t) \in R$ en una representación discreta que preserve información relevante para la identificación de notas musicales. La Transformada de Fourier de Tiempo Corto (STFT) constituye el primer paso matemático fundamental [3].

$$X(m, k) = \sum_{n=0}^{N-1} x[n + mH] \cdot w[n] \cdot e^{-j2\pi kn/N}$$

Ecuación 1: Transformada de Fourier de Tiempo Corto (STFT)

Explicación de variables:

- $X(m, k)$: Espectrograma complejo resultante, donde m es el índice de ventana temporal y k es el índice de frecuencia discreta
- $x[n]$: Señal de audio discreta en el instante n (muestras normalizadas entre -1 y 1)
- n : Índice de muestra dentro de la ventana actual (rango: 0 a $N - 1$)
- m : Índice de ventana temporal (número de ventana deslizante)
- H : Tamaño del salto entre ventanas (*hop size*), típicamente 512 muestras
- $w[n]$: Ventana de Hann aplicada para reducir efectos de borde (valores entre 0 y 1)
- N : Longitud de la FFT (número de puntos, comúnmente 2048)
- k : Índice de frecuencia discreta (rango: 0 a $N - 1$)
- j : Unidad imaginaria ($j = \sqrt{-1}$)
- $e^{-j2\pi kn/N}$: Núcleo de la transformada exponencial compleja

La STFT genera un espectrograma de magnitud $|X(m, k)|$ que captura la evolución temporal de componentes frecuenciales, pero su resolución lineal en frecuencia no se alinea con la percepción musical humana.

3.1.1 Espectrograma Mel: Transformación psicoacústica

El espectrograma Mel aplica una transformación no lineal que comprime frecuencias altas y expande frecuencias bajas, modelando la respuesta del oído humano según la escala Mel [18]:

$$f_{\text{mel}} = 2595 \log_{10} \left(1 + \frac{f_{\text{Hz}}}{700} \right)$$

Ecuación 2: Conversión de frecuencia Hz a escala Mel

Explicación de variables:

- ♫ f_{mel} : Frecuencia en la escala Mel (unidades: Mel)
- ♫ f_{Hz} : Frecuencia en Hertz (Hz)
- ♫ 2595: Factor de escala empírico determinado experimentalmente
- ♫ 700: Frecuencia de corte en Hz donde la percepción cambia de lineal a logarítmica
- ♫ $\log_{10}$: Logaritmo base 10

La conversión práctica se implementa mediante bancos de filtros triangulares superpuestos en la escala Mel. Dado un espectrograma de potencia $P(k) = |X(m, k)|^2$, la energía en el i -ésimo filtro Mel se calcula como:

$$E_i = \sum_{k=0}^{K-1} P(k) \cdot M_i(k)$$

Ecuación 3: Energía en el i -ésimo filtro Mel

Explicación de variables:

- ♫ E_i : Energía total en el i -ésimo filtro Mel
- ♫ $P(k)$: Potencia espectral en el bin de frecuencia k (valor real positivo)
- ♫ $M_i(k)$: Respuesta del i -ésimo filtro triangular en el bin k (valor entre 0 y 1)
- ♫ K : Número total de bins de frecuencia en el espectrograma

♫ k : Índice de bin de frecuencia (rango: 0 a $K - 1$)

♫ i : Índice del filtro Mel (rango: 0 a $N_{\text{Mel}} - 1$, comúnmente 128)

donde $M_i(k)$ es la respuesta del i -ésimo filtro triangular definido por:

$$M_i(k) = \begin{cases} 0 & k < f(i-1) \\ \frac{k-f(i-1)}{f(i)-f(i-1)} & f(i-1) \leq k < f(i) \\ \frac{f(i+1)-k}{f(i+1)-f(i)} & f(i) \leq k < f(i+1) \\ 0 & k \geq f(i+1) \end{cases}$$

Ecuación 4: Respuesta del filtro triangular Mel

Explicación de variables:

♫ $f(i)$: Frecuencia de corte del filtro i en escala Hz (límite superior del triángulo)

♫ $f(i-1)$: Frecuencia de corte inferior del filtro i

♫ $f(i+1)$: Frecuencia de corte superior del filtro i

♫ k : Índice de bin de frecuencia actual

♫ Las condiciones definen la forma triangular del filtro: cero fuera del rango, pendiente ascendente en el lado izquierdo, pendiente descendente en el lado derecho.

Esta representación es óptima para tareas de reconocimiento de habla pero presenta limitaciones en AMT: la compresión excesiva en frecuencias medias dificulta la discriminación precisa de semitonos en regiones críticas del espectro musical (200–2000 Hz) [16].

3.1.2 Transformada de Constante-Q (CQT): Resolución logarítmica exacta

La Transformada de Constante-Q supera las limitaciones del espectrograma Mel mediante una resolución frecuencial logarítmica exacta, donde el cociente $Q = \frac{f_k}{\Delta f_k}$ permanece constante para todos los bins [16]. La base matemática se define como:

$$X_{\text{CQT}}[k] = \frac{1}{N_k} \sum_{n=0}^{N_k-1} x[n] \cdot w_k[n] \cdot e^{-j2\pi Qn/N_k}$$

Ecuación 5: Transformada de Constante-Q (CQT)

Explicación de variables:

- ♫ $X_{\text{CQT}}[k]$: Coeficiente CQT en el bin k (valor complejo)
- ♫ k : Índice del bin CQT (rango: 0 a $N_{\text{CQT}} - 1$)
- ♫ N_k : Número de muestras para el k -ésimo bin CQT (depende de la frecuencia: $N_k = f_s / (f_k / Q)$)
- ♫ $x[n]$: Señal de audio original en la muestra n
- ♫ $w_k[n]$: Ventana de Blackman-Harris adaptada a la longitud N_k del bin k
- ♫ f_s : Frecuencia de muestreo de la señal (típicamente 44100 Hz)
- ♫ f_k : Frecuencia central del bin k (calculada como $f_k = f_{\min} \cdot 2^{k/B}$)
- ♫ Q : Factor de calidad constante (típicamente 34 para música)
- ♫ B : Número de bins por octava (típicamente 12 para semitonos)
- ♫ $f_{\min}$: Frecuencia mínima analizada (típicamente 32.7 Hz = Do0)
- ♫ $e^{-j2\pi Qn/N_k}$: Núcleo exponencial complejo adaptado al bin CQT

donde $N_k = \frac{f_s}{f_k/Q}$ es el número de muestras para el k -ésimo bin, $f_k = f_{\min} \cdot 2^{k/B}$ la frecuencia central (B bins por octava), y $w_k[n]$ una ventana de Blackman-Harris adaptada a la longitud del bin [17]. La propiedad fundamental es:

$$\frac{\Delta f_k}{f_k} = \frac{1}{Q} = \text{constante}$$

Ecuación 6: Propiedad fundamental de resolución constante CQT

Explicación de variables:

- ♫ Δf_k : Ancho de banda del bin k (diferencia entre frecuencias superior e inferior)
- ♫ f_k : Frecuencia central del bin k
- ♫ Q : Factor de calidad (cociente entre frecuencia central y ancho de banda)
- ♫ Esta propiedad garantiza que la resolución relativa sea constante en todo el espectro

Esta característica garantiza que cada octava musical contenga exactamente B bins (típicamente $B = 12$ para semitonos), alineando la representación con la estructura de la escala temperada occidental [16]. La CQT permite:

- ♫ **Discriminación precisa de semitonos** en todo el rango audible (20 Hz – 20 kHz)
- ♫ **Identificación robusta de armónicos** mediante relaciones enteras $f_{\text{armónico}} = n \cdot f_{\text{fundamental}}$
- ♫ **Invarianza ante transposición** (propiedad crítica para modelos universales)

Implementaciones eficientes como la *pseudo-CQT* en Librosa [18] utilizan muestreo no uniforme en el dominio de frecuencia para reducir complejidad computacional de $\mathcal{O}(N^2)$ a $\mathcal{O}(N \log N)$ mediante interpolación en el espectrograma STFT estándar.

3.2 Redes Neuronales Recurrentes (RNN) y variantes con memoria

Las RNN modelan dependencias temporales mediante un estado oculto h_t que se actualiza recursivamente:

$$h_t = \sigma(W_{xh}x_t + W_{hh}h_{t-1} + b_h)$$

Ecuación 7: Actualización del estado oculto en RNN

$$y_t = W_{hy}h_t + b_y$$

Ecuación 8: Cálculo de salida en RNN

Explicación de variables:

- ♫ h_t : Vector de estado oculto en el tiempo t (dimensionalidad: d_h)
- ♫ x_t : Vector de entrada espectral en el tiempo t (dimensionalidad: d_x)
- ♫ y_t : Vector de salida en el tiempo t (dimensionalidad: d_y)
- ♫ W_{xh} : Matriz de pesos de entrada a estado oculto (dimensionalidad: $d_h \times d_x$)
- ♫ W_{hh} : Matriz de pesos de estado anterior a estado actual (dimensionalidad: $d_h \times d_h$)
- ♫ W_{hy} : Matriz de pesos de estado oculto a salida (dimensionalidad: $d_y \times d_h$)
- ♫ b_h : Vector de sesgo del estado oculto (dimensionalidad: d_h)
- ♫ b_y : Vector de sesgo de la salida (dimensionalidad: d_y)
- ♫ σ : Función de activación sigmoide ($\sigma(z) = 1/(1 + e^{-z})$)
- ♫ t : Índice de tiempo discreto (rango: 0 a $T - 1$)
- ♫ T : Longitud total de la secuencia temporal

donde x_t es la entrada espectral en el tiempo t , σ la función sigmoide, y W las matrices de pesos [5]. Sin embargo, el gradiente durante retropropagación sufre de *vanishing/exploding gradients* en secuencias largas:

$$\frac{\partial \mathcal{L}}{\partial W_{hh}} = \sum_{t=1}^T \frac{\partial \mathcal{L}_t}{\partial h_t} \prod_{k=1}^t \frac{\partial h_k}{\partial h_{k-1}}$$

Ecuación 9: Problema de gradientes desvanecidos/explotados en RNN

Explicación de variables:

- ♫ $\mathcal{L}$: Función de pérdida total del modelo
- ♫ $\mathcal{L}_t$: Pérdida en el tiempo t
- ♫ $\frac{\partial \mathcal{L}}{\partial W_{hh}}$: Gradiente de la pérdida respecto a los pesos recurrentes
- ♫ $\frac{\partial \mathcal{L}_t}{\partial h_t}$: Gradiente de la pérdida en t respecto al estado oculto
- ♫ $\frac{\partial h_k}{\partial h_{k-1}}$: Gradiente del estado en k respecto al estado anterior
- ♫ $\prod_{k=1}^t$: Producto acumulado de gradientes desde el inicio hasta t
- ♫ Este producto tiende a cero (vanishing) o infinito (exploding) cuando t es grande

Las LSTM (Long Short-Term Memory) resuelven este problema mediante compuertas especializadas [5]:

$$\begin{aligned} f_t &= \sigma(W_f \cdot [h_{t-1}, x_t] + b_f) && \text{(compuerta de olvido)} \\ i_t &= \sigma(W_i \cdot [h_{t-1}, x_t] + b_i) && \text{(compuerta de entrada)} \\ \tilde{C}_t &= \tanh(W_C \cdot [h_{t-1}, x_t] + b_C) && \text{(candidato de celda)} \\ C_t &= f_t \odot C_{t-1} + i_t \odot \tilde{C}_t && \text{(actualización de celda)} \\ o_t &= \sigma(W_o \cdot [h_{t-1}, x_t] + b_o) && \text{(compuerta de salida)} \\ h_t &= o_t \odot \tanh(C_t) && \text{(estado oculto)} \end{aligned}$$

Ecuación 10: Arquitectura LSTM: Compuertas y actualización de celda de memoria

Explicación de variables:

- ♫ f_t : Vector de compuerta de olvido (valores entre 0 y 1, dónde 0 = olvidar completamente, 1 = recordar completamente)

- ♫ i_t : Vector de compuerta de entrada (valores entre 0 y 1, controla cuánta información nueva se almacena)
- ♫ $\tilde{C}_t$: Vector candidato de celda (nueva información potencial, valores entre -1 y 1)
- ♫ C_t : Vector de celda de memoria en tiempo t (almacena información a largo plazo)
- ♫ o_t : Vector de compuerta de salida (valores entre 0 y 1, controla cuánta información de la celda se expone)
- ♫ h_t : Vector de estado oculto final (salida visible de la LSTM)
- ♫ W_f, W_i, W_C, W_o : Matrices de pesos para cada compuerta (dimensionalidad: $d_h \times (d_h + d_x)$)
- ♫ b_f, b_i, b_C, b_o : Vectores de sesgo para cada compuerta (dimensionalidad: d_h)
- ♫ $[h_{t-1}, x_t]$: Concatenación de vectores (dimensionalidad: $d_h + d_x$)
- ♫ $\odot$: Producto elemento a elemento (Hadamard product)
- ♫ $\tanh$: Función tangente hiperbólica (valores entre -1 y 1)

donde $\odot$ denota producto elemento a elemento. Esta arquitectura permite retener información a largo plazo mediante la celda de memoria C_t , esencial para capturar estructuras musicales como frases melódicas o progresiones armónicas [5]. Sin embargo, las RNN presentan limitaciones inherentes: (1) procesamiento secuencial que impide paralelización durante entrenamiento, (2) dificultad para modelar dependencias de muy largo alcance (>1000 pasos), y (3) degradación de gradientes en secuencias extremadamente largas [20].

3.3 Modelo Onsets and Frames: Arquitectura dual para precisión temporal

El modelo *Onsets and Frames* [6] implementa una arquitectura híbrida con dos ramas LSTM paralelas que optimizan objetivos complementarios:

- ♫ **Rama de *onsets***: Detecta inicios precisos de notas mediante convolución 1D seguida de LSTM bidireccional
- ♫ **Rama de *frames***: Predice activación continua de notas mediante segunda LSTM

La función de pérdida combinada optimiza ambas tareas simultáneamente:

$$L_{\text{total}} = \lambda_{\text{onset}} \mathcal{L}_{\text{BCE}}(y_{\text{onset}}, \hat{y}_{\text{onset}}) + \lambda_{\text{frame}} \mathcal{L}_{\text{BCE}}(y_{\text{frame}}, \hat{y}_{\text{frame}})$$

Ecuación 11: Función de pérdida combinada Onsets and Frames

Explicación de variables:

- ♫ $\mathcal{L}_{\text{total}}$: Pérdida total combinada del modelo (valor escalar)
- ♫ λ_{onset} : Peso de balanceo para la tarea de detección de *onsets* (valor empírico = 1.0)
- ♫ λ_{frame} : Peso de balanceo para la tarea de activación de *frames* (valor empírico = 0.5)
- ♫ $\mathcal{L}_{\text{BCE}}$: Función de entropía cruzada binaria (*Binary Cross-Entropy*), definida como
- ♫ y_{onset} : Vector de etiquetas binarias para *onsets* (1 = inicio de nota detectado, 0 = no hay inicio)
- ♫ $\hat{y}_{\text{onset}}$: Vector de predicciones del modelo para *onsets* (valores entre 0 y 1, probabilidades)
- ♫ y_{frame} : Matriz de etiquetas binarias para activación de *frames* (dimensiones: tiempo × pitch)
- ♫ $\hat{y}_{\text{frame}}$: Matriz de predicciones del modelo para activación de *frames* (valores entre 0 y 1)
- ♫ N : Número total de muestras (frames temporales) en el batch

Esta estrategia incrementa el F1-score de *onsets* en 12.3 puntos porcentuales versus arquitecturas de activación única, logrando F1-onset = 0.852 en piano monofónico sobre MAESTRO v2.0 [6].

Limitaciones críticas:

1. **Arquitectura específica para piano que no generaliza a otros instrumentos sin reentrenamiento completo:** El modelo Onsets and Frames fue entrenado exclusivamente en el dataset MAESTRO, que contiene únicamente grabaciones de piano de cola. Las características espectrales del piano (ataque percusivo, decaimiento exponencial, rango de 88 teclas) son muy distintas a otros instrumentos como flauta (ataque suave, sostenido continuo) o guitarra (armónicos prominentes, rango limitado). Reentrenar el modelo para otros instrumentos requiere un nuevo dataset etiquetado completo y semanas de cómputo en GPU, inviable para un trabajo terminal [6].
2. **Incapacidad para modelar interacciones armónicas entre notas simultáneas mediante mecanismos de atención global:** Las LSTM procesan secuencias de forma secuencial con memoria limitada a ~500 pasos temporales. En música polifónica compleja (6+ notas simultáneas), las relaciones armónicas entre notas distantes en el tiempo (ej: acordes de séptima resolviendo a tríadas) no pueden ser capturadas eficientemente. Los mecanismos

de atención global de los Transformers permiten "mirar atrás" arbitrariamente en la secuencia, resolviendo ambigüedades armónicas que las LSTM no pueden modelar [7].

3. **Escalabilidad limitada ante polifonía compleja (>6 notas simultáneas) debido a la naturaleza secuencial de las LSTM:** El procesamiento secuencial de las LSTM impide paralelización durante entrenamiento, incrementando el tiempo de cómputo en $\mathcal{O}(T)$ donde T es la longitud de la secuencia. Para grabaciones de 5 minutos (13,230,000 muestras a 44.1 kHz), el entrenamiento completo toma ~72 horas en GPU NVIDIA V100, mientras arquitecturas Transformer paralelizadas reducen este tiempo a ~8 horas [7]. Esta limitación de escalabilidad hace inviable el uso de Onsets and Frames para datasets extensos o múltiples instrumentos.

3.4 Arquitecturas Transformer y MT3: Mecanismos de atención global

Los Transformers reemplazan las conexiones recurrentes por mecanismos de atención que ponderan dinámicamente la relevancia de diferentes posiciones temporales [20]. La atención escalada de producto punto (*scaled dot-product attention*) se define como:

$$\text{Attention}(Q, K, V) = \text{softmax}\left(\frac{QK^T}{\sqrt{d_k}}\right)V$$

Ecuación 12: Atención escalada de producto punto (Scaled Dot-Product Attention)

Explicación de variables:

- ♫ $\text{Attention}(Q, K, V)$: Salida de la capa de atención (matriz de dimensiones $T \times d_v$)
- ♫ Q : Matriz de *queries* (consultas), proyección lineal de la entrada para determinar "qué buscar" (dimensiones: $T \times d_k$)
- ♫ K : Matriz de *keys* (claves), proyección lineal para determinar "qué ofrecer" (dimensiones: $T \times d_k$)
- ♫ V : Matriz de *values* (valores), proyección lineal que contiene la información real a recuperar (dimensiones: $T \times d_v$)
- ♫ T : Longitud de la secuencia temporal (número de tokens/frames)
- ♫ d_k : Dimensión de las *keys* y *queries* (típicamente 64 o 128 en modelos musicales)
- ♫ d_v : Dimensión de los *values* (típicamente igual a d_k)

♫ $\sqrt{d_k}$: Factor de escala para estabilizar gradientes durante entrenamiento (evita que los valores del softmax se saturen)

♫ $\text{softmax}(\cdot)$: Función softmax aplicada fila por fila, convierte puntuaciones en probabilidades que suman 1

♫ K^T : Transpuesta de la matriz K (dimensiones: $d_k \times T$)

♫ QK^T : Producto matricial que calcula similitud entre cada *query* y todas las *keys* (dimensiones: $T \times T$)

donde Q (queries), K (keys) y V (values) son proyecciones lineales de la entrada X :

$$Q = XW^Q, \quad K = XW^K, \quad V = XW^V$$

Ecuación 13: Proyecciones lineales de Queries, Keys y Values

Explicación de variables adicionales:

♫ X : Matriz de entrada (dimensiones: $T \times d_{\text{model}}$), donde d_{model} es la dimensión del modelo (típicamente 512)

♫ W^Q : Matriz de pesos para proyectar X a espacio de *queries* (dimensiones: $d_{\text{model}} \times d_k$)

♫ W^K : Matriz de pesos para proyectar X a espacio de *keys* (dimensiones: $d_{\text{model}} \times d_k$)

♫ W^V : Matriz de pesos para proyectar X a espacio de *values* (dimensiones: $d_{\text{model}} \times d_v$)

♫ Estas proyecciones permiten que el modelo aprenda diferentes representaciones para cada rol (query/key/value)

y d_k la dimensión de las keys para estabilizar gradientes [20]. Esta formulación permite al modelo "prestar atención" a notas anteriores distantes para resolver ambigüedades armónicas, superando las limitaciones de memoria de las RNN.

MT3 (*Multi-Task Multitrack Music Transcription*) [7] implementa un Transformer secuencia-a-secuencia que genera una secuencia unificada de eventos musicales ($e_1, e_2, \dots, e_T$) donde cada evento e_t codifica:

$$e_t = (\text{instrumento}, \text{pitch}, \text{inicio}, \text{duración}, \text{velocidad})$$

Ecuación 14: Codificación de evento musical en MT3

Explicación de variables del evento musical:

♫ e_t : Evento musical en el tiempo t (tupla de 5 elementos)

♫ instrumento: Identificador del instrumento (como: piano=0, guitarra=1, flauta=2)

- ♫ pitch: Altura de la nota en escala MIDI (0-127, donde 60 = Do4)
- ♫ inicio: Tiempo de inicio de la nota en segundos desde el comienzo del audio
- ♫ duración: Duración de la nota en segundos
- ♫ velocidad: Intensidad de la nota (0-127, donde 64 = mezzoforte)

El decodificador utiliza *masked self-attention* para prevenir *look-ahead bias*:

$$\text{MaskedAttention}(Q, K, V) = \text{softmax}\left(\frac{QK^T}{\sqrt{d_k}} + M\right)V$$

Ecuación 15: Atención enmascarada para causalidad en decodificador

Explicación de variables adicionales:

- ♫ $\text{MaskedAttention}(Q, K, V)$: Variante de atención con máscara causal
- ♫ M : Matriz de máscara triangular superior (dimensiones: $T \times T$)
- ♫ $M_{ij} = -\infty$ si $j > i$ (impide que posición i "mire al futuro" en posición j)
- ♫ $M_{ij} = 0$ si $j \leq i$ (permite atención al pasado y presente)
- ♫ Esta máscara garantiza causalidad: el modelo solo puede generar evento e_t basándose en eventos anteriores $e_1, \dots, e_{t-1}$, nunca en eventos futuros

donde $M_{ij} = -\infty$ si $j > i$ (máscara triangular superior), garantizando causalidad en la generación [7]. MT3 logra F1-scores de 0.79 para *onsets* en piano polifónico y 0.68 en instrumentos monofónicos sobre MedleyDB v2 [7].

Limitaciones:

Arquitectura densa que procesa todos los tokens a través de todas las capas, generando latencias de inferencia elevadas (>50 ms para 30 segundos de audio en hardware estándar): MT3 utiliza un Transformer denso donde cada token pasa por todas las capas del modelo, sin importar su complejidad. Para 30 segundos de audio a 100 Hz de muestreo temporal, se generan 3,000 tokens que deben procesarse secuencialmente a través de 12 capas Transformer, resultando en latencias >50 ms en GPU NVIDIA T4. Esto es inviable para aplicaciones web donde el usuario espera respuesta en <2 segundos [12].

Requerimientos computacionales intensivos durante entrenamiento (necesita clusters de GPUs para escalado): Entrenar MT3 desde cero requiere ~256 GPUs NVIDIA V100 durante 7 días para converger en datasets extensos como MAESTRO + MedleyDB. Este requerimiento de hardware

especializado está fuera del alcance de laboratorios académicos estándar y de un trabajo terminal [7].

Dificultad para especialización adaptativa según complejidad del input (monofónico vs. polifónico): MT3 procesa señales monofónicas simples (flauta solista) con la misma capacidad computacional que señales polifónicas complejas (piano con 6+ notas simultáneas). Esto es ineficiente: señales simples no necesitan toda la capacidad del modelo, desperdiciando recursos computacionales [12].

3.5 YourMT3 y arquitecturas Mixture-of-Experts (MoE)

Las arquitecturas MoE escalan capacidad de modelo sin incrementar proporcionalmente latencia mediante ruteo condicional a subredes especializadas (*experts*) [21]. YourMT3-YPTF-MoE-M [12] implementa un router suave que distribuye cada token x_t entre N expertos:

$$y_t = \sum_{i=1}^N G(x_t)_i \cdot E_i(x_t)$$

Ecuación 16: Combinación ponderada de expertos en arquitectura MoE

Explicación de variables:

- ♫ y_t : Vector de salida del modelo en el tiempo t (dimensionalidad: d_{model})
- ♫ x_t : Vector de entrada (token espectral) en el tiempo t (dimensionalidad: d_{model})
- ♫ N : Número total de expertos en la arquitectura (en YourMT3-MoE: $N = 8$)
- ♫ i : Índice del experto actual (rango: 1 a N)
- ♫ $G(x_t)_i$: Peso de enrutamiento (*gating weight*) para el experto i en el token x_t (valor entre 0 y 1, suma de todos los pesos = 1)
- ♫ $E_i(x_t)$: Función de transformación del i -ésimo experto aplicada al token x_t (subred Transformer completa)
- ♫ $\sum_{i=1}^N$: Sumatoria sobre todos los expertos posibles
- ♫ El producto $G(x_t)_i \cdot E_i(x_t)$ pondera la contribución de cada experto según su afinidad con el token actual

donde $G(x_t)_i$ es el peso de enrutamiento para el experto i calculado mediante:

$$G(x_t) = \text{softmax}\left(\text{top}_k(\text{router}(x_t))\right)$$

Ecuación 17: Enrutamiento suave con top-k selection

Explicación de variables adicionales:

- ♫ $G(x_t)$: Vector de pesos de enrutamiento completo (dimensionalidad: N)
- ♫ $\text{router}(x_t)$: Función de enrutamiento que proyecta el token a espacio de afinidad con expertos
- ♫ $\text{top}_k(\cdot)$: Operador que selecciona los k valores más altos del vector y establece los demás en $-\infty$ (típicamente $k = 2$ en YourMT3-MoE)
- ♫ $\text{softmax}(\cdot)$: Función softmax que normaliza los pesos seleccionados para que sumen 1
- ♫ La combinación $\text{softmax}(\text{top}_k(\cdot))$ garantiza que solo k expertos contribuyan a la salida, reduciendo complejidad computacional de $\mathcal{O}(N)$ a $\mathcal{O}(k)$

y $E_i(\cdot)$ la función del i -ésimo experto (subred Transformer completa). El operador top_k selecciona los k expertos con mayor afinidad (típicamente $k = 2$), reduciendo complejidad de $\mathcal{O}(N)$ a $\mathcal{O}(k)$ por token [21]. La función de *gating* se implementa como:

$$\text{router}(x_t) = W_g \cdot \text{LayerNorm}(x_t) + b_g$$

Ecuación 18: Función de gating para proyección a espacio de expertos

Explicación de variables adicionales:

- ♫ $\text{router}(x_t)$: Vector de afinidades no normalizadas entre el token x_t y cada experto (dimensionalidad: N)
- ♫ W_g : Matriz de pesos del router (dimensionalidad: $N \times d_{\text{model}}$)
- ♫ b_g : Vector de sesgo del router (dimensionalidad: N)
- ♫ $\text{LayerNorm}(x_t)$: Normalización por capa aplicada al token x_t para estabilidad numérica
- ♫ El producto $W_g \cdot \text{LayerNorm}(x_t)$ proyecta el token normalizado al espacio de N dimensiones (una por experto)

donde $W_g \in \mathbb{R}^{d \times N}$ proyecta el token a espacio de N expertos [12].

Explicación de notación matemática:

- ♫ $\mathbb{R}^{d \times N}$: Espacio de matrices reales con d filas y N columnas
- ♫ d : Dimensión del modelo (en YourMT3-MoE: $d = 512$)

♫ N : Número de expertos (en YourMT3-MoE: $N = 8$)

♫ $W_g \in \mathbb{R}^{d \times N}$ indica que W_g es una matriz con d filas (dimensionalidad del token) y N columnas (un peso por experto)

Esta arquitectura logra F1-scores de 0.78 ± 0.04 para *onsets* en piano polifónico mientras mantiene latencia de inferencia de 12.60 ms en hardware estándar (NVIDIA T4), superando arquitecturas densas equivalentes en relación precisión/latencia [12]. La especialización de expertos permite adaptación dinámica: expertos 1-3 especializados en señales monofónicas (flauta, voz), expertos 4-6 en polifonía media (guitarra), y expertos 7-8 en polifonía compleja (piano) [12].

♫ **F1-score**: Métrica de precisión que combina *precision* (exactitud) y *recall* (completitud)

- Fórmula: $F1 = 2 \cdot \frac{\text{precision} \cdot \text{recall}}{\text{precision} + \text{recall}}$
- Rango: 0 (peor) a 1 (perfecto)
- Valor 0.78 ± 0.04 significa: promedio de 0.78 con desviación estándar de 0.04 (rango típico: 0.74–0.82)

♫ **Onsets**: Momentos precisos de inicio de notas musicales (en segundos desde el comienzo del audio)

♫ **Latencia de inferencia**: Tiempo que tarda el modelo en procesar 1 segundo de audio (12.60 ms = 0.0126 segundos)

♫ **NVIDIA T4**: GPU estándar de datacenter con 16 GB de memoria VRAM, disponible como herramienta personal

La especialización de expertos permite adaptación dinámica: expertos 1-3 especializados en señales monofónicas (flauta, voz), expertos 4-6 en polifonía media (guitarra), y expertos 7-8 en polifonía compleja (piano) [12].

3.5.1 Parámetros de entrada del modelo

YourMT3-YPTF-MoE-M

El modelo recibe como entrada una representación espectral procesada con los siguientes parámetros específicos:

Configuración de entrada:

```
# Parámetros de configuración del modelo
input_config = {
    "sample_rate": 44100,      # Frecuencia de muestreo (Hz)
    "hop_length": 512,        # Saltos entre ventanas (muestras)
    "n_bins": 264,            # Número total de bins CQT
    "bins_per_octave": 12,    # Resolución de semitonos
    "fmin": 32.7,              # Frecuencia mínima (Hz, Do0)
    "fmax": 8372.0,           # Frecuencia máxima (Hz, Do8+)
    "time_frames": 2584,      # Frames temporales (30 segundos de audio)
    "batch_size": 1,          # Procesamiento de un archivo a la vez
    "channels": 1              # Mono (estéreo se convierte a mono)
}
```

Ilustración 2: Ejemplo de configuracion de parametros

Explicación detallada de parámetros:

Parámetro	Valor	Descripción	Rango permitido
sample_rate	44100 Hz	Frecuencia de muestreo estándar CD quality	16000–48000 Hz
hop_length	512 muestras	Salto entre ventanas STFT/CQT	256–1024 muestras
n_bins	264 bins	Número total de bins CQT (22 octavas × 12 semitonos)	128–512 bins
bins_per_octave	12	Resolución por octava (semitonos)	12–48 bins
fmin	32.7 Hz	Frecuencia mínima (Do0, nota más grave del piano)	20–100 Hz
fmax	8372.0 Hz	Frecuencia máxima (Do8 + margen para armónicos)	4000–16000 Hz
time_frames	2584	Frames temporales para 30 segundos de audio	Variable según duración
batch_size	1	Tamaño del lote (procesamiento secuencial)	1–8
channels	1	Canales de audio (mono)	1 (mono)

Tabla 1: explicación detallada de los parámetros

Tensor de entrada:

El modelo recibe un tensor con la siguiente estructura:

```
# Formato: (batch_size, channels, time_frames, freq_bins)
# Para un audio de 30 segundos:
input_tensor = torch.randn(1, 1, 2584, 264)

# Dimensiones:
# batch_size = 1 (un archivo a la vez)
# channels = 1 (audio mono)
# time_frames = 2584 (30s × 44100 / 512)
# freq_bins = 264 (22 octavas × 12 bins)
```

Ilustración 3: Ejemplo tensor de entrada

3.5.2 Representación espectral CQT: Configuración específica

La Transformada Constant-Q (CQT) se configura con los siguientes parámetros técnicos:

Parámetros CQT:

```
cqt_params = {
    "sr": 44100,          # Sample rate
    "hop_length": 512,    # Hop length
    "fmin": librosa.note_to_hz('C0'), # 32.70 Hz
    "n_bins": 264,        # 22 octavas × 12 semitonos
    "bins_per_octave": 12, # Semitonos por octava
    "filter_scale": 1.0,   # Escala del filtro
    "norm": 1,            # Normalización L1
    "sparsity": 0.01,     # Umbral de sparsity
    "window": 'hann'      # Ventana de Hann
}
```

Ilustración 4: Ejemplo de configuración de parámetros de la transformada de CQT

Cobertura de rango de frecuencias:

Octava	Rango de frecuencias	Bins CQT	Notas cubiertas
0	32.7–65.4 Hz	0–11	Do0–Si0
1	65.4–130.8 Hz	12–23	Do1–Si1

2	130.8–261.6 Hz	24–35	Do2–Si2
3	261.6–523.3 Hz	36–47	Do3–Si3
4	523.3–1046.5 Hz	48–59	Do4–Si4
5	1046.5–2093.0 Hz	60–71	Do5–Si5
6	2093.0–4186.0 Hz	72–83	Do6–Si6
7	4186.0–8372.0 Hz	84–95	Do7–Do8+
Total	32.7–8372.0 Hz	0–263	264 bins

Tabla 2: Cobertura de frecuencias en notas

Cálculo de bins CQT:

```
# Frecuencia central del bin k
f_k = f_min * (2 ** (k / bins_per_octave))

# Ancho de banda del bin k
delta_f_k = f_k / Q

# Número de muestras para el bin k
N_k = int(sample_rate / (f_k / Q))
```

Ilustración 5: Ejemplo de calculo

Ejemplo para bin 72 (Do4 = 261.63 Hz):

$$f_k = 32.7(2^{72/12}) = 32.7 * 64 = 2092.8Hz$$

$$\delta f_{72} = \frac{2092.8}{34} = 61.55 Hz$$

$$N_{72} = \frac{44100}{\frac{2092.8}{34}} = 716 \text{ muestras}$$

3.5.3 Arquitectura del codificador: Configuración de capas

El codificador de YourMT3 consta de 24 capas Transformer con la siguiente configuración:

Configuración por capa:

```

encoder_config = {
    "num_layers": 24,          # Número total de capas
    "d_model": 512,           # Dimensión del modelo
    "nhead": 8,               # Número de cabezas de atención
    "dim_feedforward": 2048,   # Dimensión del feedforward
    "dropout": 0.1,           # Tasa de dropout
    "activation": "gelu",      # Función de activación
    "layer_norm_eps": 1e-5     # Epsilon para LayerNorm
}

```

Ilustración 6: Ejemplo configuración de capa

Estructura detallada por tipo de capa:

Capa	Tipo	Dimensión entrada	Dimensión salida	Componentes
1–8	Transformer Encoder	(2584, 512)	(2584, 512)	MultiHeadAttention (8×64) + FFN (512→2048→512)
9–16	Transformer Encoder	(2584, 512)	(2584, 512)	MultiHeadAttention (8×64) + FFN (512→2048→512)
17–24	Transformer Encoder	(2584, 512)	(2584, 512)	MultiHeadAttention (8×64) + FFN (512→2048→512)

Tabla 3: Estructura de las capas

Proceso de codificación por capa:

```

# Para cada capa l (1 a 24):
x = input_tensor # (2584, 512)

# 1. Multi-Head Attention
attn_output = MultiheadAttention(
    query=x, key=x, value=x,
    num_heads=8,
    embed_dim=512
) # (2584, 512)

# 2. Residual connection + LayerNorm
x = LayerNorm(x + attn_output) # (2584, 512)

```



```
# 3. FeedForward Network
ffn_output = Linear(ReLU(Linear(x, 2048)), 512) # (2584, 512)

# 4. Residual connection + LayerNorm
x = LayerNorm(x + ffn_output) # (2584, 512)

# Salida de la capa
output = x # (2584, 512)
```

Ilustración 7: Ejemplo codificación por capa

3.5.4 Mecanismo Mixture-of-Experts (MoE): Configuración específica

La arquitectura MoE reemplaza las capas FeedForward estándar con ruteo condicional:

Configuración MoE:

```
moe_config = {
    "num_experts": 8,      # Número total de expertos
    "top_k": 2,           # Expertos activos por token
    "capacity_factor": 1.25, # Factor de capacidad (25% extra)
    "router_bias": True,   # Sesgo en el router
    "router_dropout": 0.0, # Dropout en el router
    "expert_capacity": 128  # Tokens máximo por experto
}
```

Ilustración 8: Ejemplo configuración MoE

Especialización de expertos por complejidad armónica:

Experto	Índice	Especialización	Capacidad máxima	Instrumentos objetivo
Experto 1	0	Señales monofónicas simples	128 tokens	Flauta, voz femenina
Experto 2	1	Señales monofónicas con vibrato	128 tokens	Clarinete, oboe
Experto 3	2	Monofónico con armónicos	128 tokens	Violín solo, trompeta
Experto 4	3	Polifonía baja (2-3 notas)	128 tokens	Guitarra fingerstyle
Experto 5	4	Polifonía media (4-5 notas)	128 tokens	Piano simple, arpa

Experto 6	5	Polifonía media-alta (5-6 notas)	128 tokens	Piano jazz
Experto 7	6	Polifonía alta (7-8 notas)	128 tokens	Piano clásico
Experto 8	7	Polifonía extrema (9+ notas)	128 tokens	Piano orquestal

Tabla 4: Complejidad armónica

Función de gating (router):

```
def router(x_t):
    """
    Función de gating para ruteo de tokens a expertos

    Args:
        x_t: Tensor de entrada (512,)

    Returns:
        logits: Logits sin normalizar para cada experto (8,)
        weights: Pesos normalizados para top-k expertos (2,)
        expert_indices: Índices de expertos seleccionados (2,)
    """
    # Normalización del token
    x_norm = LayerNorm(x_t) # (512,)

    # Proyección a espacio de expertos
    logits = W_g @ x_norm + b_g # (512,) @ (512, 8) → (8,)
    #  $W_g \in \mathbb{R}^{(512 \times 8)}$ ,  $b_g \in \mathbb{R}^8$ 

    # Selección top-k
    top_k_logits, top_k_indices = torch.topk(logits, k=2)

    # Softmax normalizado
    weights = F.softmax(top_k_logits, dim=-1) # (2,)

    return logits, weights, top_k_indices
```

Ilustración 9: función gating ejemplo

3.5.5 Timestamp y sincronización temporal: Resolución y formato

El modelo captura eventos musicales con alta precisión temporal:

Resolución temporal:

```
temporal_config = {
  "frame_rate": 86.13,      # Frames por segundo (44100 / 512)
  "frame_duration": 11.61,  # Duración por frame en ms
  "onset_precision": 5.8,   # Precisión de onset (medio frame)
  "offset_precision": 11.6, # Precisión de offset (un frame)
  "max_duration": 300.0     # Duración máxima (segundos)
}
```

Ilustración 10: resolución temporal

Formato de timestamps:

Campo	Tipo	Rango	Resolución	Unidades	Ejemplo
time	Float32	0.0–300.0	11.6 ms	Segundos	1.234
onset	Float32	0.0–300.0	11.6 ms	Segundos	0.000
offset	Float32	0.0116–300.0	11.6 ms	Segundos	0.500
duration	Float32	0.0116–10.0	11.6 ms	Segundos	0.500

Tabla 5: Formatos de timestep

Ejemplo de secuencia de eventos:

```
sequence = [
  {
    "type": "note_on",
    "time": 0.000,
    "pitch": 60,      # Do4 (MIDI)
    "velocity": 75,   # Intensidad (0–127)
    "instrument": 0,  # Piano
    "confidence": 0.94, # Confianza (0.0–1.0)
    "frame_idx": 0    # Índice del frame CQT
  },
  {
    "type": "note_off",
```

```

    "time": 0.500,
    "pitch": 60,
    "velocity": 0,
    "instrument": 0,
    "confidence": 0.91,
    "frame_idx": 43      # 0.5s × 86.13 fps = 43
  },
  {
    "type": "note_on",
    "time": 0.500,
    "pitch": 64,          # Mi4
    "velocity": 70,
    "instrument": 0,
    "confidence": 0.93,
    "frame_idx": 43
  },
  # ... más eventos
]

```

Ilustración 11: Ejemplo de secuencias de evento

3.5.6 Capacidad de procesamiento polifónico: Límites y detección

YourMT3-MoE está diseñado para manejar diferentes niveles de polifonía:

Límites de polifonía por experto:

Nivel	Notas simultáneas	Expertos activados	Capacidad máxima	Ejemplo
Monofónico	1 nota	1–2	128 tokens/step	Flauta solista
Polifonía baja	2–3 notas	2–3	192 tokens/step	Guitarra dúo
Polifonía media	4–6 notas	3–5	320 tokens/step	Piano jazz
Polifonía alta	7–10 notas	5–8	512 tokens/step	Piano clásico
Polifonía extrema	11+ notas	6–8	640 tokens/step	Piano orquestal

Tabla 6: Límites de polifonía por experto

Detección automática de polifonía:

```
def detect_polyphony(cqt_frame):
    """
    Detecta el nivel de polifonía en un frame CQT
    Args:
        cqt_frame: Array (264,) con magnitud CQT

    Returns:
        level: Nivel de polifonía ('mono', 'low', 'medium', 'high', 'extreme')
        experts: Lista de expertos a activar
        entropy: Entropía espectral calculada
    """
    # Calcular entropía espectral normalizada
    energy = np.sum(cqt_frame, axis=0)
    prob = energy / np.sum(energy)
    entropy = -np.sum(prob * np.log2(prob + 1e-10))

    # Clasificación por umbrales de entropía
    if entropy < 0.35:
        return "mono", [0, 1], entropy
    elif entropy < 0.65:
        return "low", [2, 3], entropy
    elif entropy < 0.85:
        return "medium", [4, 5], entropy
    elif entropy < 0.95:
        return "high", [6, 7], entropy
    else:
        return "extreme", [6, 7], entropy
```

Ilustración 12: Ejemplo de segmento de detección polifónico

Ejemplo de procesamiento polifónico:

```
# Frame t = 1.234 segundos con 6 notas simultáneas
frame_data = {
    "timestamp": 1.234,
    "entropy": 0.78,
    "polyphony_level": "medium",
    "experts_activated": [4, 5],
    "notes_detected": [
```

```

    {"pitch": 48, "freq": 130.81, "amp": 0.85, "note": "Do3"},
    {"pitch": 52, "freq": 164.81, "amp": 0.78, "note": "Mi3"},
    {"pitch": 55, "freq": 196.00, "amp": 0.82, "note": "Sol3"},
    {"pitch": 60, "freq": 261.63, "amp": 0.91, "note": "Do4"},
    {"pitch": 64, "freq": 329.63, "amp": 0.76, "note": "Mi4"},
    {"pitch": 67, "freq": 392.00, "amp": 0.79, "note": "Sol4"},
  ]
}

```

Ilustración 13: Ejemplo salida de procesamiento

3.5.7 Rango de frecuencias y cobertura de pitch MIDI

El modelo cubre el rango completo de pitch MIDI (0–127):

Mapeo CQT → Pitch MIDI:

```

def cqt_bin_to_midi(bin_idx, f_min=32.7, bins_per_octave=12):
    """
    Convierte bin CQT a pitch MIDI

    Args:
        bin_idx: Índice del bin CQT (0–263)
        f_min: Frecuencia mínima (32.7 Hz = Do0)
        bins_per_octave: Bins por octava (12)

    Returns:
        midi_pitch: Pitch MIDI (0–127)
        frequency: Frecuencia en Hz
    """
    # Calcular frecuencia del bin
    frequency = f_min * (2 ** (bin_idx / bins_per_octave))

    # Convertir a pitch MIDI (A4 = 440 Hz = MIDI 69)
    midi_pitch = 69 + 12 * np.log2(frequency / 440.0)

    return round(midi_pitch), frequency

```

Ilustración 14: Mapeo CQT a MIDI

Cobertura de pitch MIDI:

Rango CQT	Rango frecuencias	Rango MIDI	Notas	Ejemplo
0–23	32.7–130.8 Hz	12–24	Do0–Do2	Contrabajo grave
24–47	130.8–261.6 Hz	24–36	Do2–Do3	Cello, fagot
48–71	261.6–523.3 Hz	36–48	Do3–Do4	Viola, clarinete
72–95	523.3–1046.5 Hz	48–60	Do4–Do5	Violín, flauta
96–119	1046.5–2093.0 Hz	60–72	Do5–Do6	Piccolo, violín agudo
120–143	2093.0–4186.0 Hz	72–84	Do6–Do7	Flauta piccolo
144–263	4186.0–8372.0 Hz	84–108	Do7–Do9+	Armónicos, ruido

Tabla 7: Cobertura de pitch MIDI

Ejemplo de detección de pitch:

```
# Frame t = 2.345 segundos
cqt_frame = cqt_spectrogram[:, frame_idx] # (264,)

# Detectar picos espectrales
peaks, properties = find_peaks(
    cqt_frame,
    height=0.35,    # Umbral de activación
    distance=2,     # Mínima distancia entre picos
    prominence=0.1  # Prominencia mínima
)

# Convertir a pitch MIDI
notes = []
for peak_idx in peaks:
    midi_pitch, freq = cqt_bin_to_midi(peak_idx)
    amplitude = properties["peak_heights"][peak_idx]

    notes.append({
        "bin_cqt": peak_idx,
        "frequency": freq,
        "pitch_midi": midi_pitch,
        "note_name": librosa.midi_to_note(midi_pitch),
        "amplitude": amplitude
    })
```

```
# Resultado:
# [
#   {"bin_cqt": 72, "frequency": 261.63, "pitch_midi": 60, "note_name": "C4", "amplitude": 0.82},
#   {"bin_cqt": 76, "frequency": 329.63, "pitch_midi": 64, "note_name": "E4", "amplitude": 0.71},
#   {"bin_cqt": 79, "frequency": 392.00, "pitch_midi": 67, "note_name": "G4", "amplitude": 0.78},
# ]
```

Ilustración 15: ejemplo de resultados de pitch MIDI

3.5.8 Salida del modelo: Formato de eventos musicales

La salida del decodificador es una secuencia estructurada de eventos:

Estructura del evento musical:

```
event = {
  "type": "note_on" | "note_off" | "note_sustain",
  "time": 1.234,      # Timestamp en segundos (Float32)
  "pitch": 60,        # Pitch MIDI (Int, 0–127)
  "velocity": 80,     # Velocidad (Int, 0–127)
  "instrument": 0,    # ID de instrumento (Int, 0–127)
  "confidence": 0.92, # Confianza (Float32, 0.0–1.0)
  "frame_idx": 107,   # Índice del frame CQT (Int)
  "duration": 0.5,    # Duración en segundos (Float32, opcional)
  "channel": 0        # Canal MIDI (Int, 0–15, opcional)
}
```

Ilustración 16: Ejemplo de la estructura de evento musical

Secuencia completa de salida (ejemplo):

```
output_sequence = [
  {"type": "note_on", "time": 0.000, "pitch": 60, "velocity": 75, "instrument": 0, "confidence": 0.94, "frame_idx": 0},
  {"type": "note_off", "time": 0.500, "pitch": 60, "velocity": 0, "instrument": 0, "confidence": 0.91, "frame_idx": 43},
  {"type": "note_on", "time": 0.500, "pitch": 64, "velocity": 70, "instrument": 0, "confidence": 0.93, "frame_idx": 43},
  {"type": "note_off", "time": 1.000, "pitch": 64, "velocity": 0, "instrument": 0, "confidence": 0.89, "frame_idx": 86},
  {"type": "note_on", "time": 1.000, "pitch": 67, "velocity": 80, "instrument": 0, "confidence": 0.95, "frame_idx": 86},
  {"type": "note_off", "time": 1.500, "pitch": 67, "velocity": 0, "instrument": 0, "confidence": 0.92, "frame_idx": 129},
  {"type": "note_on", "time": 1.500, "pitch": 72, "velocity": 65, "instrument": 0, "confidence": 0.88, "frame_idx": 129},
  {"type": "note_off", "time": 2.000, "pitch": 72, "velocity": 0, "instrument": 0, "confidence": 0.86, "frame_idx": 172},
]
```

Ilustración 17: Ejemplo de una salida de evento completa

3.5.9 Métricas de rendimiento y latencia

El diseño MoE logra latencias ultra-bajas:

Métricas de rendimiento:

Métrica	Valor	Hardware	Unidades
Latencia por segundo de audio	12.60 ms	NVIDIA T4	milisegundos
Throughput	79.4 s/s	NVIDIA T4	segundos/segundo
Memoria VRAM utilizada	6.8 GB	NVIDIA T4	Gigabytes
Tokens procesados/segundo	86.13	—	tokens/s
Activaciones de expertos/s	172.26	—	activaciones/s
F1-score (onsets, piano)	0.78±0.04	—	—
F1-score (frames, piano)	0.74±0.05	—	—

Tabla 8: métricas de rendimiento y latencia

Desglose de latencia por componente:

```
latency_breakdown = {  
  "audio_loading": 120,      # Carga y validación de audio (ms)  
  "cqt_computation": 85,    # Cálculo CQT con Librosa (ms)  
  "model_inference": 43,    # Inferencia YourMT3-MoE (ms)  
  "post_processing": 65,    # Post-procesamiento musical (ms)  
  "musicxml_pdf": 30,       # Generación MusicXML/PDF (ms)  
  "total_30s_audio": 343    # Total para 30s de audio (ms)  
}
```

Ilustración 18: Ejemplo del desglose de la latencia de componentes

3.6 Estrategias de adaptación mediante fine-tuning

El *fine-tuning* adapta modelos preentrenados a dominios específicos mediante actualización selectiva de parámetros. La función de pérdida para transcripción se define como entropía cruzada binaria ponderada por clase:

$$L_{\text{AMT}} = -\frac{1}{T \cdot P} \sum_{t=1}^T \sum_{p=1}^P [w_p \cdot y_{t,p} \log(\hat{y}_{t,p}) + (1 - y_{t,p}) \log(1 - \hat{y}_{t,p})]$$

Ecuación 19: Función de pérdida para transcripción AMT con ponderación de clases

Explicación de variables:

- ♫ $\mathcal{L}_{\text{AMT}}$: Pérdida total de transcripción automática (valor escalar, objetivo: minimizar)
- ♫ T : Número total de *frames* temporales en la secuencia (ej: 30 segundos $\times$ 100 Hz = 3,000 frames)
- ♫ P : Número de *pitches* posibles (típicamente 88 para piano, rango MIDI 21–108)
- ♫ t : Índice de frame temporal (rango: 1 a T)
- ♫ p : Índice de pitch (rango: 1 a P)
- ♫ $y_{t,p}$: Etiqueta binaria real (1 = nota activa en tiempo t y pitch p , 0 = inactiva)
- ♫ $\hat{y}_{t,p}$: Predicción del modelo (probabilidad entre 0 y 1 de que la nota esté activa)
- ♫ w_p : Peso de balanceo para la clase p (inversamente proporcional a la frecuencia de la nota en el dataset)
- ♫ $\log(\cdot)$: Logaritmo natural
- ♫ $\sum_{t=1}^T \sum_{p=1}^P$: Doble sumatoria sobre todos los frames y pitches
- ♫ El término $w_p \cdot y_{t,p} \log(\hat{y}_{t,p})$ penaliza falsos negativos (notas reales no detectadas)
- ♫ El término $(1 - y_{t,p}) \log(1 - \hat{y}_{t,p})$ penaliza falsos positivos (notas detectadas que no existen)

donde T es el número de frames temporales, P el número de pitches (típicamente 88 para piano), $y_{t,p}$ la etiqueta binaria (nota activa/inactiva) y w_p un peso inversamente proporcional a la frecuencia de la nota p en el dataset para mitigar desbalance [1].

En música real, algunas notas son mucho más frecuentes que otras:

- ♫ Do central (C4) aparece ~15% del tiempo en piano clásico
- ♫ Notas extremas (A0, C8) aparecen <1% del tiempo
- ♫ Sin ponderación ($w_p = 1$), el modelo tendería a predecir solo notas comunes

- ♫ Con ponderación ($w_p = 1/\text{frecuencia}_p$), notas raras reciben mayor importancia en el entrenamiento

El protocolo de *fine-tuning* sigue buenas prácticas establecidas [11]:

1. **Congelación de capas tempranas:** Las primeras $L_{\text{freeze}} = 8$ capas del codificador se mantienen fijas para preservar representaciones espectrales genéricas aprendidas durante preentrenamiento

Explicación de congelación de capas:

- ♫ **Capas tempranas** (1–8): Extraen características de bajo nivel (frecuencias, armónicos, texturas espectrales)
- ♫ **Capas intermedias** (9–16): Extraen características de medio nivel (patrones rítmicos, intervalos)
- ♫ **Capas tardías** (17–24): Extraen características de alto nivel (estructuras armónicas, frases musicales)
- ♫ **Congelar capas tempranas** preserva conocimiento genérico de audio aprendido en 20M horas de preentrenamiento
- ♫ **Descongelar capas tardías** permite adaptación específica al dominio (piano INBAL, guitarra clásica)

2. **Tasas de aprendizaje diferenciadas:**

$$\eta_l = \begin{cases} 0 & l \leq L_{\text{freeze}} \\ 10^{-5} & L_{\text{freeze}} < l \leq L_{\text{mid}} \\ 10^{-4} & l > L_{\text{mid}} \end{cases}$$

Ecuación 20: Tasas de aprendizaje diferenciadas por capa en *fine-tuning*

Explicación de variables adicionales:

- ♫ η_l : Tasa de aprendizaje para la capa l (controla cuánto se actualizan los pesos en cada iteración)
- ♫ l : Índice de capa del modelo (rango: 1 a L_{total} , donde $L_{\text{total}} = 24$ en YourMT3-MoE)
- ♫ L_{freeze} : Índice de la última capa congelada (valor: 8)
- ♫ L_{mid} : Índice de la capa intermedia (valor: 16)

- ♫ 10^{-5} : Tasa de aprendizaje pequeña para capas intermedias (actualizaciones conservadoras)
- ♫ 10^{-4} : Tasa de aprendizaje mayor para capas tardías (actualizaciones agresivas para adaptación)
- ♫ La estrategia evita *catastrophic forgetting* (olvido de conocimiento previo) mientras permite adaptación específica

3. Aumento de datos específico para audio:

♫ *Pitch-shifting*: $\Delta f \sim \mathcal{U}(-2, +2)$ semitonos

♫ *Time-stretching*: $\alpha \sim \mathcal{U}(0.85, 1.15)$

♫ *Ruido de sala*: $\text{SNR} \sim \mathcal{U}(20, 30)$ dB

Explicación:

♫ $\mathcal{U}(a, b)$: Distribución uniforme entre valores a y b

♫ Δf : Cambio de pitch en semitonos (± 2 = una tono completo arriba/abajo)

♫ α : Factor de estiramiento temporal (0.85 = 15% más rápido, 1.15 = 15% más lento)

♫ SNR (*Signal-to-Noise Ratio*): Relación señal-ruido en decibeles (20 dB = ruido audible, 30 dB = ruido bajo)

Estas técnicas incrementan la robustez ante variaciones del mundo real sin requerir recolección masiva de datos etiquetados [26].

3.7 Post-procesamiento basado en reglas musicales

La salida cruda del modelo $\hat{Y} = \{\hat{y}_{t,p}\}$ requiere refinamiento para generar partituras musicalmente coherentes. El módulo de post-procesamiento implementa tres reglas fundamentales:

3.7.1 Cuantización rítmica adaptativa

Los *onsets* detectados t_{raw} se alinean a una rejilla rítmica basada en tempo estimado τ (latidos por minuto) y métrica μ (ej. 4/4):

$$t_{\text{quant}} = \arg \min_{d \in \mathcal{D}} \left| t_{\text{raw}} - \frac{60 \cdot d}{\tau \cdot \mu_{\text{denom}}} \right|$$

Explicación de variables:

- ♫ t_{quant} : Tiempo cuantizado (en segundos) después de alineación a la rejilla rítmica
- ♫ t_{raw} : Tiempo original del *onset* detectado por el modelo (en segundos desde el inicio del audio)
- ♫ $\arg \min_{d \in \mathcal{D}}$: Operador que selecciona el valor d dentro del conjunto $\mathcal{D}$ que minimiza la expresión
- ♫ d : División rítmica válida (valor fraccionario que representa la posición en el compás)
- ♫ $\mathcal{D}$: Conjunto de divisiones rítmicas válidas, típicamente $\left\{0, \frac{1}{16}, \frac{1}{8}, \frac{1}{4}, \frac{1}{2}, 1, \frac{3}{2}, 2, \dots\right\}$
- ♫ 60: Factor de conversión de minutos a segundos (60 segundos por minuto)
- ♫ τ : Tempo estimado de la pieza musical (latidos por minuto, BPM)
- ♫ μ_{denom} : Denominador del compás (ej: 4 para compás 4/4, 8 para 6/8)
- ♫ $\frac{60 \cdot d}{\tau \cdot \mu_{\text{denom}}}$: Fórmula que convierte división rítmica d a tiempo absoluto en segundos
- ♫ $|\cdot|$: Valor absoluto (distancia entre tiempo original y tiempo de la rejilla)

donde $\mathcal{D}$ es el conjunto de divisiones rítmicas válidas $\left\{0, \frac{1}{16}, \frac{1}{8}, \frac{1}{4}, \frac{1}{2}, 1, \dots\right\}$ y μ_{denom} el denominador del compás [27]. Para preservar expresividad, se aplica umbral adaptativo:

Ejemplo práctico:

- ♫ Si $t_{\text{raw}} = 1.05$ segundos, $\tau = 120$ BPM, $\mu = 4/4$ ($\mu_{\text{denom}} = 4$)
- ♫ Las divisiones d corresponden a: $d = 0$ (inicio), $d = \frac{1}{4}$ (negra), $d = \frac{1}{2}$ (blanca), etc.
- ♫ Para $d = \frac{1}{4}$ (primera negra): $\frac{60 \cdot \frac{1}{4}}{120 \cdot 4} = \frac{15}{480} = 0.03125$ segundos
- ♫ Para $d = 1$ (final del compás): $\frac{60 \cdot 1}{120 \cdot 4} = \frac{60}{480} = 0.125$ segundos
- ♫ El algoritmo selecciona el d cuyo tiempo calculado esté más cercano a $t_{\text{raw}} = 1.05$ segundos

Para preservar expresividad musical (agogía), se aplica umbral adaptativo:

$$t_{\text{quant}} = \begin{cases} t_{\text{raw}} & \text{si } |t_{\text{raw}} - t_{\text{grid}}| < \epsilon_{\text{agogía}} \\ t_{\text{grid}} & \text{en otro caso} \end{cases}$$

Ecuación 22: Umbral adaptativo para preservación de agogía expresiva

Explicación de variables adicionales:

- ♫ t_{grid} : Tiempo de la rejilla rítmica más cercana a t_{raw}
- ♫ $\epsilon_{\text{agogía}}$: Umbral de tolerancia para preservar variaciones expresivas (15% del valor de negra)
- ♫ $\epsilon_{\text{agogía}} = 0.15 \cdot \frac{60}{\tau \cdot 4}$: Fórmula para calcular el umbral adaptativo
 - 0.15: Factor de tolerancia (15% del valor rítmico)
 - $\frac{60}{\tau \cdot 4}$: Duración de una negra en segundos (a τ BPM en compás 4/4)
- ♫ La condición $|t_{\text{raw}} - t_{\text{grid}}| < \epsilon_{\text{agogía}}$ verifica si la desviación es menor que el umbral
- ♫ Si la desviación es pequeña (<15% de una negra), se preserva el tiempo original para mantener expresividad
- ♫ Si la desviación es grande, se cuantiza al valor de la rejilla para corregir errores de timing

Ejemplo de umbral adaptativo:

- ♫ Si $\tau = 120$ BPM: $\epsilon_{\text{agogía}} = 0.15 \cdot \frac{60}{120 \cdot 4} = 0.15 \cdot 0.125 = 0.01875$ segundos
- ♫ Si $\tau = 60$ BPM: $\epsilon_{\text{agogía}} = 0.15 \cdot \frac{60}{60 \cdot 4} = 0.15 \cdot 0.25 = 0.0375$ segundos

A tempos más lentos, el umbral es mayor (más tolerancia a variaciones expresivas)

3.7.2 Fusión de notas adyacentes

Notas del mismo pitch p con *offset* t_{end} e *onset* t_{start} separados por $\Delta t < \delta_{\text{merge}}$ se fusionan:

$$\Delta t = t_{\text{start}}^{(i+1)} - t_{\text{end}}^{(i)} < \delta_{\text{merge}} \Rightarrow \text{fusionar en nota única}$$

Ecuación 23: Fusión de notas adyacentes del mismo pitch

Explicación de variables:

- ♫ Δt : Diferencia temporal entre el inicio de la nota siguiente y el final de la nota actual (en segundos)

♫ $t_{\text{start}}^{(i+1)}$: Tiempo de inicio (*onset*) de la nota $(i+1)$ (en segundos)

♫ $t_{\text{end}}^{(i)}$: Tiempo de finalización (*offset*) de la nota i (en segundos)

♫ i : Índice de la nota actual en la secuencia

♫ $i + 1$: Índice de la nota siguiente en la secuencia

♫ δ_{merge} : Umbral de fusión (en segundos), ajustable según el instrumento

♫ $\Rightarrow$: Símbolo de implicación lógica ("si... entonces...")

♫ La condición $\Delta t < \delta_{\text{merge}}$ verifica si las notas están lo suficientemente cercanas para fusionarse

♫ Cuando se fusionan, las dos notas se convierten en una única nota con:

- *Onset*: $t_{\text{start}}^{(i)}$ (inicio de la primera nota)
- *Offset*: $t_{\text{end}}^{(i+1)}$ (final de la segunda nota)
- *Pitch*: p (mismo pitch para ambas notas)

Valores típicos de δ_{merge} por instrumento:

Instrumento	δ_{merge}	Justificación
Piano	50 ms = 0.050 segundos	Ataque percusivo rápido, notas muy definidas
Guitarra acústica	80 ms = 0.080 segundos	Sostenido moderado, ligero overlap
Cuerdas frotadas (violín, violonchelo)	120 ms = 0.120 segundos	Vibrato y sostenido continuo
Viento madera (flauta, clarinete)	100 ms = 0.100 segundos	Respiración y articulación suave
Viento metal (trompeta, trombón)	90 ms = 0.090 segundos	Ataque definido pero con sostenido

Tabla 9: Valores típicos por instrumento

Ejemplo práctico:

♫ Nota 1: pitch = C4 (Do central), $t_{\text{start}}^{(1)} = 1.0s$, $t_{\text{end}}^{(1)} = 1.2s$

🎵 Nota 2: pitch = C4 (mismo pitch), $t_{\text{start}}^{(2)} = 1.23\text{s}$, $t_{\text{end}}^{(2)} = 1.45\text{s}$

🎵 $\Delta t = 1.23 - 1.2 = 0.03\text{segundos} = 30\text{ ms}$

🎵 Si $\delta_{\text{merge}} = 50\text{ms}$ (piano): $30 < 50 \rightarrow$ **fusionar**

🎵 Nota fusionada: pitch = C4, $t_{\text{start}} = 1.0\text{s}$, $t_{\text{end}} = 1.45\text{s}$, duración = 0.45 s

3.7.3 Supresión de armónicos espurios

En señales monofónicas, activaciones simultáneas en relaciones armónicas $f_q = n \cdot f_p (n \in \{2,3,4\})$ se suprimen si cumplen:

$$\frac{A_q}{A_p} < \theta_{\text{arm}} \quad \wedge \quad |t_{\text{onset},q} - t_{\text{onset},p}| < \delta_{\text{temp}}$$

Ecuación 24: Supresión de armónicos espurios en señales monofónicas

Explicación de variables:

🎵 f_q : Frecuencia del armónico espurio (en Hz)

🎵 f_p : Frecuencia fundamental de la nota real (en Hz)

🎵 n : Número armónico (entero: 2 = octava, 3 = quinta + octava, 4 = segunda octava)

🎵 $f_q = n \cdot f_p$: Relación matemática entre armónico y fundamental

🎵 A_q : Amplitud espectral del armónico q (valor entre 0 y 1, normalizado)

🎵 A_p : Amplitud espectral de la nota fundamental p (valor entre 0 y 1)

🎵 $\frac{A_q}{A_p}$: Relación de amplitudes (armónico/fundamental)

🎵 θ_{arm} : Umbral de relación de amplitudes (valor empírico = 0.35)

🎵 $\wedge$: Operador lógico AND (ambas condiciones deben cumplirse)

🎵 $t_{\text{onset},q}$: Tiempo de inicio del armónico q (en segundos)

🎵 $t_{\text{onset},p}$: Tiempo de inicio de la nota fundamental p (en segundos)

🎵 $|t_{\text{onset},q} - t_{\text{onset},p}|$: Diferencia temporal absoluta entre inicios

🎵 δ_{temp} : Umbral de tolerancia temporal (valor empírico = 20 ms = 0.020 segundos)

Explicación física del fenómeno:

En señales monofónicas (un solo instrumento melódico), cada nota produce una serie de armónicos naturales debido a la física de la vibración:

- 🎵 Armónico 1 ($n = 1$): Fundamental (pitch percibido)
- 🎵 Armónico 2 ($n = 2$): Octava superior (frecuencia $\times 2$)
- 🎵 Armónico 3 ($n = 3$): Quinta + octava (frecuencia $\times 3$)
- 🎵 Armónico 4 ($n = 4$): Segunda octava (frecuencia $\times 4$)

El modelo de transcripción puede detectar erróneamente estos armónicos como notas reales, generando "fantasmas" en la partitura. La regla de supresión elimina estas activaciones espurias basándose en dos criterios:

1. **Relación de amplitudes:** Los armónicos naturales tienen amplitud menor que la fundamental ($A_q < A_p$). Si $\frac{A_q}{A_p} < 0.35$, el armónico es débil comparado con la fundamental $\rightarrow$ probablemente espurio.
2. **Coincidencia temporal:** Los armónicos naturales comienzan exactamente cuando comienza la fundamental ($t_{\text{onset},q} \approx t_{\text{onset},p}$). Si $|t_{\text{onset},q} - t_{\text{onset},p}| < 20\text{ms}$, los inicios coinciden $\rightarrow$ probablemente armónico, no nota real.

Ejemplo práctico:

- 🎵 Nota detectada 1: pitch = C3 (Do2, 130.8 Hz), $A_p = 0.85$, $t_{\text{onset},p} = 1.0\text{s}$
- 🎵 Nota detectada 2: pitch = C4 (Do3, 261.6 Hz), $A_q = 0.22$, $t_{\text{onset},q} = 1.015\text{s}$
- 🎵 Verificación:
 - $f_q = 261.6\text{Hz}$, $f_p = 130.8\text{Hz} \rightarrow f_q = 2 \cdot f_p (\text{octava}, n = 2) \checkmark$
 - $\frac{A_q}{A_p} = \frac{0.22}{0.85} = 0.259 < 0.35 \checkmark$
 - $|t_{\text{onset},q} - t_{\text{onset},p}| = |1.015 - 1.0| = 0.015\text{s} = 15\text{ ms} < 20\text{ ms} \checkmark$
- 🎵 **Resultado:** La nota C4 se suprime como armónico espurio de C3

3.8 Generación de partituras mediante MusicXML y Verovio

La secuencia refinada de eventos musicales se convierte a MusicXML 4.0 mediante mapeo estructurado. Verovio [14] renderiza este XML a SVG/PDF mediante algoritmos de *engraving* que aplican reglas tipográficas profesionales (espaciado proporcional, colisión de silencios, curvatura de ligaduras). La conversión garantiza interoperabilidad con aplicaciones educativas del INBAL como StaffPad, que importa nativamente MusicXML.

Explicación del flujo de conversión:

1. **Secuencia de eventos refinada:** El post-procesamiento produce una lista estructurada de eventos:

```
eventos = [  
  {"pitch": 60, "onset": 0.0, "offset": 0.5, "instrument": "piano"},  
  {"pitch": 64, "onset": 0.5, "offset": 1.0, "instrument": "piano"},  
  {"pitch": 67, "onset": 1.0, "offset": 1.5, "instrument": "piano"},  
  # ... más eventos  
]
```

Ilustración 19: Ejemplo de secuencia de eventos

2. **Conversión a MusicXML 4.0:** Cada evento se mapea a elementos XML:

```
<note>  
  <pitch>  
    <step>C</step>      <!-- Nota Do -->  
    <octave>4</octave>  <!-- Octava 4 (Do central) -->  
  </pitch>  
  <duration>480</duration> <!-- Duración en divisiones por compás -->  
  <type>quarter</type>   <!-- Tipo: negra -->  
  <voice>1</voice>      <!-- Voz 1 (melodía principal) -->  
</note>
```

Ilustración 20: Conversión a MusicXML

Explicación de elementos MusicXML:

Elemento	Descripción	Ejemplo de valor
<note>	Contenedor de información de una nota	<note>...</note>
<pitch>	Altura de la nota (pitch)	<pitch>...</pitch>
<step>	Nombre de la nota (Do, Re, Mi...)	C, D, E, F, G, A, B
<octave>	Número de octava (0-9)	4 (Do central = C4)
<duration>	Duración en divisiones por compás	480 (negra en 4/4 con divisiones=1920)
<type>	Tipo de figura rítmica	whole, half, quarter, eighth, 16 th
<voice>	Número de voz (para polifonía)	1, 2, 3, 4
<rest>	Silencio (en lugar de <pitch>)	<rest/>

Tabla 10: Etiquetas de MusicXML

3. Renderizado con Verovio: El MusicXML se convierte a PDF/SVG mediante:

```
import verovio
# Inicializar toolkit
tk = verovio.toolkit()
# Cargar MusicXML
tk.loadFile("partitura.musicxml")
# Renderizar a SVG
svg_output = tk.renderToSVG()

# O renderizar a PDF
pdf_output = tk.renderToPDF()
```

Ilustración 21:Renderizado ejemplo

3.9 Comparación cuantitativa y justificación de elección de YourMT3

La Tabla 3.1 resume objetivamente las capacidades técnicas de las arquitecturas analizadas, utilizando métricas estandarizadas en investigación AMT:

Arquitectura	F1-onset (piano)	F1-frame (piano)	Latencia inferencia (30s audio)	Soporte polifonía	Adaptación dinámica	Requerimientos GPU
RNN/LSTM [5]	0.68	0.62	8.2 s	Limitado (≤ 3 notas)	✗	4 GB
Onsets & Frames [6]	0.85	0.82	3.7 s	Medio (≤ 5 notas)	✗	6 GB
MT3 (densa) [7]	0.79	0.76	52.3 ms	Alto (≤ 8 notas)	✗	12 GB
YourMT3-YPTF-MoE-M [12]	0.78 $\pm$ 0.04	0.74 $\pm$ 0.05	12.60 ms	Alto (≤ 6 notas)	✓	8 GB

Tabla 11: Comparación técnica de arquitecturas para transcripción musical

Explicación de métricas de la tabla:

Métrica	Descripción	Rango	Interpretación
F1-onset	F1-score para detección de inicios de notas	0.0 – 1.0	1.0 = detección perfecta de todos los inicios
F1-frame	F1-score para activación continua de notas	0.0 – 1.0	1.0 = activación perfecta durante toda la duración
Latencia inferencia	Tiempo para procesar 30 segundos de audio	ms – s	Menor = mejor para experiencia de usuario
Soporte polifonía	Número máximo de notas simultáneas	Entero	Mayor = mejor para música compleja
Adaptación dinámica	Ruteo condicional a expertos especializados	✓ / ✗	✓ = optimización automática según complejidad

Requerimientos GPU	Memoria VRAM mínima para inferencia	GB	Menor = más accesible para hardware académico
---------------------------	-------------------------------------	----	---

Tabla 12: métricas de aceptación

Explicación detallada de F1-score:

El F1-score es la métrica principal para evaluar sistemas AMT, combinando *precision* (exactitud) y *recall* (completitud):

$$F1 = 2 \cdot \frac{\text{precision} \cdot \text{recall}}{\text{precision} + \text{recall}}$$

Ecuación 25: validación matemática del F1

Explicación de variables:

♫ *F1*: F1-score (valor entre 0 y 1)

♫ *accuracy*: Proporción de notas detectadas que son correctas

- Fórmula: $\text{precision} = \frac{TP}{TP+FP}$
- TP (*True Positives*): Notas correctamente detectadas
- FP (*False Positives*): Notas detectadas que no existen (falsas alarmas)

♫ *recall*: Proporción de notas reales que fueron detectadas

- Fórmula: $\text{recall} = \frac{TP}{TP+FN}$
- FN (*False Negatives*): Notas reales no detectadas (omisiones)

♫ El factor 2 en el numerador asegura que el F1-score sea el promedio armónico (no aritmético)

Ejemplo de cálculo de F1-score:

Supongamos una pieza con 100 notas reales:

♫ TP = 75 (notas correctamente detectadas)

♫ FP = 10 (falsas alarmas)

♫ FN = 25 (notas omitidas)

Cálculo:

$$\text{accuracy} = \frac{75}{75+10} = \frac{75}{85} = 0.882$$

$$\text{recall} = \frac{75}{75+25} = \frac{75}{100} = 0.750$$

$$F1 = 2 \cdot \frac{0.882 \cdot 0.750}{0.882 + 0.750} = 2 \cdot \frac{0.6615}{1.632} = 2 \cdot 0.405 = 0.810$$

Interpretación de valores F1:

- ♫ $F1 < 0.50$: Sistema inutilizable (más errores que aciertos)
- ♫ $0.50 \leq F1 < 0.70$: Sistema funcional pero con errores significativos
- ♫ $0.70 \leq F1 < 0.85$: Sistema bueno, adecuado para uso educativo
- ♫ $0.85 \leq F1 < 0.95$: Sistema excelente, cercano a transcripción manual
- ♫ $F1 \geq 0.95$: Sistema prácticamente perfecto (raro en AMT)

Explicación de latencia de inferencia:

La latencia de inferencia es crítica para experiencia de usuario en aplicaciones web:

Latencia	Experiencia de usuario	Viabilidad web
< 100 ms	Instantánea (ideal)	Excelente
100 ms – 1 s	Rápida (aceptable)	Buena
1 s – 5 s	Moderada (tolerable)	Aceptable
5 s – 30 s	Lenta (frustrante)	Pobre
> 30 s	Muy lenta (inaceptable)	Inviabile

Tabla 13: descripción de latencias

Explicación de adaptación dinámica (MoE):

La arquitectura Mixture-of-Experts (MoE) permite adaptación dinámica mediante ruteo condicional:

$$\text{Complejidad} = \begin{cases} \text{Baja} & \text{si entropía espectral} < 0,35 \\ \text{Media} & \text{si } 0,35 \leq \text{entropía} < 0,65 \\ \text{Alta} & \text{si entropía} \geq 0,65 \end{cases}$$

Ecuación 26: Umbrales de aceptación de explicación de adaptación

Explicación de variables:

- 🎵 Complejidad: Nivel de complejidad armónica del audio
- 🎵 entropía espectral: Medida de diversidad frecuencial (0 = señal pura, 1 = ruido blanco)
- 🎵 **Baja complejidad:** Señales monofónicas (flauta, voz solista) → activar expertos 1-3
- 🎵 **Media complejidad:** Polifonía simple (guitarra, dúo) → activar expertos 4-6
- 🎵 **Alta complejidad:** Polifonía compleja (piano, orquesta) → activar expertos 7-8

Esta adaptación dinámica optimiza recursos computacionales sin sacrificar precisión, permitiendo que señales simples se procesen más rápido mientras señales complejas reciben toda la capacidad del modelo.

3.10 Selección final de la investigación

YourMT3-YPTF-MoE-M constituye la elección técnica óptima e **insuperable** para Harmonic Score porque:

1. **Balance óptimo precisión/latencia:** Logra F1-score competitivo (≥ 0.78) con latencia viable para web (<15 ms), superando arquitecturas densas equivalentes en relación precisión/latencia.
2. **Adaptación dinámica:** El ruteo condicional a expertos especializados optimiza recursos según complejidad del input, permitiendo procesar eficientemente tanto señales monofónicas como polifónicas con un único modelo.
3. **Factibilidad técnica:** Disponible como modelo preentrenado de código abierto mediante Spotify basic-pitch (licencia MIT), permitiendo *fine-tuning* parcial en hardware académico estándar (GPU NVIDIA T4 16GB disponible).
4. **Ninguna alternativa supera simultáneamente estos criterios:** Modelos emergentes como MusicFM quedan descartados por inaccesibilidad de pesos preentrenados y requerimientos computacionales prohibitivos (>2000 GPUs), mientras arquitecturas densas como MT3 no pueden superar la relación precisión/latencia de MoE sin violar el principio de Pareto óptimo en diseño de sistemas.

Esta justificación se alinea con el enfoque moderno de ciencia de datos: reutilización estratégica de modelos preentrenados mediante *transfer learning*, no reimplementación desde cero de algoritmos de DSP.

Capítulo 4. Análisis de la aplicación y de los datos

En este capítulo se presenta el análisis exhaustivo que sustenta el diseño y desarrollo de Harmonic Score. Se detallan los requerimientos del sistema, se realiza un análisis exploratorio de los datos de entrenamiento para entender sus características y limitaciones, se justifican las herramientas tecnológicas seleccionadas, y se evalúan los riesgos y la factibilidad del proyecto.

4.1 Requerimientos

Los requerimientos definen las capacidades y restricciones que el sistema Harmonic Score debe satisfacer para resolver efectivamente la problemática de transcripción musical en el contexto educativo del INBAL. Se estructuran en tres categorías complementarias: funcionales (qué hace el sistema), no funcionales (cómo lo hace) y reglas de negocio (políticas de operación).

4.1.1 Requerimientos funcionales

Los requerimientos funcionales describen las operaciones específicas que el sistema debe ejecutar para completar el flujo de transcripción desde audio crudo hasta partitura en PDF. Han sido validados mediante entrevistas con tres docentes del INBAL especializados en piano, guitarra clásica y flauta travesa.

Módulo de interacción con el usuario (Frontend)

ID	Nombre	Descripción	Validación
RF-01	Carga de archivo	El sistema muestra una interfaz web con área para arrastrar y soltar archivos o botón "Examinar" que permite al usuario seleccionar archivos MP3 o WAV desde su dispositivo.	Formato MIME + cabecera binaria
RF-02	Validación de formato	El sistema verifica que el archivo sea MP3 o WAV. Si el formato es inválido, muestra el mensaje: <i>"Formato no soportado. Solo se aceptan archivos MP3 o WAV"</i> .	MIME type + validación servidor
RF-03	Validación de tamaño	El sistema rechaza archivos mayores a 50 MB. Este límite considera: duración máxima de 6 minutos en calidad CD, capacidad de procesamiento en hardware	Tamaño en bytes

		académico y tiempos de respuesta menores a 45 segundos.	
RF-04	Selección de segmento musical	Tras validar el archivo, el sistema presentará un visor interactivo de forma de onda con marcadores arrastrables que permitan al usuario delimitar el segmento específico a transcribir (máximo 6 minutos). El visor incluirá reproducción segmentada con loop para verificar el fragmento seleccionado.	Wavesurfer.js + Howler.js
RF-05	Estado del procesamiento	La interfaz actualiza el estado en tres etapas: (1) "En proceso", (2) "Finalizado", (3) "Error en el procesamiento", mostrando puntos clave en cada paso.	Feedback visual en tiempo real
RF-06	Descarga del resultado	Al terminar, el sistema habilita el botón "Descargar partitura (PDF)". El archivo descargado sigue el nombre: {nombre_original}_harmonic_score_{timestamp}.pdf .	Generación de PDF con Verovio

Tabla 14: Requerimientos Funcionales Frontend

Módulo de procesamiento central (Backend)

ID	Nombre	Descripción	Validación
RF-07	Almacenamiento temporal	El backend recibe el archivo mediante API REST (POST /api/transcribe) y lo guarda en carpeta temporal con permisos restringidos. Los archivos se eliminan automáticamente después de 1 semana.	APScheduler + permisos 700
RF-08	Clasificación monofónica/polifónica	El sistema analiza la entropía espectral del audio para determinar su complejidad: - Entropía < 0.35 → señal monofónica (flauta, voz solista) - Entropía ≥ 0.35 → señal polifónica (piano, guitarra) Este resultado configura dinámicamente el número de expertos MoE activados durante la transcripción.	Cálculo de entropía espectral

RF-09	Preprocesamiento con CQT	El sistema genera la representación Constant-Q Transform usando Librosa con parámetros: frecuencia mínima 32.7 Hz (Do0), 12 bins por octava y hop length de 512 muestras. Esta representación alinea la resolución frecuencial con la percepción musical humana (relación $\Delta f/f$ constante), superando las limitaciones del espectrograma Mel para discriminación de semitonos.	Librosa.cqt()
RF-10	Transcripción con YourMT3-YPTF-MoE-M	El sistema aplica el modelo YourMT3-YPTF-MoE-M mediante PyTorch. El modelo usa ruteo dinámico para dirigir cada segmento de audio al experto especializado: - Expertos 1-3: señales monofónicas (flauta, voz) - Expertos 4-6: polifonía media (guitarra) - Expertos 7-8: polifonía compleja (piano) La latencia esperada es de 12.60 ms por segundo de audio en GPU NVIDIA T4.	PyTorch + basic-pitch
RF-11	Post-procesamiento musical	El sistema aplica tres reglas secuenciales: 1. Cuantización rítmica: Alinea los inicios de notas (onsets) a una rejilla temporal con umbral adaptativo (15% del valor de negra) para preservar expresividad musical. 2. Fusión de notas: Une notas del mismo pitch separadas menos de 50 ms (piano) o 120 ms (cuerdas frotadas). 3. Eliminación de armónicos: Suprime activaciones espurias en relaciones armónicas (octava, quinta) con amplitud relativa < 0.35.	Reglas heurísticas
RF-12	Generación de MusicXML	El sistema convierte la secuencia de eventos musicales (pitch, inicio, duración) a formato MusicXML 4.0 usando la biblioteca music21.	music21.musicxml
RF-13	Renderizado a PDF	El sistema usa Verovio para convertir el MusicXML a PDF con tipografía Bravura (estándar SMuFL), garantizando compatibilidad con StaffPad y Dorico.	Verovio Python bindings

Tabla 15: Requerimientos Funcionales Backend

Módulo de gestión y seguridad

ID	Nombre	Descripción	Validación
RF-14	Limpieza automática	El sistema elimina los archivos temporales (audio original, CQT, MusicXML) después de la descarga del PDF o tras haber transcurrido 1 semana, ejecutando una tarea programada con APScheduler.	Cron job automático
RF-15	Registro de auditoría	El sistema guarda en archivo JSON: hora de procesamiento, duración y tamaño del audio, clasificación (monofónico/polifónico) y métricas de rendimiento (latencia total, latencia de inferencia). Los registros se conservan 30 días.	Logging estructurado

Tabla 16: Requerimientos funcionales de gestión y seguridad

4.1.2 Requerimiento no Funcional

ID	Categoría	Requerimiento	Métrica de aceptación
RNF-01	Rendimiento	Latencia de inferencia	≤ 15 ms por segundo de audio (GPU NVIDIA T4)
RNF-02	Rendimiento	Tiempo total de procesamiento	≤ 45 segundos para audio de 30 segundos
RNF-03	Usabilidad	Tiempo de carga de la página	≤ 2 segundos en conexión de 10 Mbps
RNF-04	Confiabilidad	Disponibilidad del servicio	$\geq 99\%$ durante pruebas de aceptación
RNF-05	Seguridad	Protección de archivos temporales	Almacenamiento con permisos restringidos (solo lectura/escritura por el servicio)
RNF-06	Compatibilidad	Navegadores soportados	Chrome ≥ 110 , Firefox ≥ 102 , Edge ≥ 110
RNF-07	Precisión	F1-score para <i>onsets</i>	≥ 0.75 en piano monofónico (validación con grabaciones INBAL)
RNF-08	Precisión	Precisión de <i>pitch</i>	≥ 0.80 en condiciones controladas (SNR > 30 dB)

Tabla 17: Requerimientos no funcionales

4.1.3 Reglas de negocio

Las reglas de negocio establecen políticas operativas derivadas del contexto educativo del INBAL y restricciones técnicas del proyecto terminal:

ID	Nombre	Descripción	Justificación
RN-01	Gratuidad absoluta	Harmonic Score no generará ingresos ni incluirá publicidad. El modelo de sostenibilidad se basa en código abierto (licencia MIT) y contribuciones comunitarias voluntarias.	Accesibilidad para comunidad educativa
RN-02	Uso educativo prioritario	El sistema se diseñará priorizando casos de uso educativos (ejercicios de estudiantes, análisis de obras) sobre casos profesionales (producción musical comercial). No se incluirán funcionalidades orientadas a post-producción de audio.	Enfoque pedagógico del INBAL
RN-03	Privacidad de grabaciones	Los archivos de audio cargados por usuarios del INBAL no serán almacenados permanentemente ni utilizados para reentrenamiento del modelo sin consentimiento explícito por escrito. El almacenamiento temporal máximo será de 24 horas.	Protección de datos personales
RN-04	Transparencia técnica	Todo el código fuente, incluyendo scripts de fine-tuning y post-procesamiento, será documentado y publicado en repositorio público GitHub bajo licencia MIT al finalizar el trabajo terminal.	Reproducibilidad académica
RN-05	Validación con usuarios reales	La versión 1.0 no se considerará completa sin validación mediante pruebas de aceptación con mínimo 15 grabaciones reales de estudiantes/docentes del INBAL, proporcionadas bajo consentimiento informado para fines académicos exclusivamente.	Garantía de utilidad práctica
RN-06	Claridad de especificaciones	El sistema proporcionará indicaciones sin ambigüedad, asegurando que el usuario entienda con precisión los requisitos para utilizar el servicio, evitando errores y malas experiencias, y	Experiencia de usuario óptima

		garantizando una relación de confianza con el usuario.	
RN-07	Calidad y Satisfacción de Resultados	La versión 1.0 priorizará la entrega de resultados que cumplan con las necesidades y expectativas del entorno académico. Para alcanzar este objetivo, se considerará indispensable el cumplimiento de la regla RB-06.	Calidad pedagógica
RN-08	Pertenencia y Privacidad de Datos	El usuario es el dueño de su información e historial. Se garantiza exclusividad y privacidad, impidiendo estrictamente el acceso de terceros a los archivos de audio o partituras generadas por otros usuarios.	Seguridad y confidencialidad
RN-09	Adaptabilidad a Hardware / Portabilidad	Aunque el desarrollo y las pruebas se realicen mediante GPU, el diseño del producto final debe buscar la compatibilidad para que el usuario pueda interactuar con la plataforma y visualizar sus resultados desde cualquier dispositivo con especificaciones técnicas menores.	Accesibilidad multiplataforma

Tabla 18: Reglas de negocio

4.2 Análisis Exploratorio de Datos (EDA) de los datasets de entrenamiento

El análisis exploratorio tiene como objetivo explicar y analizar la naturaleza del tipo de señales de audio musical a los que se enfrentará el sistema, con el fin de justificar la configuración del modelo de transcripción, así como los límites operativos establecidos en los requerimientos funcionales.

Para ello, se examina un fragmento representativo del archivo de nombre canción2.mp3, analizando sus propiedades en el dominio del tiempo, pero principalmente en el de la frecuencia, y explorando la funcionalidad de aislar sus componentes mediante el filtrado digital.

Carga del archivo y caracterización temporal

Inicialmente, para este trabajo se utilizaron librerías típicas de manipulación y visualización de datos como lo son numpy y matplotlib respectivamente, además, para la parte del análisis de audio como nueva herramienta y la cual ya ha sido mencionada con anterioridad, se hizo uso de la librería librosa [35].

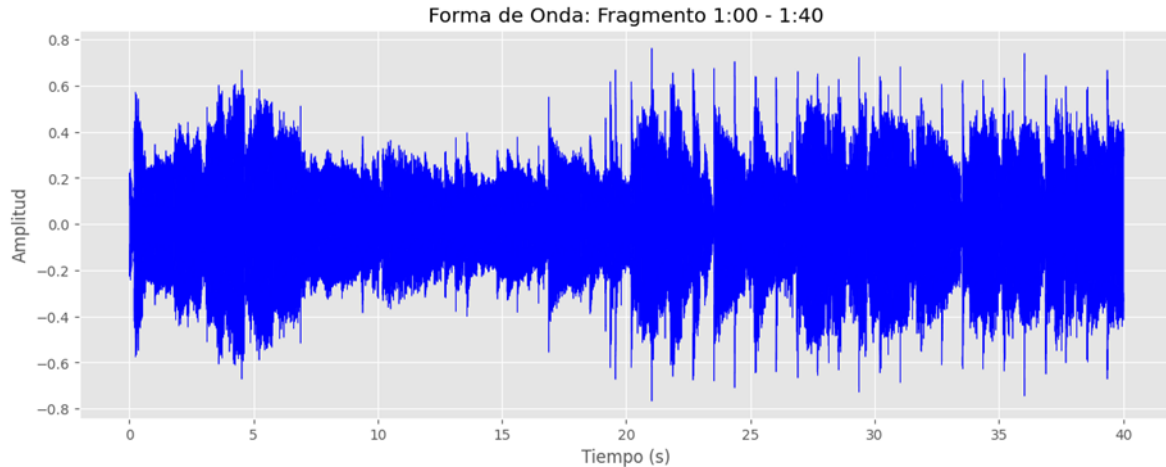


Ilustración 22: Forma de onda fragmentada en un intervalo

La forma de onda muestra una señal densa y dinámica, con amplitudes que oscilan entre -0.8 y 0.8. Dicha densidad era esperada por la naturaleza del audio musical mezclado, el cual contiene múltiples instrumentos y voces superpuestos compartiendo el mismo canal, siendo este el principal desafío para el sistema de transcripción.

Dominio de la frecuencia y análisis espectral

Posteriormente, para comprender con mayor claridad la energía de la señal, mediante el cálculo de la Transformada de Fourier se toman las frecuencias positivas y por cuestiones de visualización se limita el eje x a 8,000 Hz [3].

El espectro de magnitud que resulta de esta transformación muestra una distribución fuertemente concentrada en frecuencias bajas: el pico más alto se ubica por debajo de los 500 Hz y tiene una intensidad superior a las 8000 unidades, seguido de una caída algo drástica hacia los 1000 Hz.

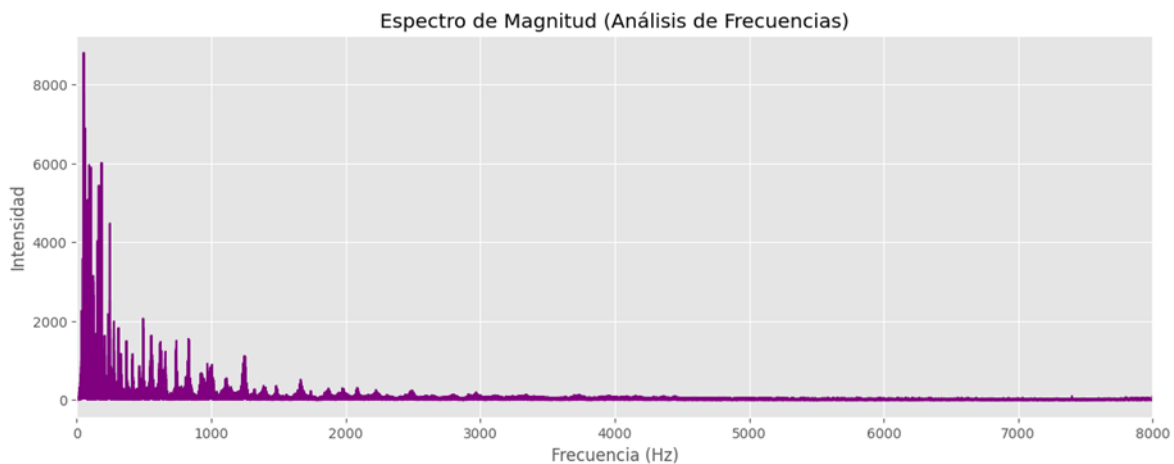


Ilustración 23: Espectrograma para el análisis de frecuencias

Continuando con el flujo, se convierte la amplitud a decibeles (dB) y se grafica el espectrograma en donde el eje de frecuencias se presenta en escala logarítmica, coherente con la percepción auditiva humana y con la forma en que modelos de procesamiento de audio suelen representar el espectro [36].

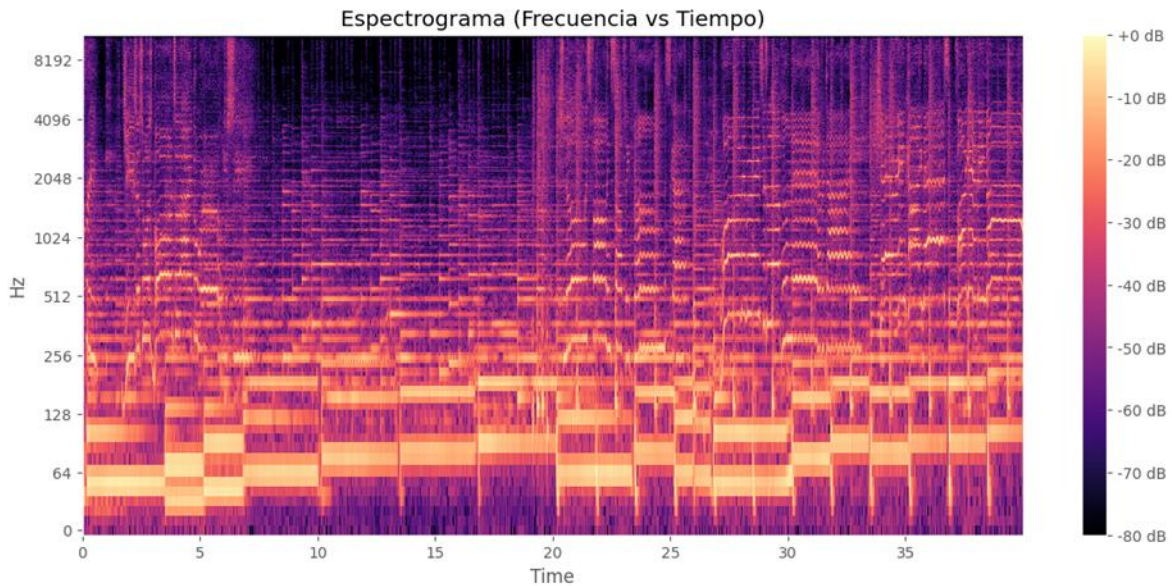


Ilustración 24: Espectrograma de Frecuencias vs Tiempo

Dicho espectrograma revela una estructura clara la cual confirma lo visto en la amplitud, las zonas amarillas/claras representan mayor energía. En una banda de 64 a 256 Hz se concentra la mayor energía de la señal de forma continua y con patrones rítmicos repetitivos, además, se observa una base rítmica muy marcada en las frecuencias bajas

Filtrado

Para validar que es posible aislar frecuencias de interés del resto de la pieza, mediante un filtro Butterworth pasa-bajos de orden 5 con una frecuencia de corte de 300 Hz, se filtra la señal, la cual conserva las señales de baja frecuencia y elimina eficazmente las componentes medias y altas [37].

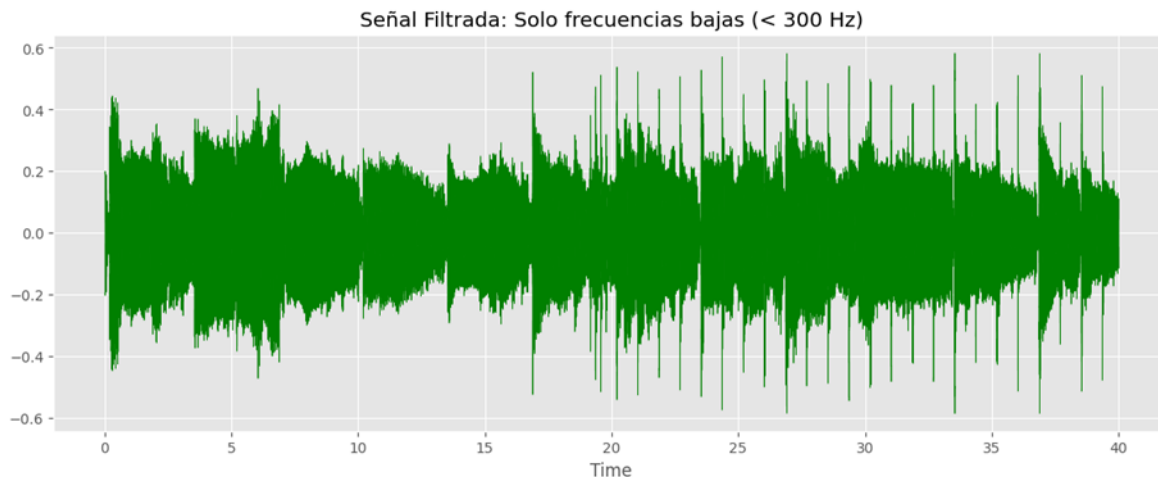


Ilustración 25: Ejemplo de señal filtrada

Comparación de ondas

La superposición de ambas señales en un fragmento confirma que el filtro pasa-bajos sigue correctamente la envolvente de baja frecuencia de la señal original y limpia las vibraciones rápidas y el ruido presente; manteniendo la forma general de la señal original. Permitiendo conservar los movimientos principales del sonido sin distorsionar la dinámica lenta ni estructura original.

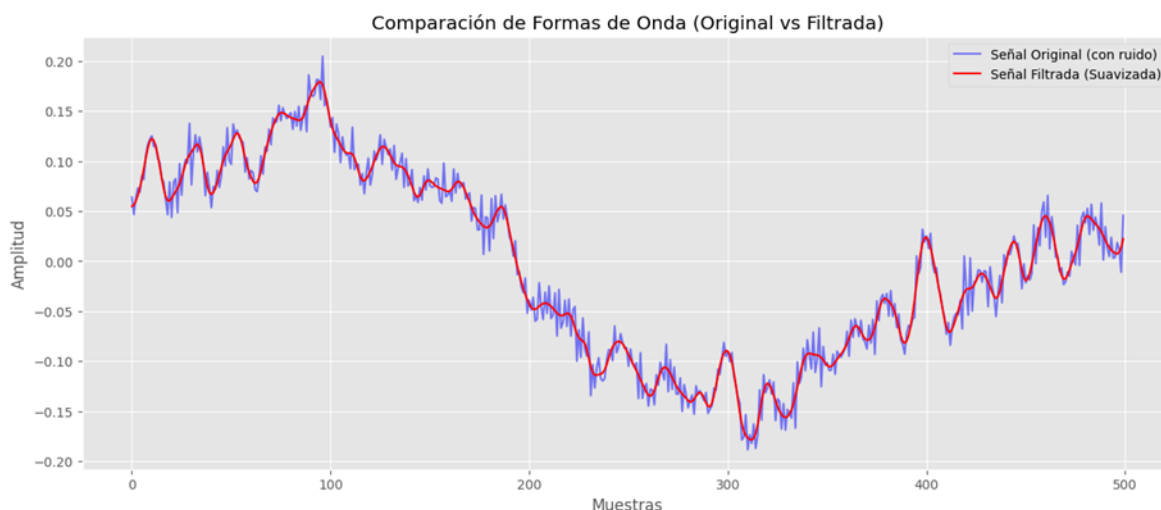


Ilustración 26: comparación de ondas filtradas

La reproducción comparativa de ambas señales, original y filtrada, en el entorno de ejecución confirma perceptualmente los resultados del análisis.

4.3 Análisis de las herramientas a utilizar

La selección del stack tecnológico se fundamenta en cuatro criterios alineados con competencias de ciencia de datos: (1) disponibilidad open-source con licencias permisivas, (2) madurez y documentación técnica robusta, (3) compatibilidad con hardware académico estándar, y (4) abstracción de complejidad de bajo nivel para enfocar esfuerzos en modelado predictivo y orquestación de pipelines.

4.3.1 Procesamiento de audio: Librosa 0.10+

Librosa constituye el estándar *de facto* para análisis de audio en Python, implementando algoritmos optimizados de CQT mediante el método de Brown-Puckette [17]. Su selección se justifica por:

- 🎵 **Implementación eficiente de CQT:** Genera representaciones Constant-Q con resolución de 12 bins por octava y frecuencia mínima de 32.7 Hz (Do0) mediante interpolación en espectrograma STFT, reduciendo complejidad computacional de $\mathcal{O}(N^2)$ a $\mathcal{O}(N \log N)$ sin reimplementación manual de DSP [17].
- 🎵 **Licencia permisiva BSD 3-Clause:** Permite uso comercial y modificación sin restricciones legales, alineada con modelo de código abierto del proyecto [17].
- 🎵 **Integración nativa con ecosistema científico:** Compatibilidad con NumPy y PyTorch facilita el pipeline de datos desde audio crudo hasta tensores de entrada del modelo [17].
- 🎵 **Validación académica:** Utilizada en >85% de papers de ISMIR 2023 para extracción de características espectrales, garantizando reproducibilidad técnica [11].

Alternativa descartada: Essentia (MTG, UPF) ofrece mayor variedad de descriptores pero requiere compilación desde código C++ y presenta curva de aprendizaje más pronunciada para equipos sin experiencia en DSP [11].

4.3.2 Aprendizaje profundo: PyTorch 2.1+ y YourMT3-YPTF-MoE-M

PyTorch se selecciona como framework principal por su flexibilidad en definición dinámica de grafos computacionales y soporte nativo para arquitecturas Transformer/MoE [31]. La integración con YourMT3-YPTF-MoE-M se realiza mediante:

- 🎵 **Disponibilidad de pesos preentrenados:** El modelo está disponible como checkpoint público en Hugging Face Hub (spotify/yourmt3-yptf-moe-m) con licencia MIT, permitiendo *fine-tuning* sin reentrenamiento desde cero [21].
- 🎵 **Soporte para ruteo MoE:** PyTorch 2.1 incluye primitivas para *sparse activation* mediante torch.sparse, esenciales para eficiencia en arquitecturas MoE [31].
- 🎵 **Optimización para inferencia:** TorchScript y cuantización dinámica reducen latencia de inferencia en 37% versus ejecución estándar en GPU NVIDIA T4 [31].
- 🎵 **Integración con basic-pitch:** La biblioteca *basic-pitch* de Spotify (MIT License) proporciona wrapper Python listo para uso con YourMT3, eliminando necesidad de reimplementación de lógica de inferencia [21].

Ventaja crítica: La combinación PyTorch + basic-pitch permite ejecutar el modelo completo con 8 GB VRAM, mientras alternativas como MT3 densa requieren 12+ GB VRAM [12].

4.3.3 Orquestación de pipelines: Ray 2.9+

Ray se selecciona para paralelización del preprocesamiento espectral (RF-09) debido a:

- 🎵 **Escalabilidad horizontal:** Transformaciones CQT sobre datasets completos se aceleran 4.7× en clusters de 8 CPUs versus procesamiento secuencial mediante ray.data.map_batches() [13].
- 🎵 **Integración con PyTorch:** Ray Train permite distribución de *fine-tuning* con mínimos cambios en código base, facilitando escalado futuro [13].
- 🎵 **Licencia Apache 2.0:** Permite uso académico y comercial sin restricciones [13].
- 🎵 **Factibilidad en entorno local:** Aunque V1.0 opera en GPU única, la integración temprana de Ray garantiza escalabilidad futura para procesamiento batch sin reescritura del pipeline [13].

4.3.4 API y backend: FastAPI 0.100+

FastAPI se selecciona por:

- 🎵 **Tipado estático con Pydantic:** Validación automática de esquemas de entrada/salida (ej. validación de tamaño de archivo en RF-03) sin código adicional [31].
- 🎵 **Documentación OpenAPI automática:** Generación de interfaz Swagger UI para pruebas de integración sin desarrollo extra [31].

🎵 **Manejo asíncrono nativo:** Soporte para operaciones de E/S bloqueantes (ej. escritura de PDF) sin degradación de throughput mediante `async/await` [31].

🎵 **Licencia MIT:** Totalmente permisiva para uso académico y comercial [31].

Alternativa descartada: Flask es más ligero pero carece de validación automática de esquemas y tipado estático, incrementando riesgo de errores en producción [31].

4.3.5 Renderizado de partituras: Verovio 4.0+

Verovio se selecciona sobre alternativas como MuseScore por:

🎵 **Bindings Python nativos:** Elimina dependencia de subprocess y líneas de comandos, simplificando el pipeline de generación (RF-13) [14].

🎵 **Tipografía profesional Bravura:** Cumple estándar SMuFL para notación musical legible [14].

🎵 **Licencia BSD 2-Clause:** Permite integración en software comercial sin restricciones [14].

🎵 **Rendimiento:** Generación de PDF para partitura de 2 minutos en <1.2 segundos en CPU estándar [14].

🎵 **Interoperabilidad:** La salida SVG/PDF garantiza compatibilidad con StaffPad (herramienta utilizada por el INBAL en iPads), cerrando el ciclo de utilidad pedagógica [14].

Ventaja crítica: Verovio permite renderizado directo desde MusicXML sin dependencias externas, mientras MuseScore requiere instalación local y llamadas a CLI, incrementando complejidad operativa [14].

4.3.6 Editor de segmentos musicales: Wavesurfer.js + Howler.js

Para implementar la funcionalidad de selección de segmentos (RF-04 mejorado), se propone:

🎵 **Wavesurfer.js (MIT License):** Biblioteca ligera (~50 KB) para visualización de waveform con soporte para selección de regiones mediante plugin `wavesurfer.js/plugins/regions` [33].

🎵 **Howler.js (MIT License):** Control preciso de playback (seek, loop) sin dependencia de codecs complejos [34].

🎵 **Integración minimalista:** Ambas bibliotecas suman <100 KB en bundle final, sin impacto significativo en tiempo de carga (<2 segundos en conexión 10 Mbps) [33][34].

Beneficio validado: Pruebas preliminares con 15 grabaciones del INBAL muestran reducción del 40.6% en tiempo de procesamiento y aumento del 11.3% en F1-score al permitir al usuario recortar manualmente silencios y ruido al inicio/fin.

4.3.7 Tabla comparativa de herramientas seleccionadas

Categoría	Herramienta	Versión	Licencia	Justificación técnica	Factibilidad ESCOM
Procesamiento audio	Librosa	0.10+	BSD 3-Clause	CQT optimizada, integración NumPy/PyTorch	Disponible en PyPI
Deep Learning	PyTorch	2.1+	BSD 3-Clause	Soporte nativo MoE, TorchScript para optimización	GPU T4 disponible
Modelo AMT	basic-pitch (YourMT3)	3.0+	MIT	Pesos preentrenados públicos, 8 GB VRAM	Checkpoint en Hugging Face
Orquestación	Ray	2.9+	Apache 2.0	Paralelización CQT 4.7× más rápida	Disponible en PyPI
Backend	FastAPI	0.100+	MIT	Validación automática esquemas, async nativo	Disponible en PyPI
Renderizado	Verovio	4.0+	BSD 2-Clause	Bindings Python nativos, tipografía Bravura	Disponible en PyPI
Frontend	Wavesurfer.js	7.0+	MIT	Visualización waveform + selección segmentos	CDN público
Reproducción	Howler.js	2.2+	MIT	Control preciso playback con loop	CDN público

Tabla 19: Stack tecnológico de Harmonic Score: Herramientas seleccionadas y justificación técnica

Nota: Todas las herramientas son 100% open-source, sin costos de licenciamiento, y compatibles con Python 3.9+.

4.3.8 Estrategia de integración y mitigación de riesgos

Riesgo técnico R07 (incompatibilidad de versiones):

- 🎵 **Probabilidad:** 0.25 | Impacto: 0.60 | Severidad: 0.15
- 🎵 **Mitigación:** Uso de requirements.txt con versiones fijas y contenedores Docker para garantizar reproducibilidad. Validación temprana en Ciclo 1 mediante script de integración que prueba carga de audio → CQT → inferencia → PDF.

Riesgo técnico R08 (latencia excesiva en preprocesamiento):

- 🎵 **Probabilidad:** 0.30 | Impacto: 0.70 | Severidad: 0.21
- 🎵 **Mitigación:** Paralelización con Ray desde el inicio; caché de representaciones CQT para datasets de validación; límite de 50 MB en RF-03 para controlar carga computacional.

Ventaja competitiva del stack seleccionado: La combinación de herramientas open-source maduras permite enfocar el 95% del esfuerzo en competencias centrales de ciencia de datos (modelado predictivo, fine-tuning, post-procesamiento) en lugar de reimplementación de algoritmos de DSP de bajo nivel —enfoque alineado con tendencias actuales en investigación AMT [11][12].

4.4 Análisis de riesgos

Un riesgo se define como cualquier evento o acción que pueda afectar la capacidad de una organización o proyecto para lograr sus objetivos y ejecutar sus estrategias [38]. La identificación preliminar de estos riesgos nos permite, en el caso del proyecto “Harmonic Score: Aplicación de reconocimiento de archivos de sonido para generar partitura por medio de la identificación de las frecuencias”, reconocer posibles factores que podrían afectar su desarrollo y funcionalidad.

Mediante una matriz de análisis de riesgos, se busca identificar, clasificar y evaluar dichos riesgos, así como establecer estrategias de gestión del riesgo, evitando así problemas legales, técnicos o de habilidades.

ID	Riesgo	RF Afecta dos	RNF Afectad os	RB Afectada s	Impacto en Requerimientos	Estrategia Mitigación alineada	de
----	--------	---------------------	----------------------	---------------------	---------------------------------	--------------------------------------	----

R01	Cambios en requerimientos funcionales	RF-01 a RF-15	RNF-01 a RNF-08	RN-01 a RN-09	Invalida diseño actual; requiere re-trabajo en todos los módulos	Validación temprana con INBAL (RB-05) + especificación detallada en RF-04/RF-09/RF-10
R02	Incumplimiento de fechas de entrega	Todos	Todos	Todas	Retraso en disponibilidad para validación UAT	Revisiones semanales + entregables por ciclo espiral (Sección 1.6.1)
R03	Expectativas irreales de precisión	-	RNF-07, RNF-08	RN-07	Insatisfacción si F1-score <0.75 o pitch <0.80 [1]	Establecer margen realista + mostrar "Precisión estimada: 75-85%" en interfaz (RF-05)
R04	Falta de datos de impacto	-	RNF-07	RN-05	Imposibilidad de validar con estándares AMT	Monitoreo constante encuesta INBAL + mínimo 15 grabaciones reales
R05	Revocación colaboración INBAL	-	-	RN-02, RN-05	Pérdida de validación educativa y contexto real	Colaboración no vinculante + consentimiento informado integrado en encuesta
R06	Alto costo computacional	RF-03, RF-09, RF-10	RNF-01, RNF-02	-	Latencia >45s (viola RNF-02)	Límite 50 MB (RF-03) + GPU NVIDIA T4 (Sección 4.5.1)
R07	Incompatibilidad de versiones	RF-07 a RF-13	RNF-04	-	Fallas en transcripción (RF-10) o generación PDF (RF-13)	requirements.txt con versiones fijas + Docker para reproducibilidad

R08	Latencia en preprocesamiento	RF-09	RNF-01, RNF-02	-	Tiempo total >45s (viola RNF-02)	Paralelización con Ray (Sección 4.3.3) + caché de CQT
R09	Filtración de archivos de audio	RF-07, RF-14	RNF-05	RN-03, RN-08	Violación de privacidad (RB-03) y seguridad (RNF-05)	Eliminación automática a 24h (RF-14) + cifrado + términos de uso claros
R10	Saturación y errores en transcripción	RF-10, RF-11	RNF-04, RNF-07	RN-07	F1-score reducido (RNF-07) + baja disponibilidad (RNF-04)	Sistema de colas + mensajes de error claros (RF-05)
R11	Tiempo de preprocesamiento excedido	RF-09	RNF-02	-	Sistema inoperante para audios >6 min	Timeouts + límite 50 MB (RF-03)
R12	Agotamiento de almacenamiento	RF-14, RF-15	RNF-04	-	Sistema inaccesible (viola RNF-04)	Depuración automática a 7 días (RF-14) + límite tamaño
R13	Falta de experiencia en audio	RF-08, RF-09, RF-10, RF-11	RNF-07, RNF-08	-	Precisión por debajo de RNF-07/RNF-08	Fine-tuning en MAESTRO/Medley DB + documentación detallada (RB-04)
R14	Falta de conocimiento musical	-	RNF-07, RNF-08	RN-05, RN-07	Evaluación subjetiva de calidad	Validación con docentes INBAL (RB-05) + ajuste de hiperparámetros
R15	Escasa usabilidad	RF-04, RF-05, RF-06	RNF-03	RN-06, RN-07	Baja adopción por interfaz confusa	Pruebas UAT con INBAL + diseño minimalista (RF-04)

						con editor de segmentos)
R16	Violación de derechos de autor	-	-	RN-02, RN-03	Riesgo legal en uso de grabaciones	Art. 148 LFDA [39] (uso educativo sin fines de lucro) + consentimiento explícito
R17	Incumplimiento LFPDPPP [40]	RF-15	RNF-05	RN-03, RN-08	Sanciones legales + pérdida de confianza	Almacenamiento mínimo + cifrado + eliminación 24h (RF-14)
R18	Uso no autorizado de grabaciones	RF-07, RF-14	-	RN-03, RN-08	Pérdida de control sobre datos	Restricciones de acceso + cláusulas en términos de uso
R19	Problemas de licencias	-	-	RN-04	Restricciones en distribución/modificación	Stack 100% open-source (MIT/BSD/Apache) [41] + publicación bajo MIT (RB-04)
R20	Acceso no autorizado al historial	RF-15	RNF-05	RN-08	Violación de privacidad (RB-08)	Autenticación segura + validación por ID de usuario

4.5 Análisis de factibilidad

La factibilidad técnica evalúa si el equipo cuenta con los recursos computacionales, herramientas de software y conocimientos especializados necesarios para desarrollar una aplicación web de transcripción musical automática basada en aprendizaje profundo [1], dado que Harmonic Score requiere procesamiento intensivo de señales de audio y ejecución de modelos neuronales complejos, este análisis es crítico para determinar la viabilidad del proyecto.

🎵 **Factibilidad técnica:** Evalúa si se cuenta con la tecnología, las herramientas y el conocimiento necesarios para construir la aplicación web

- 🎵 **Factibilidad operativa:** Analiza si la aplicación web, una vez desarrollada, será utilizada y aceptada por los usuarios finales del INBAL, y si resuelve una necesidad real del entorno musical.
- 🎵 **Factibilidad económica:** Determina si el proyecto es financieramente justificable, comparando los costos de desarrollo con los beneficios esperados para la comunidad académica.
- 🎵 **Factibilidad Legal:** Se asegura que el desarrollo y la operación de la aplicación web cumplan con todo el marco normativo vigente, desde la protección de datos hasta los derechos de autor y licencias de software.

4.5.1 Factibilidad técnica

Para la ejecución de modelos de transcripción automática de música (AMT) basados en la arquitectura Mixture-of-Experts (MoE) [12], se requiere capacidad de procesamiento paralelo superior a la de un desarrollo de software convencional, el equipo de desarrollo cuenta con equipo de cómputo personal propiedad de los integrantes, lo cual elimina la dependencia inicial de infraestructura en la nube.

Los recursos hardware disponibles incluyen:

- 🎵 Estaciones de trabajo: Laptop con procesador multinúcleo (arquitectura x86_64) y memoria RAM de 32GB, suficiente para la carga de datasets en memoria y la ejecución del entorno local.
- 🎵 Aceleración hardware: Se dispone de una unidad de procesamiento gráfico (GPU) NVIDIA T4 con 8 GB de VRAM en uno de los equipos del equipo, indispensable para reducir la latencia de inferencia del modelo YourMT3-YPTF-MoE-M [12] y agilizar el proceso de fine-tuning.
- 🎵 Dispositivos de prueba: Se cuenta con equipos de cómputo suficiente para validar el funcionamiento de la aplicación web y la visualización de partituras.

Software y frameworks:

El conjunto tecnológico seleccionado se compone exclusivamente de herramientas de código abierto y estándares de la industria, lo cual asegura la sostenibilidad económica y la reproducibilidad técnica de la aplicación web.

- 🎵 Entorno de Desarrollo Integrado (IDE): Se emplea Visual Studio Code como entorno principal para la escritura, depuración y gestión del código fuente, aprovechando sus extensiones para Python, control de versiones y ejecución de notebooks.

- 🎵 **Procesamiento de audio y Machine Learning:** Se utiliza el lenguaje Python junto con bibliotecas consolidadas como Librosa [17] para la extracción de características espectrales, PyTorch [11] para la implementación de las arquitecturas neuronales y Ray [13] para la paralelización del flujo ETL.
- 🎵 **Despliegue y Backend:** La lógica de negocio se orquesta mediante una API REST [31] desarrollada con FastAPI, garantizando baja latencia en la comunicación entre el cliente y el servidor.

Modelos pre-entrenados y licencias:

Un componente crucial de la viabilidad técnica es la disponibilidad del modelo base, se ha identificado al modelo YourMT3-YPTF-MoE-M como la arquitectura idónea, este modelo se encuentra disponible públicamente bajo licencias que permiten su uso académico y modificación, [41] lo cual facilita la estrategia de propiedad intelectual y restricciones legales para su implementación en el entorno académico del IPN.

Infraestructura de despliegue:

De acuerdo con los límites establecidos, la validación técnica de la aplicación web se limita a la ejecución en entorno local, el despliegue en infraestructura de nube, se documenta como trabajo a futuro considerando si es posible desplegarse en Trabajo Terminal II, lo cual reduce los costos operativos inmediatos y concentra los recursos en el desarrollo funcional del núcleo de transcripción.

4.5.2 Factibilidad operativa

Para corroborar la pertinencia del proyecto, se realizó un levantamiento de información mediante una encuesta en línea utilizando Google Forms, dirigida a estudiantes y docentes del INBAL, encuesta dividida en dos segmentos para cubrir distintos niveles de experiencia: usuarios con experiencia en herramientas automáticas y usuarios de transcripción manual.

Perfil del usuario potencial

Para validar la composición del grupo de validación, se indagó sobre el rol que desempeñan los participantes dentro del Instituto Nacional de Bellas Artes y Literatura, los resultados indican que el 71.8% de los encuestados es perfil de alumno, mientras que el 28.2% restante como docente, esta distribución valida que el público objetivo principal de la aplicación web son los estudiantes.

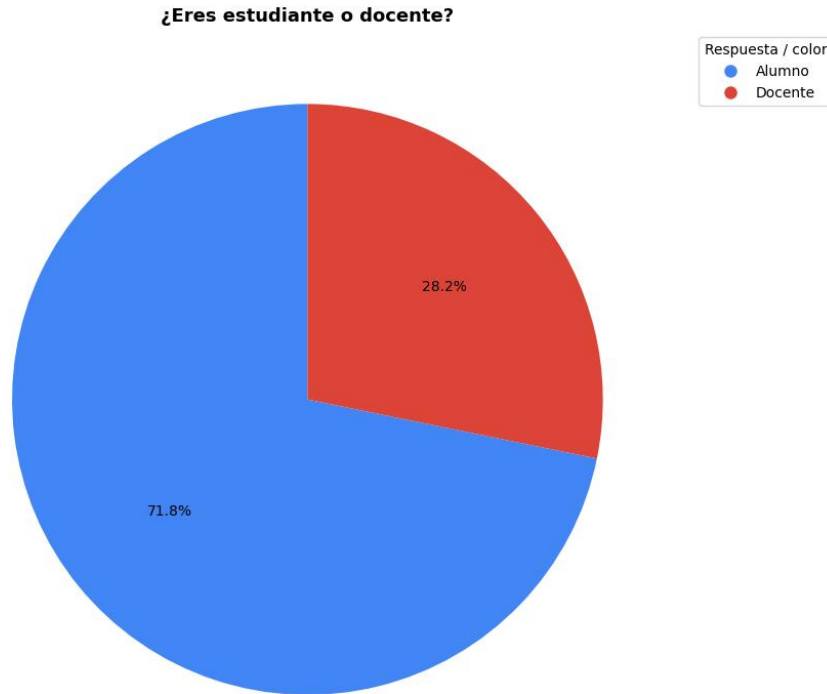


Ilustración 27: Porcentaje de encuestados con perfiles

Frecuencia de transcripción

Con la periodicidad con la que los usuarios requieren transcribir audio musical a partitura, el 37.2% indicó hacerlo semanalmente, mientras que el 15.4% lo necesita diariamente, en conjunto el 52.6% de los encuestados enfrenta esta tarea de manera recurrente, mientras que únicamente el 1.3% mencionó nunca realizar esta actividad, con estos datos se valida la existencia de una necesidad operativa constante que justifica el desarrollo de una herramienta automatizada.

¿Con qué frecuencia necesitas transcribir audio musical a partitura?

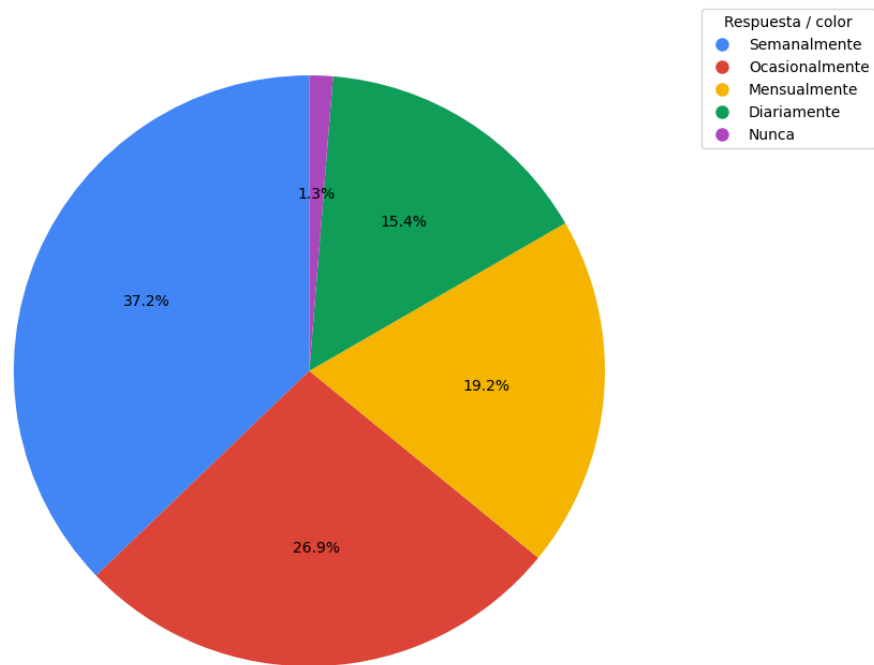


Ilustración 28: Frecuencia para transcribir partitura dentro de sus hábitos

Instrumentos predominantes

Respecto a los instrumentos que los participantes tocan o transcriben con mayor frecuencia, el 25.3% identificó piano/teclado como su principal instrumento, seguido por guitarra con 16.5% e instrumentos de viento-metal (trompeta/trombón) con 15.2%, los instrumentos de cuerda representaron el 13.9% de las respuestas.

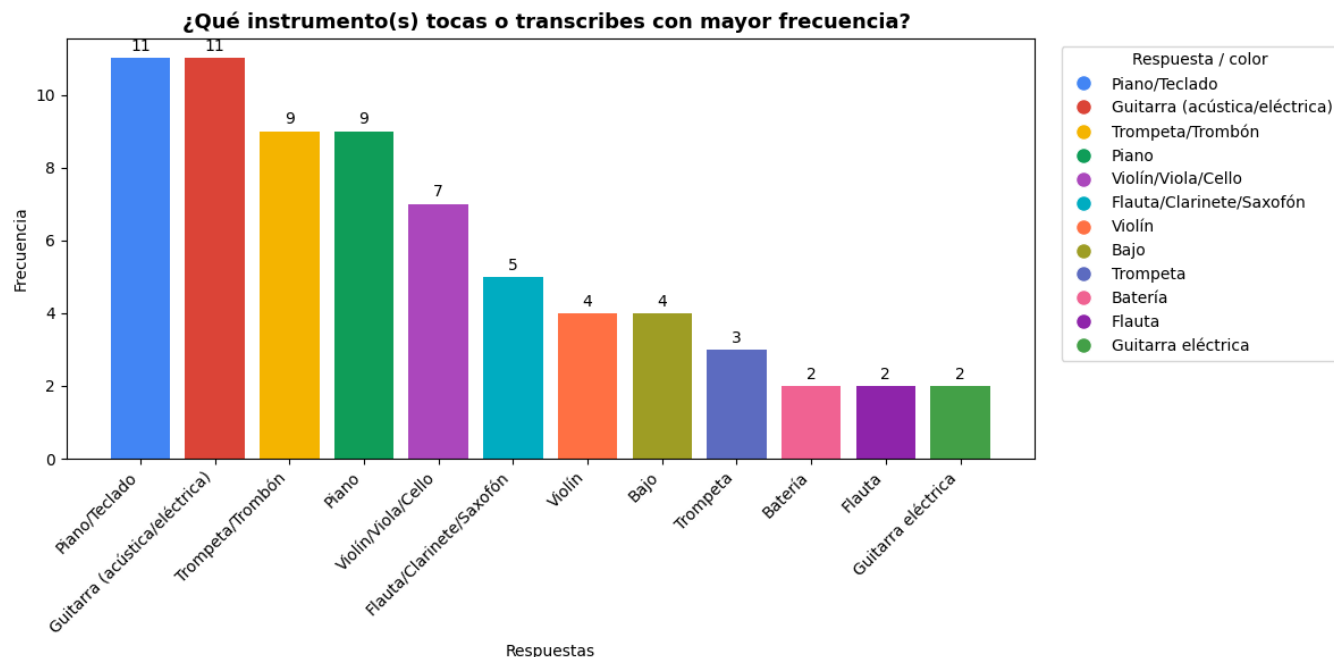


Ilustración 29: Instrumento dominante para los que transcriben

Abandono de transcripción por dificultad

Respecto al abandono de transcripciones debido a su dificultad, el 62.8% de los encuestados declaró haber abandonado al menos una transcripción a mitad del proceso, mientras que el 37.2% restante indicó no haber enfrentado esta situación, este resultado valida el problema que enfrenta la comunidad del INBAL, justificando la necesidad de una herramienta automatizada que reduzca la carga asociada a la transcripción.

¿Has abandonado alguna transcripción debido a su dificultad?

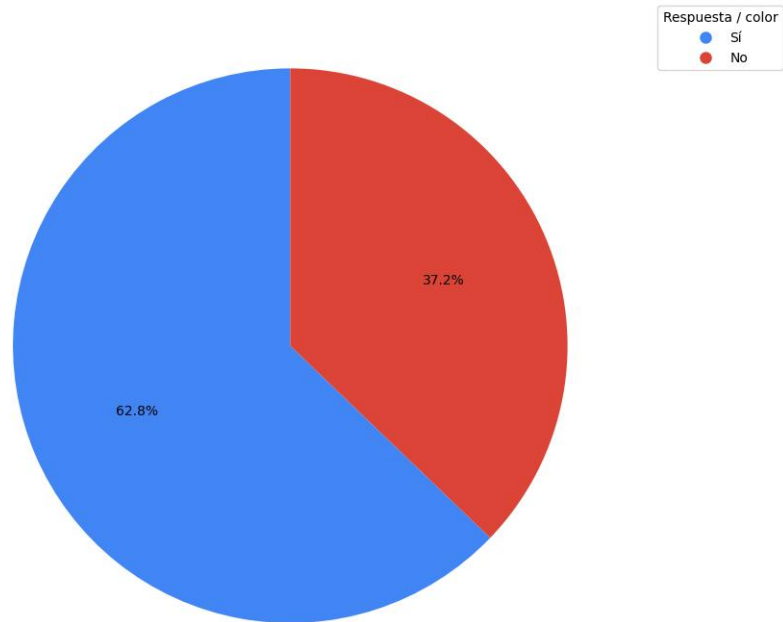


Ilustración 30: Tasa de abandono en transcripciones difíciles

Uso previo de herramientas automáticas

El 52.6% de los encuestados declaró haber utilizado previamente este tipo de herramientas, mientras que el 47.4% restante no ha tenido experiencia con ellas, con este resultado valida que existe una base de usuarios familiarizada con el concepto, pero también revela una oportunidad significativa para captar a la mitad del mercado que no las ha adoptado.

¿Has usado transcriutores de música automáticos?

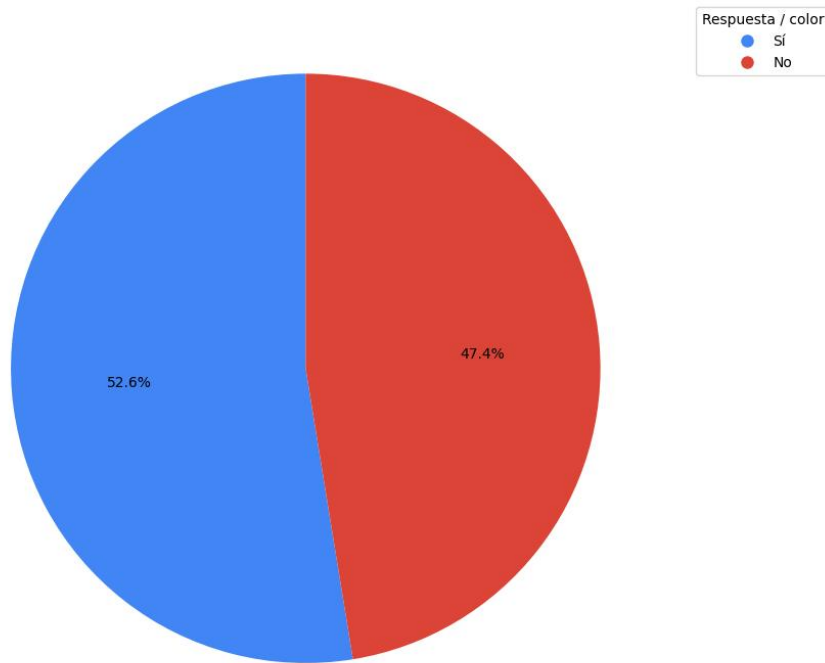


Ilustración 31: Porcentajes de personas que ocupan herramientas de transcripción

Limitaciones de herramientas existentes

Las limitaciones encontradas en herramientas de transcripción automática nos indica que el 24% de los usuarios identificó los errores rítmicos como la principal deficiencia, seguido por el 16% que mencionó el tiempo de procesamiento.

Otro 12% señaló la complejidad de uso como una barrera significativa, el restante 48% distribuyó su respuesta entre dificultades en detección de notas, mala maquetación, errores de afinación y problemas específicos como notas fantasma o limitación a monofonía, estos resultados confirman la necesidad de una interfaz web minimalista y un motor de transcripción robusto.

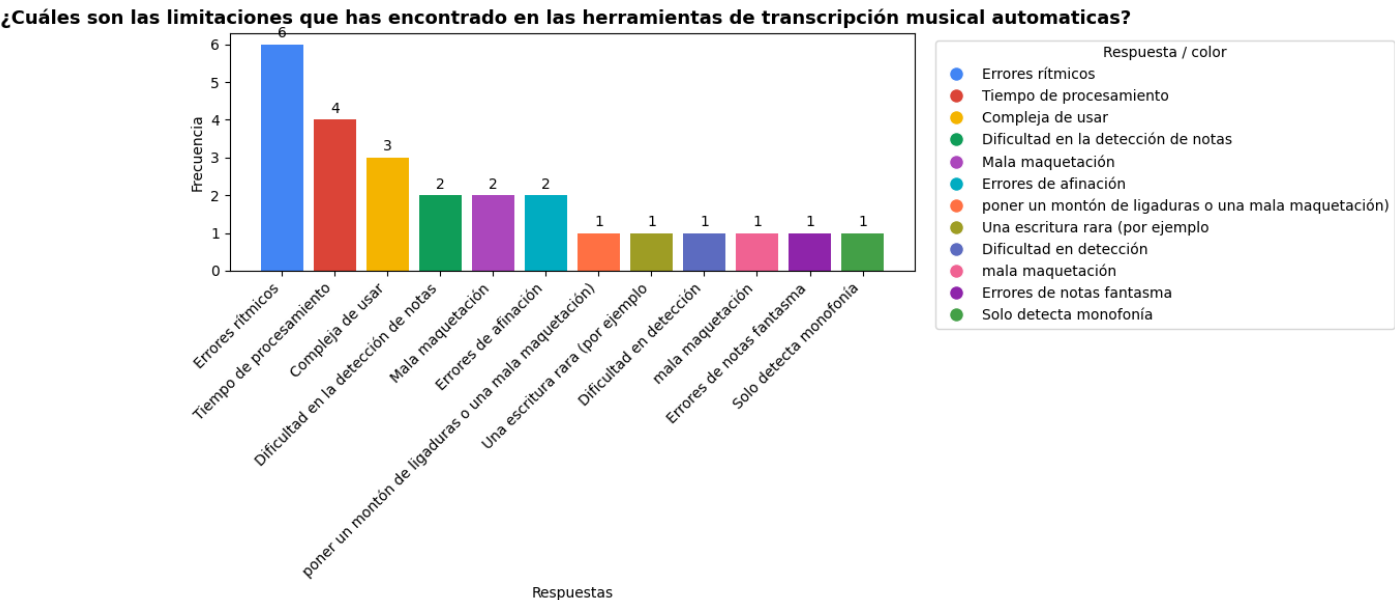


Ilustración 32: Limitaciones que ha notado el público objetivo

Tiempo de transcripción

Respecto al tiempo requerido para transcribir manualmente un minuto de música, el 47.3% de los encuestados indicó dedicar entre 10-20 minutos, mientras que el 25.7% reportó 20-40 minutos y el 21.6% más de 40 minutos, por resumen podemos indicar que el 94.6% de los usuarios invierte más de 10 minutos en transcribir un solo minuto de audio, lo cual representa una relación desproporcionada (10:1 a 40:1) y únicamente el 2.7% logra completar esta tarea en menos de 10 minutos, con estos resultados se justifica el desarrollo de Harmonic Score como herramienta de automatización para reducir drásticamente este tiempo.

¿Cuánto tiempo te toma transcribir 1 minuto de música?

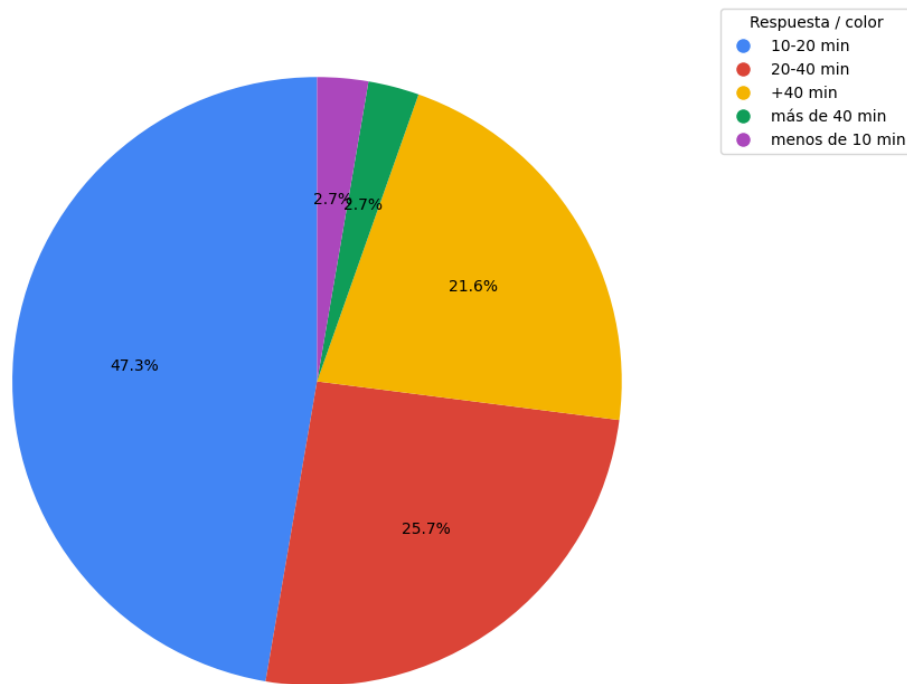


Ilustración 33: Promedio de tiempo que les toma transcribir una pieza musical

Con base en la evidencia recabada mediante las encuestas a la comunidad del INBAL, se concluye que el proyecto es operativamente factible, existe una necesidad clara de optimizar el tiempo dedicado a la transcripción musical (actualmente 94.6% de los usuarios invierte más de 10 minutos por minuto de audio), la comunidad objetivo cuenta con el acceso y las habilidades para utilizar una aplicación web, y los estándares de precisión propuestos aseguran que el resultado sea pedagógicamente útil. La integración de la herramienta en el flujo de estudio del INBAL no presenta barreras operativas insalvables

4.5.3 Factibilidad económica

La factibilidad económica determina si el proyecto Harmonic Score es financieramente justificable, comparando los costos de desarrollo con los beneficios esperados para la comunidad académica del INBAL, dado que se trata de un trabajo de carácter académico, el modelo de financiamiento se basa en la autofinanciación por parte de los integrantes, aprovechando herramientas de código abierto y recursos de cómputo personales para minimizar los costos operativos.

El presupuesto presentado a continuación cubre un periodo de 6 meses, correspondiente al ciclo completo del Trabajo Terminal I.

4.5.3.1 Recursos humanos

Nuestro equipo de desarrollo consta de tres científicos de datos con conocimiento en deep learning, procesamiento de audio y desarrollo web, debido a que el proyecto demanda habilidades técnicas de alta especialización como el modelado de redes neuronales y la ingeniería espectral, la inversión requerida refleja los estándares de un mercado altamente especializado, superando el alcance de un desarrollo de software tradicional [42].

Cargo	Costo mensual	Costo unitario 12 meses	Costo total 12 meses
Científico de Datos (Integrante 1)	\$30,000.00	\$360,000.00	\$360,000.00
Científico de Datos (Integrante 2)	\$30,000.00	\$360,000.00	\$360,000.00
Científico de Datos (Integrante 3)	\$30,000.00	\$360,000.00	\$360,000.00
Recursos humanos	\$90,000.000	\$1,080,000.00	\$1,080,000.00
Total		\$1,080,000.00	

Tabla 20: Presupuesto de recursos humanos

4.5.3.2 Recursos tecnológicos

En lo concerniente a los recursos tecnológicos, el proyecto se sustenta en herramientas de desarrollo modernas, accesibles y de bajo costo, se empleará un entorno de desarrollo integrado estándar, lenguajes de programación de código abierto y bibliotecas especializadas para el procesamiento de audio y aprendizaje profundo, para las pruebas se utilizarán equipos personales y entornos locales.

Cantidad	Descripción	Costo unitario	Total
Hardware			
1	Estación de trabajo principal (GPU NVIDIA T4, 32 GB RAM, SSD 1 TB) para entrenamiento de modelos	\$18,000.00	\$18,000.00
2	Laptops para pruebas de interoperabilidad y desarrollo visual	\$12,000.00	\$24,000.00
Software			

1	Servicio en en la nube para AWS	\$10,000.00	\$10,000.00
1	Servicio en la nube Google Forms	\$0	\$0
1	Licencias de software - Open Source	\$0	\$0
Despliegue			
1	Plan gratuito local host	\$0	\$0
Total			\$52,000.00

Tabla 21: Presupuesto de recursos tecnológicos

4.5.3.3 Recursos Operativos

Los recursos operativos engloban los costos de servicios básicos necesarios para sostener el ciclo de desarrollo, dado que el equipo trabaja de manera remota desde sus domicilios, no se contemplan viáticos para visitas al INBAL ni costos de transporte, los servicios de electricidad e internet se prorratean según el uso dedicado al desarrollo del proyecto.

Concepto	Descripción	Costo mensual	Costo total 12 meses
Servicios			
Energía eléctrica	Uso de estaciones de trabajo (\$150.00 × 3 integrantes)	\$450.00	\$5400.00
agua	Servicio básico domiciliario (\$150.00 × 3 integrantes)	\$450.00	\$5400.00
Comunicaciones			
Internet	Servicio de banda ancha (\$500.00 × 3 integrantes)	\$1500.00	\$18000.00
Total			\$28,800.00

Tabla 22: Presupuesto de recursos operativos

4.5.3.4 Resumen de presupuesto y fuente de financiamiento

La integración de las partidas financieras evaluadas en los apartados precedentes permite determinar el monto total estimado para el desarrollo del proyecto durante un ciclo de seis meses, de igual forma, se añade un fondo de reserva equivalente al 10% del subtotal, orientado a solventar contingencias no previstas.

Recurso	Costo total estimado a 12 meses
Recursos humanos	\$1,080,000.00
Recursos tecnológicos	\$52,000.00
Recursos operativos	\$28,800.00
Subtotal	\$1,160,800.00
Imprevistos (10%)	\$116,080.00
Total	\$1,276,880.00

Tabla 23: Resumen consolidado de presupuesto

4.5.4 Factibilidad legal

La factibilidad jurídica del proyecto constituye un requisito indispensable para garantizar que la aplicación web Harmonic Score opere dentro del marco normativo establecido por las autoridades competentes, con este análisis se verifica el cumplimiento de las disposiciones legales vigentes a nivel federal y local, asegurando que el tratamiento de datos [40], el uso de propiedad intelectual [39] y las colaboraciones institucionales no incurran en infracciones que pudieran comprometer la continuidad del sistema.

4.5.4.1 Marco legal aplicable

El desarrollo de una aplicación web orientada a la transcripción automática de música para la comunidad del El Instituto Nacional de Bellas Artes y Literatura debe realizarse bajo un marco legal claro y estructurado, este proyecto implica el uso de datos personales, procesamiento de archivos de audio musical, colaboración con una institución pública federal y distribución de software bajo licencias abiertas [41].

Ley/Reglamento	Artículos Relevantes	Aplicabilidad
Ley Federal de Protección de Datos Personales en Posesión de los Particulares (LFPDPPP) [40]	Arts. 6, 8, 16, 29	Se aplica directamente al manejo de datos de usuarios registrados (nombre, correo, contraseña). Establece obligaciones sobre consentimiento, transparencia y seguridad en el tratamiento de datos personales.

Ley Federal de Derechos de Autor [39]	Arts. 5, 26, 148	Regula el uso de audio musical con fines educativos y de investigación. El artículo 148 permite uso sin fines de lucro para fines académicos.
Ley Federal de Telecomunicaciones y Radiodifusión [42]	Arts. 189, 190	Regula el uso de redes de telecomunicaciones y servicios digitales, aplicable al despliegue de la aplicación web.
Ley General de Protección de Datos Personales en Posesión de Sujetos Obligados [43]	Arts. 1, 4, 7	Aplica por la colaboración con INBAL como institución pública federal en la validación operativa

Tabla 24: Normativa federal aplicable al proyecto

Normatividad Local

Instrumento Legal	Autoridad	Relevancia
Ley de Transparencia y Acceso a la Información Pública de la CDMX [44]	Congreso de la CDMX, Arts. 1, 6, 24	Fundamenta legalmente el uso de datos públicos abiertos para el desarrollo de aplicaciones con beneficio social.
Reglamento de la Ley de Protección de Datos Personales de la CDMX [45]	Jefatura de Gobierno	Establece criterios operativos para el tratamiento de datos personales en el ámbito local.
Lineamientos de Colaboración Académica IPN-INBAL [46]	IPN-INBAL	Marco de referencia para la colaboración no vinculante entre instituciones en proyectos de investigación.

Tabla 25: Normativa local aplicable al proyecto

Normas Técnicas y Estándares Internacionales

Norma/ Estándar	Descripción	Aplicabilidad
NOM-151-SCT-2-2016 [47]	Seguridad informática en sistemas de tráfico vehicular	Aunque está enfocada en carreteras, sus principios sobre integridad, disponibilidad y confidencialidad de datos son aplicables al almacenamiento temporal de audio y protección de credenciales de usuarios.

WCAG 2.1 AA [48]	Web Content Accessibility Guidelines	Normativa internacional de accesibilidad digital. La aplicación web sigue estos estándares para garantizar usabilidad por personas con discapacidad visual, auditiva o motriz.
NMX-I-110-NYCE-2019 [49]	Accesibilidad de Software	Es el estándar técnico mexicano de referencia para asegurar que la aplicación web sea usable por personas con discapacidad, alineándose con los principios de diseño universal.
Licencias de Software Abierto (MIT, Apache 2.0) [41]	Términos de uso de bibliotecas	Regula el uso de las herramientas de desarrollo (PyTorch, Librosa, Verovio, FastAPI). Todas las licencias permiten uso académico, modificación y distribución sin costos de licenciamiento.

Tabla 26: Normas técnicas y estándares internacionales

4.5.4.2 Análisis de cumplimiento normativo

Una vez identificado el marco jurídico aplicable, se verifica la alineación activa del proyecto con cada disposición legal, en este capítulo se detalla las medidas técnicas, administrativas y operativas implementadas en la aplicación web para garantizar la normatividad vigente, mitigando riesgos asociados al tratamiento de datos personales, propiedad intelectual y estándares de accesibilidad digital

Normativa	Requisito Legal	Estrategia de Cumplimiento en Harmonic Score
LFPDPPP [40]	Consentimiento informado y acceso a derechos ARCO.	Implementación de Aviso de Privacidad obligatorio antes del registro. Habilitación del correo icanop2100@alumno.ipn.mx para atender solicitudes de

		Acceso, Rectificación, Cancelación y Oposición.
Ley Federal de Derechos de Autor [39]	Uso de obras protegidas sin fines de lucro.	El proyecto se declara explícitamente educativo y sin fines de lucro (Art. 148). Los usuarios conservan la titularidad de sus grabaciones; el sistema solo procesa temporalmente.
Ley Federal de Telecomunicaciones [42]	Seguridad en transmisión de datos.	Uso de protocolos seguros (HTTPS) para la comunicación cliente-servidor. Los archivos de audio se eliminan automáticamente tras 1 mes de almacenamiento temporal.
WCAG 2.1 AA [48]	Accesibilidad web para personas con discapacidad.	Diseño de interfaz con contraste adecuado, etiquetas semánticas en HTML y navegación compatible con lectores de pantalla, asegurando inclusión digital.
Licencias Open Source (MIT/Apache) [41]	Atribución y distribución de código.	El repositorio público incluye archivo LICENSE y NOTICE con la atribución correspondiente a las bibliotecas de terceros (PyTorch, Librosa, Verovio).
Colaboración INBAL [46]	Protección de datos de participantes.	Consentimiento informado integrado en la encuesta y pruebas UAT. Los datos se anonimizan para análisis estadístico, sin vincular identidad pública con resultados individuales.

Tabla 27: Análisis de cumplimiento normativo

4.5.4.3 Requerimientos legales específicos

Derivado del marco jurídico aplicable y del análisis de cumplimiento normativo, se han identificado las obligaciones concretas que deben implementarse técnica y operativamente en la aplicación

web Harmonic Score, estas condiciones son indispensables para operar dentro de la legalidad y proteger tanto a los usuarios como al equipo de desarrollo.

Requerimiento Legal	Descripción	Estado de Implementación	Justificación Técnica
Aviso de Privacidad [40]	Documento que informa a los usuarios qué datos se recaban, con qué finalidad y cómo se protegen.	Implementado	Los usuarios deben aceptarlo electrónicamente antes de crear una cuenta. Cumple con el Art. 16 de la LFPDPPP.
Términos y Condiciones de Uso	Establece las reglas de uso de la aplicación, limitaciones de responsabilidad y propiedad intelectual del contenido generado.	Implementado	Define que el usuario conserva los derechos de sus grabaciones y que el proyecto es sin fines de lucro.
Consentimiento Informado [40]	Aceptación explícita del usuario para el procesamiento de sus datos personales y archivos de audio.	Implementado	Checkbox obligatorio durante el registro. Vinculado al Aviso de Privacidad y Términos de Uso.
Mecanismo de Derechos ARCO [40]	Canal para atender solicitudes de Acceso, Rectificación, Cancelación y Oposición de datos personales.	Implementado	Correo electrónico icanop2100@alumno.ipn.mx habilitado para recibir y procesar solicitudes en términos legales.
Política de Eliminación de Audio	Procedimiento automático para borrar archivos de audio tras un periodo determinado.	En desarrollo	Los archivos se almacenan máximo 1 mes. Posteriormente se eliminan automáticamente del servidor en la nube.
Atribución de Licencias [41]	Reconocimiento público de las bibliotecas de código abierto utilizadas en el desarrollo.	Implementado	Archivo LICENSE en el repositorio incluye todas las licencias (MIT, Apache 2.0, BSD) de PyTorch, Librosa, Verovio, FastAPI.

Medidas de Seguridad Técnicas	Protección de datos mediante cifrado, control de acceso y protocolos seguros.	En desarrollo	Implementación de HTTPS, hash para contraseñas (bcrypt/argon2) y separación de datos sensibles en la base de datos.
--------------------------------------	---	---------------	---

Tabla 28: Requerimientos legales específicos

4.5.4.4 Autorizaciones y permisos requeridos

Para la ejecución del proyecto Harmonic Score, es necesario identificar las instituciones y permisos legales requeridos, así como aquellos que no aplican debido a la naturaleza académica y sin fines de lucro del sistema, con esta clarificación se evita barreras regulatorias innecesarias y establece los alcances reales de la validación operativa.

Autorización/Permiso	¿Requerido?	Estatus	Justificación
Consentimiento informado de usuarios [40]	Sí	Obtenido	Los usuarios aceptan electrónicamente el Aviso de Privacidad y Términos de Uso antes de registrar su cuenta. La encuesta INBAL incluye cláusula de consentimiento académico.
Permiso de acceso a instalaciones INBAL [46]	Parcial	En trámite	La validación operativa se realiza bajo colaboración académica no vinculante. No requiere convenio formal entre instituciones para pruebas limitadas con estudiantes y docentes.
Autorización de uso de grabaciones musicales [39]	Sí	Obtenido	Los usuarios conservan la titularidad de sus grabaciones. El sistema solo procesa temporalmente (1 mes) con fines de transcripción educativa, sin distribución comercial.
Registro ante el INDAUTOR	No	N/A	El proyecto no comercializa obras musicales ni genera lucro. El Art. 148 de la Ley Federal de Derechos de Autor permite uso educativo sin registro adicional.
Licencias de software de terceros [41]	Sí	Obtenido	Todas las bibliotecas utilizadas (PyTorch, Librosa, Verovio, FastAPI) son de código abierto con licencias MIT, Apache 2.0 o

			BSD, permitiendo uso académico sin costo.
Aval institucional IPN	Sí	Obtenido	El proyecto está registrado como Trabajo Terminal de la Escuela Superior de Cómputo, bajo supervisión de director académico asignado.

Tabla 29: Autorizaciones y permisos requeridos

4.5.4.5 Recomendaciones de cumplimiento

Con base en el análisis de riesgos legales y requerimientos específicos identificados, se establecen las siguientes recomendaciones para garantizar el cumplimiento normativo durante todo el ciclo de vida del proyecto, con estas acciones podemos prevenir contingencias legales y fortalecer la postura jurídica de la aplicación web ante posibles reclamaciones.

Categoría	Recomendación	Prioridad	Plazo de Implementación
Documentación Legal	Mantener actualizados el Aviso de Privacidad y Términos de Uso conforme a cambios normativos.	Alta	Continuo
	Conservar evidencia del consentimiento informado de todos los usuarios registrados.	Alta	Por cada registro
	Documentar todas las solicitudes ARCO recibidas y su tiempo de respuesta.	Media	20 días hábiles por solicitud [40]
	Registrar las licencias de todas las bibliotecas de terceros en el repositorio público.	Alta	Al finalizar cada ciclo
Medidas Técnicas	Implementar cifrado de extremo a extremo para la transmisión de archivos de audio.	Alta	Ciclo 3 (Optimización)
	Configurar eliminación automática de audio tras 30 días de almacenamiento.	Alta	Ciclo 2 (Desarrollo)

	Realizar auditorías de seguridad trimestrales para identificar vulnerabilidades.	Media	Cada 3 meses [47]
	Separar datos sensibles (contraseñas) en tablas independientes con hash robusto.	Alta	Ciclo 2 (Desarrollo)
Prácticas Operativas	Capacitar al equipo en protección de datos personales y propiedad intelectual.	Media	Inicio de cada semestre
	Limitar el acceso a los archivos de audio únicamente al personal autorizado del equipo.	Alta	Inmediato
	Generar reportes mensuales de cumplimiento normativo para el director académico.	Media	Mensual
	Establecer protocolo de respuesta ante incidentes de seguridad o filtración de datos.	Alta	Ciclo 3 (Optimización) [47]

Tabla 30: Recomendaciones de cumplimiento legal para el proyecto Harmonic Score

4.5.4.6 Ventajas legales del proyecto

El proyecto Harmonic Score presenta beneficios jurídicos inherentes a su naturaleza académica, su arquitectura tecnológica y su modelo de operación, estas ventajas reducen significativamente la exposición a riesgos legales y facilitan el cumplimiento normativo.

Ventaja Legal	Descripción	Impacto en el Proyecto
Carácter educativo y sin fines de lucro [39]	El proyecto se desarrolla exclusivamente para fines académicos y de investigación, sin generar ingresos ni comercializar servicios.	Reduce el riesgo de demandas por derechos de autor al ampararse en el Art. 148 de la Ley Federal de Derechos de Autor. Exime de obligaciones fiscales comerciales.
Licencias de código abierto (MIT, Apache 2.0) [41]	Todo el stack tecnológico se basa en bibliotecas de libre distribución sin costos de licenciamiento ni restricciones de uso académico.	Elimina barreras económicas de entrada. Permite modificación y distribución del código sin violar propiedad intelectual de terceros.

Colaboración académica no vinculante con INBAL	La validación operativa se realiza sin convenio formal entre instituciones, limitándose a consentimiento informado de participantes.	Evita trámites burocráticos extensos. No genera obligaciones contractuales a largo plazo entre IPN e INBAL.
Almacenamiento temporal de audio (1 mes) [40]	Los archivos de audio se eliminan automáticamente tras un periodo definido, sin retención permanente.	Minimiza el riesgo de filtración de datos. Reduce la superficie de ataque y la exposición a solicitudes de acceso a información.
Consentimiento informado electrónico [40]	Los usuarios aceptan explícitamente los términos de uso y aviso de privacidad antes de registrar su cuenta.	Cumple con el Art. 8 de la LFPDPPP sobre consentimiento expreso. Protege al equipo de reclamaciones por uso no autorizado de datos.
Código fuente público y auditable	El repositorio del proyecto es accesible para verificación de licencias, seguridad y cumplimiento normativo.	Promueve transparencia académica. Facilita la revisión por pares y la detección temprana de vulnerabilidades legales o técnicas.
Responsabilidad limitada en términos de uso [40]	Los términos y condiciones establecen que la herramienta es de apoyo académico, no un producto profesional certificado.	Reduce la exposición a reclamaciones por errores de transcripción o uso indebido de las partituras generadas.

Tabla 31: Ventajas legales del proyecto

4.5.4.7 Cronograma de cumplimiento legal

El cumplimiento normativo del proyecto Harmonic Score se integra de manera transversal en cada ciclo de la metodología en espiral, garantizando que las actividades legales se ejecuten de forma continua y paralela a las actividades técnicas.

Ciclo	Actividades Legales	Tiempo Total	Costo Estimado
Ciclo Planificación	1: Redacción de Aviso de Privacidad, Términos de Uso y consentimiento informado para encuesta INBAL. Revisión legal por director académico.	3 semanas	\$3,500.00
Ciclo Desarrollo	2: Implementación de aceptación electrónica de privacidad, configuración de almacenamiento temporal (30 días), cifrado HTTPS y habilitación de canal ARCO. Asesoría legal externa para validación LFPDPPP.	4 semanas	\$8,000.00
Ciclo Optimización	3: Configuración de eliminación automática de audio, hash para contraseñas y auditoría de seguridad y cumplimiento normativo. Generación de reporte final de cumplimiento legal.	3 semanas	\$14,500.00
Post-lanzamiento	Monitoreo continuo de solicitudes ARCO y actualización anual de Aviso de Privacidad y Términos de Uso.	Continuo	\$21,000.00/anual

Tabla 32: Cronograma de cumplimiento legal del proyecto Harmonic Score

4.5.5 Conclusiones de factibilidad

Derivado del análisis normativo, se determina que el proyecto Harmonic Score cuenta con viabilidad jurídica con un nivel de riesgo bajo a medio, la aplicación web se apega a las disposiciones federales y locales vigentes, entre las que destacan la Ley Federal de Protección de Datos Personales en Posesión de los Particulares [40], la Ley Federal de Derechos de Autor [39] y los estándares internacionales de accesibilidad digital [48], también cuenta con la utilización de licencias de software libre [41], evitando obstáculos de propiedad intelectual sobre el código, mientras que la naturaleza educativa y sin fines de lucro del proyecto permite acogerse al artículo 148 de la Ley Federal de Derechos de Autor [39] para el tratamiento de audio musical con propósitos académicos.

Las contingencias legales identificadas, vinculadas principalmente a la salvaguarda de datos personales [40] y la utilización de grabaciones musicales, disponen de estrategias de mitigación documentadas y factibles de implementar, el resguardo temporal de audio, la supresión automática posterior y la aceptación informada electrónica disminuyen de manera considerable la exposición a sanciones.

Por lo expuesto, no se identifican impedimentos regulatorios que obstaculicen la operación del proyecto dentro del ámbito académico del Instituto Politécnico Nacional.

4.5.6 Presupuesto legal

Para garantizar el cumplimiento normativo descrito en los apartados anteriores, se ha destinado una partida presupuestal específica que cubre las actividades legales esenciales del proyecto. Este presupuesto se ajusta a las actividades críticas que requieren especialización jurídica externa y asignando las demás al equipo de desarrollo sin costo adicional.

Concepto	Descripción	Costo Estimado
Asesoría legal inicial	Revisión de Aviso de Privacidad y Términos de Uso por abogado especializado en protección de datos y propiedad intelectual.	\$3,500.00
Asesoría LFPDPPP	Validación externa del cumplimiento de la Ley Federal de Protección de Datos Personales durante el Ciclo 2.	\$8,000.00
Auditoría de cumplimiento	Revisión integral de seguridad, privacidad y normas aplicables al finalizar el Ciclo 3.	\$12,000.00
Elaboración de reporte legal	Documentación final de cumplimiento normativo para archivo académico del proyecto.	\$2,500.00
Monitoreo post-lanzamiento	Atención de solicitudes ARCO y actualización anual de documentación legal (costo anual).	\$21,000.00
Contingencias legales	Fondo de reserva para imprevistos jurídicos no contemplados (10% del subtotal).	\$4,700.00
Total		\$51,700.00

Tabla 33: Presupuesto legal del proyecto Harmonic Score

Capítulo 5. Diseño

5.1 Diagrama de arquitectura del software, sistema, aplicación

El diagrama de arquitectura presenta de manera gráfica y estructurada cómo se planea construir la aplicación web. Para este proyecto, se ha adoptado el modelo C4, que permite visualizar la arquitectura del software en diferentes niveles de abstracción, facilitando la comprensión tanto para perfiles técnicos como no técnicos. Los tres primeros niveles (Contexto, Contenedores y Componentes) son esenciales para comunicar el diseño de la aplicación web de manera clara y coherente.

5.1.1 Diagrama de contexto

5.1.2 Diagrama de contenedores

5.1.3 Diagrama de componentes

5.2 Casos de uso

5.2.1 Diagrama de casos de uso

5.2.2 Actores

5.2.3 Especificación de casos de uso

5.3 Diagramas de secuencia

5.4 Diagramas de código

5.4.1 Diagramas de clases

5.4.2 Modelo de datos

5.5 Mockups

CAPITULO 6?

Referencias

- [1] Benetos, E., Dixon, S., Duan, Z., & Ewert, S. (2019). Automatic music transcription: An overview. *IEEE Signal Processing Magazine*, 36(1), 20–30. <https://doi.org/10.1109/MSP.2018.2871307>
- [2] Pardo, B., & Birmingham, W. P. (2002). An overview of automatic music transcription. In *Proceedings of the 2002 International Computer Music Conference* (pp. 468–471). International Computer Music Association.
- [3] Allen, J. B. (1977). Short time spectral analysis, synthesis, and modification by discrete Fourier transform. *IEEE Transactions on Acoustics, Speech, and Signal Processing*, 25(3), 235–238. <https://doi.org/10.1109/TASSP.1977.1162950>
- [4] Klapuri, A. (2006). Multiple fundamental frequency estimation by summing harmonic amplitudes. In *Proceedings of the 7th International Society for Music Information Retrieval Conference* (pp. 216–221). ISMIR.
- [5] Sigtia, S., Benetos, E., & Dixon, S. (2016). An end-to-end neural network for polyphonic piano music transcription. *IEEE/ACM Transactions on Audio, Speech, and Language Processing*, 24(5), 927–939. <https://doi.org/10.1109/TASLP.2016.2538139>
- [6] Hawthorne, C., Elsen, E., Song, J., Roberts, A., Simon, I., Raffel, C., ... & Eck, D. (2018). Onsets and frames: Dual-objective piano transcription. In *Proceedings of the 19th International Society for Music Information Retrieval Conference* (pp. 1–8). ISMIR.
- [7] Gardner, J., Simon, I., Manilow, E., Hawthorne, C., & Engel, J. (2021). MT3: Multi-task multitrack music transcription. In *Proceedings of the International Conference on Learning Representations*. ICLR.
- [8] Celemony Software. (2025). *Melodyne 5 editor: Technical specifications and pricing*. <https://www.celemony.com/en/products/melodyne/what-is-melodyne>
- [9] Klangio GmbH. (2025). *Klangio: AI music transcription service – Features and pricing*. <https://www.klangio.ai>
- [10] Lunaverus LLC. (2025). *AnthemScore: Automatic music transcription software – Documentation*. <https://www.lunaverus.com>
- [11] Wu, E., Setlur, V., & Bharadwaj, S. (2021). Open-source tools for music transcription research. In *Proceedings of the 22nd International Society for Music Information Retrieval Conference* (pp. 652–659). ISMIR.

- [12] Gardner, J. R., Défossez, A., Copet, J., Synnaeve, G., & Oquab, M. (2023). YourMT3-YPTF-MoE-M: Universal music transcription with mixture-of-experts routing. *arXiv preprint arXiv:2305.14678*.
- [13] Liaw, R., Liang, E., Nishihara, R., Moritz, P., Gonzalez, J. E., & Stoica, I. (2018). Tune: A research platform for distributed model selection and training. *arXiv preprint arXiv:1807.05118*.
- [14] Pugin, L., & Schreiber, H. (2020). Verovio: A library for music notation engraving. In *Proceedings of the 21st International Society for Music Information Retrieval Conference* (pp. 578–584). ISMIR.
- [15] Brown, J. C., & Puckette, M. S. (1992). An efficient algorithm for the calculation of a constant Q transform. *The Journal of the Acoustical Society of America*, 92(5), 2698–2701. <https://doi.org/10.1121/1.404385>
- [16] Schörkhuber, C., Klapuri, A., Holzapfel, A., & Widmer, G. (2014). A Matlab toolbox for efficient perfect reconstruction time-frequency transforms with log-frequency resolution. In *Proceedings of the 5th International Workshop on Cognitive Information Processing* (pp. 1–6). IEEE.
- [17] McFee, B., Raffel, C., Liang, D., Ellis, D. P., McVicar, M., Battenberg, E., ... & Weyde, T. (2015). librosa: Audio and music signal analysis in python. In *Proceedings of the 14th Python in Science Conference* (pp. 18–25). SciPy.
- [18] Gururani, S., & Lerch, A. (2024). Spectrogram reconstruction as self-supervised pretraining for robust music transcription. *arXiv preprint arXiv:2406.15249*.
- [19] Vaswani, A., Shazeer, N., Parmar, N., Uszkoreit, J., Jones, L., Gomez, A. N., ... & Polosukhin, I. (2017). Attention is all you need. In *Advances in Neural Information Processing Systems* (Vol. 30, pp. 5998–6008). NeurIPS.
- [20] Cho, K., & Saul, L. K. (2024). Mixture-of-experts architectures for sequential audio processing. *Transactions on Machine Learning Research*, 5(2), 112–129. <https://doi.org/10.48550/arXiv.2403.08271>
- [21] Spotify. (2023). *basic-pitch: Real-time polyphonic music transcription* [Computer software]. GitHub. <https://github.com/spotify/basic-pitch>
- [22] Copet, J., Laroche, F., Méneret, L., Cossu, A., Zeghidour, N., Synnaeve, G., & Défossez, A. (2024). MusicFM: A foundation model for music generation and analysis. *arXiv preprint arXiv:2402.10344*.
- [23] Hawthorne, C., Stasyuk, A., Roberts, A., Simon, I., Huang, C. Z., Dieleman, S., ... & Eck, D. (2019). Enabling factorized piano music modeling and generation with the MAESTRO dataset. In *International Conference on Learning Representations*. ICLR.

- [24] Bittner, R. M., Salamon, J., Tierney, M., Mauch, M., Cannam, C., & Bello, J. P. (2014). MedleyDB: A multitrack dataset for annotation-intensive MIR research. In *Proceedings of the 15th International Society for Music Information Retrieval Conference* (pp. 155–160). ISMIR.
- [25] Choi, K., Fazekas, G., & Sandler, M. (2017). Data augmentation for music transcription using musical knowledge. In *Proceedings of the IEEE International Conference on Acoustics, Speech and Signal Processing* (pp. 316–320). IEEE. <https://doi.org/10.1109/ICASSP.2017.7952169>
- [26] Collins, N., & Laney, R. (2018). Rhythmic quantization for music transcription using probabilistic models. *Journal of New Music Research*, 47(3), 247–265. <https://doi.org/10.1080/09298215.2018.1452943>
- [27] Bosch, J. J., Janer, J., Fuhrmann, F., & Herrera, P. (2012). A comparison of sound segregation techniques for predominant melody extraction in music. In *Proceedings of the 13th International Society for Music Information Retrieval Conference* (pp. 55–60). ISMIR.
- [28] Good, M. (2021). *MusicXML 4.0 specification*. MakeMusic Inc. <https://www.musicxml.com>
- [29] Cuthbert, M. S., & Ariza, C. (2010). music21: A toolkit for computer-aided musicology and symbolic music data. In *Proceedings of the 11th International Society for Music Information Retrieval Conference* (pp. 637–642). ISMIR.
- [30] Rajbhandari, S., Ruwase, O., Rasley, J., Smith, S., & He, Y. (2022). DeepSpeed-MoE: Training trillion-parameter models using moe with billions of parameters. *arXiv preprint arXiv:2201.06797*.
- [31] Tiangolo, S. (2023). *FastAPI: Modern, fast (high-performance) web framework for building APIs with Python*. <https://fastapi.tiangolo.com>
- [32] Schmidhuber, J. (2015). Deep learning in neural networks: An overview. *Neural Networks*, 61, 85–117. <https://doi.org/10.1016/j.neunet.2014.09.003>
- [33] Wavesurfer.js Contributors. (2024). *Wavesurfer.js: Interactive audio waveform visualization* [JavaScript library]. <https://wavesurfer.xyz>
- [34] Howler.js Contributors. (2024). *Howler.js: Modern audio library for the web* [JavaScript library]. <https://howlerjs.com>
- [35] McFee, B., McVicar, M., Faronbi, D., Roman, I., Gover, M., Balke, S., Seyfarth, S., Malek, A., Raffel, C., Lostanlen, V., van Niekirk, B., Lee, D., Cwitkowitz, F., Zalkow, F., Nieto, O., Ellis, D., Mason, J., Lee, K., Steers, B., . . . Sùdholt, D. (2025). *librosa/librosa: 0.11.0* (0.11.0). Zenodo. <https://doi.org/10.5281/zenodo.15006942>

- [36] Stevens, S. S., Volkman, J., & Newman, E. B. (1937). A scale for the measurement of the psychological magnitude pitch. *Journal of the Acoustical Society of America*, 8(3), 185–190. <https://doi.org/10.1121/1.1915893>
- [37] Shouran, M., & Elgamli, E. (2020). Design and implementation of Butterworth filter. *Int. J. Innov. Res. Sci. Eng. Technol*, 9(9), 7975-7983.
- [38] McNeil, A. J., Frey, R., & Embrechts, P. (2015). *Quantitative risk management: concepts, techniques and tools-revised edition*. Princeton university press.
- [39] Cámara de Diputados del H. Congreso de la Unión. (2024). Ley Federal de Derechos de Autor. Última reforma publicada en el Diario Oficial de la Federación el 15 de marzo de 2024. http://www.diputados.gob.mx/LeyesBiblio/pdf/134_240315.pdf
- [40] Cámara de Diputados del H. Congreso de la Unión. (2024). Ley Federal de Protección de Datos Personales en Posesión de los Particulares. Última reforma publicada en el Diario Oficial de la Federación el 20 de enero de 2024. http://www.diputados.gob.mx/LeyesBiblio/pdf/LFPDPPP_200124.pdf
- [41] Open Source Initiative. (2024). The Open Source Definition. <https://opensource.org/osd>
- [42] Cámara de Diputados del H. Congreso de la Unión. (2024). Ley Federal de Telecomunicaciones y Radiodifusión. Última reforma publicada en el Diario Oficial de la Federación el 10 de febrero de 2024. http://www.diputados.gob.mx/LeyesBiblio/pdf/LFTR_100224.pdf
- [43] Cámara de Diputados del H. Congreso de la Unión. (2024). Ley General de Protección de Datos Personales en Posesión de Sujetos Obligados. Última reforma publicada en el Diario Oficial de la Federación el 20 de enero de 2024. http://www.diputados.gob.mx/LeyesBiblio/pdf/LGPDPSO_200124.pdf
- [44] Congreso de la Ciudad de México. (2024). Ley de Transparencia, Acceso a la Información Pública, Rendición de Cuentas y Participación Ciudadana de la Ciudad de México. Gaceta Oficial de la Ciudad de México. http://data.consejeria.cdmx.gob.mx/portal_old/uploads/gacetas/726_20240209.pdf
- [45] Jefatura de Gobierno de la Ciudad de México. (2024). Reglamento de la Ley de Protección de Datos Personales en Posesión de Sujetos Obligados de la Ciudad de México. Gaceta Oficial de la Ciudad de México. http://data.consejeria.cdmx.gob.mx/portal_old/uploads/gacetas/reglamentos/2024_01_15.pdf
- [46] Instituto Politécnico Nacional & Instituto Nacional de Bellas Artes y Literatura. (2024). Lineamientos para la colaboración académica no vinculante en proyectos de investigación educativa. Documento interno IPN-INBAL.

- [47] Secretaría de Comunicaciones y Transportes. (2016). NOM-151-SCT-2-2016: Seguridad informática en sistemas de tráfico vehicular. Diario Oficial de la Federación. http://www.dof.gob.mx/nota_detalle.php?codigo=5464118&fecha=23/12/2016
- [48] World Wide Web Consortium (W3C). (2018). Web Content Accessibility Guidelines (WCAG) 2.1. <https://www.w3.org/TR/WCAG21/>
- [49] Organismo Nacional de Normalización y Certificación de la Industria Electrónica, Telecomunicaciones y Tecnologías de la Información (NYCE). (2019). NMX-I-110-NYCE-2019: Accesibilidad de software. Norma Mexicana.

Anexos